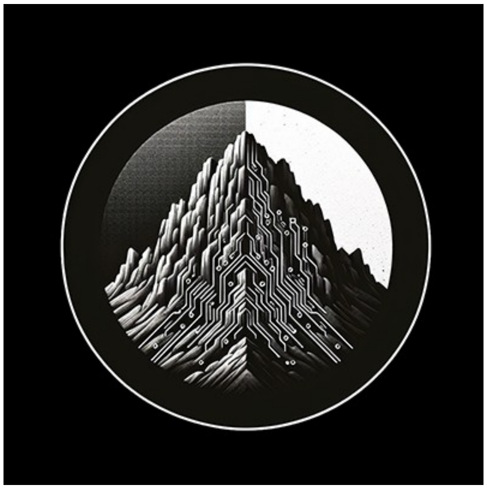


RTL Design Sherpa

APB HPET Micro-Architecture Specification 1.0

January 4, 2026

Table of Contents

1 Hpet Mas Index........16

2 APB HPET........17

2.1 Overview........17

2.1.1 Introduction........17

2.1.2 Figure 1.1: APB HPET Block Diagram........17

2.1.3 Key Features........18

2.1.4 Applications........18

2.2 Timing........19

2.2.1 Timing Accuracy........19

2.2.2 Register Access Latency........19

2.3 Design Notes........19

2.3.1 Design Philosophy........19

2.3.2 Comparison with IA-PC HPET........19

2.3.3 Resource Utilization (Post-Synthesis Estimates)........20

2.3.4 Scalability........20

2.3.5 Development Status........20

2.4 Related Modules........21

2.5 Testing........21

2.5.1 Verification Status........21

2.6 Navigation........22

3 APB HPET Architecture........23

3.1 Overview........23

3.1.1 High-Level Block Diagram....23
3.1.2 Module Hierarchy....23
3.2 Parameters....24
3.2.1 Compile-Time Parameters....24
3.3 Ports....25
3.3.1 Interface Summary....25
3.4 Functional Description....25
3.4.1 Data Flow....25
3.4.2 Clock Domains....27
3.4.3 Figure 1.2: Clocking with CDC_ENABLE = 0....28
3.4.4 Figure 1.3: Clocking with CDC_ENABLE = 1....29
3.4.5 Reset Domains....29
3.5 Usage Example....31
3.5.1 Configuration Examples....31
3.6 Design Notes....31
3.6.1 Per-Timer Data Bus Architecture....31
3.7 Navigation....32
4 APB HPET Clocks and Reset....33
4.1 Overview....33
4.1.1 Single Clock Domain (CDC_ENABLE = 0)....33
4.1.2 Dual Clock Domains (CDC_ENABLE = 1)....33
4.2 Functional Description....33
4.2.1 Clock Specifications....33
4.2.2 Reset Domains....34

    - 4.2.3 Clock Domain Crossing Details......36
    - 4.2.4 Counter Read Atomicity......37
    - 4.2.5 Clock Gating Considerations......38
  - 4.3 Timing......38
    - 4.3.1 Reset Coordination......38
    - 4.3.2 Figure 1.4: Reset Sequence with CDC_ENABLE = 1......39
    - 4.3.3 Timing Constraints......39
  - 4.4 Navigation......40
- 5 APB HPET Acronyms and Terminology......41
  - 5.1 Overview......41
  - 5.2 Functional Description......42
    - 5.2.1 Terminology......42
    - 5.2.2 Register Field Access Types......42
  - 5.3 Design Notes......43
    - 5.3.1 Signal Naming Conventions......43
    - 5.3.2 Common Abbreviations in Code......43
  - 5.4 Navigation......44
- 6 APB HPET References......45
  - 6.1 References......45
    - 6.1.1 External Standards and Specifications......45
    - 6.1.2 Internal Project Documentation......45
    - 6.1.3 Repository-Wide Documentation......46
    - 6.1.4 Related RTL Components......46
    - 6.1.5 Design Tools......47

6.1.6 Industry Best Practices References......47

6.1.7 Version Control and Issue Tracking......47

6.1.8 Related Projects......48

6.2 Navigation......48

7 APB HPET Blocks......49

7.1 Overview......49

7.1.1 Block Hierarchy......49

7.2 Functional Description......49

7.2.1 Module Responsibilities......49

7.2.2 Data Flow Overview......51

7.2.3 Clock Domain Organization......52

7.2.4 Module Communication......52

7.3 Waveforms......53

7.3.1 Timer Operation Waveforms......53

7.3.2 Waveform 2.1: HPET Config Write......54

7.3.3 Waveform 2.2: HPET Counter Read......54

7.3.4 Waveform 2.3: HPET One-Shot Timer Fire......55

7.3.5 Waveform 2.4: HPET Periodic Timer Fire......55

7.3.6 Waveform 2.5: HPET Interrupt Clear......56

7.3.7 Waveform 2.6: HPET Timer Setup......56

7.3.8 Waveform 2.7: HPET CDC Crossing......57

7.4 Design Notes......57

7.4.1 Resource Allocation......57

7.5 Testing......58

7.5.1 Integration Checklist......58

7.6 Navigation......59

8 hpet_core......60

8.1 Overview......60

8.1.1 Figure 2.8: HPET Core Block Diagram......60

8.1.2 Key Features......61

8.2 Parameters......61

8.3 Ports......61

8.3.1 Clock and Reset......61

8.3.2 Configuration Interface (from hpet_config_regs)......62

8.3.3 Status Interface (to hpet_config_regs)......64

8.3.4 Interrupt Interface......64

8.4 Functional Description......65

8.4.1 Per-Timer State Machine......65

8.4.2 Figure 2.9: Timer FSM......66

8.4.3 Main Counter Logic......68

8.4.4 Figure 2.10: Main Counter Start Timing......69

8.4.5 Timer Comparator Logic......69

8.4.6 Waveform 2.11: Comparator Match Behavior......72

8.4.7 Timer Fire Logic......72

8.4.8 Figure 2.12: Timer Fire Timing......77

8.4.9 Interrupt Generation......78

8.4.10 Waveform 2.13: Interrupt Generation......78

8.4.11 Periodic Mode Details......79

8.4.12 Waveform 2.14: Periodic Timer Operation....79
8.4.13 Figure 2.15: Periodic Mode Timing Example....80
8.4.14 One-Shot Mode Details....80
8.4.15 Waveform 2.16: One-Shot Mode Operation....81
8.4.16 Figure 2.17: One-Shot Mode Timing Example....82
8.5 Design Notes....82
8.5.1 Resource Utilization....82
8.6 Navigation....82
9 hpet_config_regs....83
9.1 Overview....83
9.1.1 Figure 2.18: HPET Config Registers Block Diagram....83
9.1.2 Key Responsibilities....83
9.2 Parameters....84
9.3 Ports....84
9.3.1 Clock and Reset....84
9.3.2 Command/Response Interface (from APB Slave)....85
9.3.3 HPET Core Interface (to hpet_core.sv)....85
9.4 Functional Description....88
9.4.1 Internal Architecture....88
9.4.2 Mapping Logic Details....89
9.4.3 Figure 2.19: Counter Write Strobe Timing....91
9.4.4 Figure 2.20: HPET_STATUS Mirror and W1C Clear Timing....94
9.4.5 Register-to-Core Signal Summary....94
9.5 Design Notes....94

9.5.1 Resource Utilization........ 94
9.6 Navigation........ 95
10 hpet_regs........ 96
10.1 Overview........ 96
10.2 Parameters........ 96
10.3 Ports........ 97
10.3.1 Clock and Reset........ 97
10.3.2 CPU Interface (Passthrough Protocol)........ 97
10.3.3 Hardware Interface (Structs)........ 98
10.4 Functional Description........ 100
10.4.1 Register Implementation........ 100
10.4.2 Field Access Semantics........ 105
10.4.3 SystemRDL Specification........ 106
10.5 Usage Example........ 109
10.5.1 Regeneration Procedure........ 109
10.6 Navigation........ 109
11 apb4_hpet........ 110
11.1 Overview........ 110
11.1.1 Figure 2.21: APB HPET Top-Level Block Diagram........ 110
11.1.2 Module Hierarchy........ 111
11.2 Parameters........ 111
11.3 Ports........ 112
11.3.1 Clock and Reset - Dual Domain........ 112
11.3.2 APB4 Slave Interface (Low Frequency Domain)........ 113

11.3.3 Timer Interrupt Outputs (High Frequency Domain)....113
11.4 Functional Description....114
11.4.1 Internal Signal Interfaces....114
11.4.2 APB Slave Conditional Generation....115
11.4.3 Clock Domain Assignment....118
11.5 Usage Example....118
11.5.1 Example 1: Synchronous Configuration (CDC_ENABLE=0)....118
11.5.2 Example 2: Asynchronous Configuration (CDC_ENABLE=1)....119
11.6 Design Notes....120
11.6.1 Resource Utilization Summary....120
11.7 Testing....120
11.7.1 Verification Checklist....120
11.8 Navigation....121
12 APB HPET FSM Summary....122
12.1 Overview....122
12.1.1 FSM Inventory....122
12.2 Functional Description....122
12.2.1 1. APB Slave Protocol FSM....122
12.2.2 Figure 2.22: APB Access Timing....123
12.2.3 2. APB Slave CDC (async-FIFO crossing)....123
12.2.4 3. HPET Core Per-Timer FSM....124
12.2.5 Figure 2.23: One-Shot Mode State Timeline....126
12.2.6 Figure 2.24: Periodic Mode State Timeline....127
12.2.7 FSM Interaction Summary....127

12.3 Design Notes....128

12.3.1 State Machine Design Patterns....128

12.4 Testing....130

12.4.1 FSM Verification Considerations....130

13 Claude Code Guide: Retro Legacy Blocks....132

13.1 Quick Context....132

13.2 Critical Rules for All Blocks....132

13.2.1 Rule #0.1: Reset Macro Standards - MANDATORY FOR ALL BLOCKS.132

13.2.2 Rule #0.2: FPGA Synthesis Attributes - MANDATORY....132

13.2.3 Rule #0.3: Testbench Architecture - MANDATORY SEPARATION....133

13.2.4 Rule #0.4: Test Hierarchy - 3 Levels Required....133

13.2.5 Rule #0.5: Register Generation - Use PeakRDL....133

13.3 Block Development Workflow....134

13.3.1 Adding a New Block....134

13.4 HPET-Specific Guidance....134

13.4.1 HPET Quick Reference....134

13.4.2 Critical HPET Rules....135

13.4.3 HPET Common Issues....136

13.5 Common User Questions....137

13.5.1 Q: “Which blocks are implemented?”....137

13.5.2 Q: “How do I integrate a block in my design?”....138

13.5.3 Q: “What’s the RLB wrapper goal?”....138

13.5.4 Q: “Why ‘Retro Legacy Blocks’?”....139

13.6 Quick Commands....139

13.7 PDF Generation Location....139

14 APB HPET Register Map....141

14.1 Overview....141

14.1.1 Figure 5.1: APB HPET Block Diagram....142

14.2 Functional Description....142

14.2.1 Register Address Map Summary....142

14.2.2 Global Register Descriptions....145

14.2.3 Per-Timer Register Descriptions....150

14.2.4 Timer Operation Modes....155

14.2.5 Figure 5.2: One-Shot Timer Operation....155

14.2.6 Figure 5.3: Periodic Timer Operation....156

14.2.7 Register Access Conventions....158

14.2.8 Figure 5.4: HPET Register Memory Map....160

14.3 Waveforms....160

14.3.1 Waveform 5.1: APB Write Timer Config....161

14.3.2 Waveform 5.2: APB Read Counter....161

14.3.3 Waveform 5.3: Interrupt W1C Sequence....161

14.3.4 Waveform 5.4: Timer Setup Sequence....162

14.4 Usage Example....163

14.4.1 Initialization Sequence....163

14.4.2 Figure 5.5: Software Initialization Flow....163

14.4.3 Reading Capabilities....164

14.4.4 Interrupt Handling....164

14.4.5 Figure 5.6: Interrupt Handling Flow....164

14.5 Related Modules....165
    14.5.1 Additional Diagrams....165

List of Figures

Figure 1.1: APB HPET Block Diagram....17
Figure 1.2: Clocking with CDC_ENABLE = 0....28
Figure 1.3: Clocking with CDC_ENABLE = 1....29
Figure 1.4: Reset Sequence with CDC_ENABLE = 1....39
Figure 2.8: HPET Core Block Diagram....60
Figure 2.9: Timer FSM....66
Figure 2.10: Main Counter Start Timing....69
Figure 2.12: Timer Fire Timing....77
Figure 2.15: Periodic Mode Timing Example....80
Figure 2.17: One-Shot Mode Timing Example....82
Figure 2.18: HPET Config Registers Block Diagram....83
Figure 2.19: Counter Write Strobe Timing....91
Figure 2.20: HPET_STATUS Mirror and W1C Clear Timing....94
Figure 2.21: APB HPET Top-Level Block Diagram....110
Figure 2.22: APB Access Timing....123
Figure 2.23: One-Shot Mode State Timeline....126
Figure 2.24: Periodic Mode State Timeline....127
Figure 5.1: APB HPET Block Diagram....142
Figure 5.2: One-Shot Timer Operation....155
Figure 5.3: Periodic Timer Operation....156
Figure 5.4: HPET Register Memory Map....160
Figure 5.5: Software Initialization Flow....163
Figure 5.6: Interrupt Handling Flow....164

List of Tables

No tables in this document.

List of Waveforms

Waveform 2.1: HPET Config Write....54
Waveform 2.2: HPET Counter Read....54
Waveform 2.3: HPET One-Shot Timer Fire....55
Waveform 2.4: HPET Periodic Timer Fire....55
Waveform 2.5: HPET Interrupt Clear....56
Waveform 2.6: HPET Timer Setup....56
Waveform 2.7: HPET CDC Crossing....57
Waveform 2.11: Comparator Match Behavior....72
Waveform 2.13: Interrupt Generation....78
Waveform 2.14: Periodic Timer Operation....79
Waveform 2.16: One-Shot Mode Operation....81
Waveform 5.1: APB Write Timer Config....161
Waveform 5.2: APB Read Counter....161
Waveform 5.3: Interrupt W1C Sequence....161
Waveform 5.4: Timer Setup Sequence....162

1 Hpet Mas Index

Generated: 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

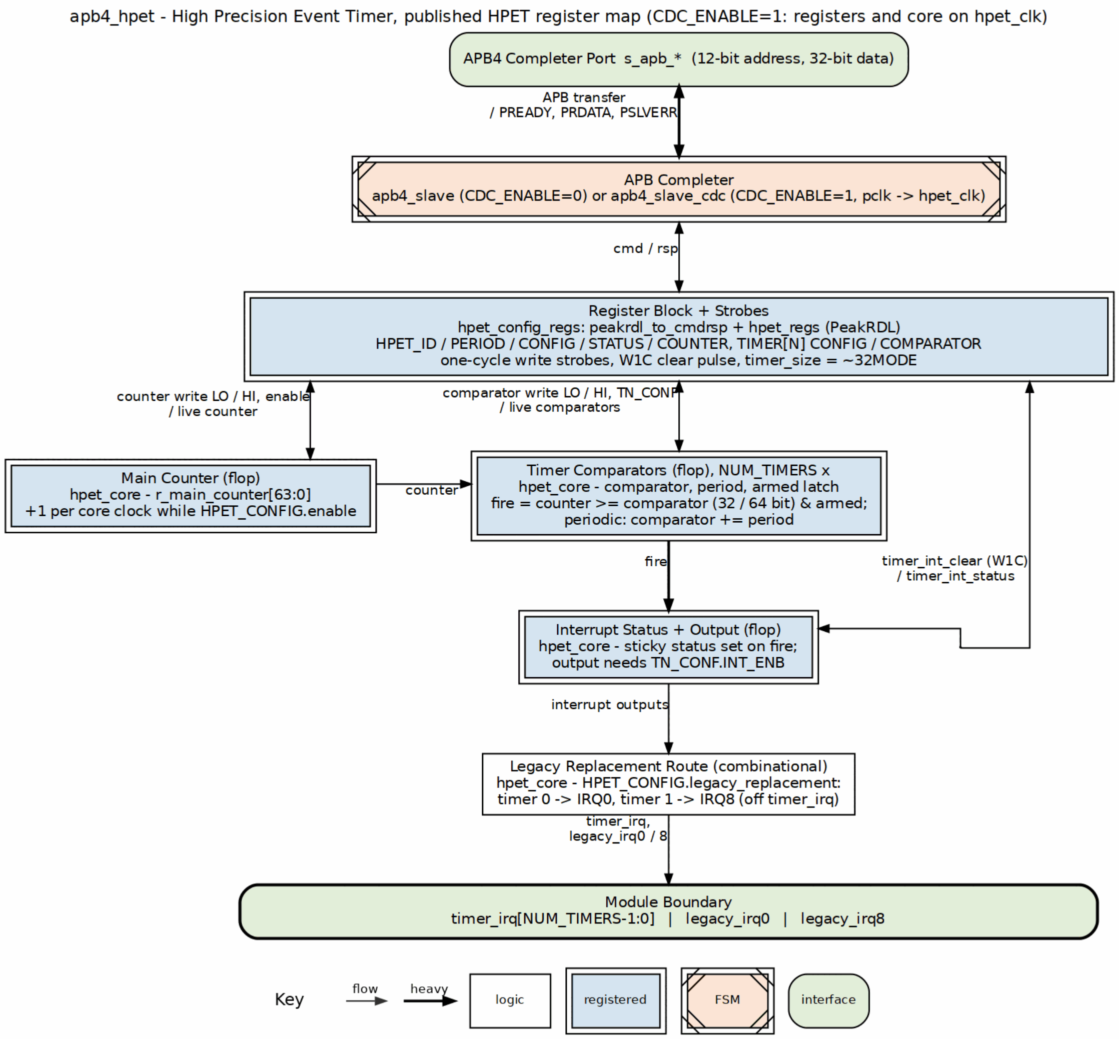
2 APB HPET

2.1 Overview

2.1.1 Introduction

The APB High Precision Event Timer (HPET) is a configurable multi-timer peripheral built for precise timing and event generation in embedded systems. You get up to 8 independent hardware timers with one-shot and periodic modes, an APB register interface, and optional clock domain crossing for the cases where your timer clock doesn't come from the bus clock.

2.1.2 Figure 1.1: APB HPET Block Diagram



apb4_hpet - High Precision Event Timer, published HPET register map (CDC_ENABLE=1: registers and core on hpet_clk)
APB4 Completer Port s_apb_* (12-bit address, 32-bit data)
APB transfer / PREADY, PRDATA, PSLVERR
APB Completer
apb4_slave (CDC_ENABLE=0) or apb4_slave_cdc (CDC_ENABLE=1, pclk -> hpet_clk)
cmd / rsp
Register Block + Strobes
hpet_config_regs: peakrdl_to_cmdrsp + hpet_regs (PeakRDL)
HPET_ID / PERIOD / CONFIG / STATUS / COUNTER, TIMER[N] CONFIG / COMPARATOR
one-cycle write strobes, W1C clear pulse, timer_size = ~32MODE
counter write LO / HI, enable / live counter
comparator write LO / HI, TN_CONF / live comparators
Main Counter (flop)
hpet_core - r_main_counter[63:0]
+1 per core clock while HPET_CONFIG.enable
counter
Timer Comparators (flop), NUM_TIMERS x
hpet_core - comparator, period, armed latch
fire = counter >= comparator (32 / 64 bit) & armed;
periodic: comparator += period
fire
timer_int_clear (W1C) / timer_int_status
Interrupt Status + Output (flop)
hpet_core - sticky status set on fire;
output needs TN_CONF.INT_ENB
interrupt outputs
Legacy Replacement Route (combinational)
hpet_core - HPET_CONFIG.legacy_replacement:
timer 0 -> IRQ0, timer 1 -> IRQ8 (off timer_irq)
timer_irq, legacy_irq0 / 8
Module Boundary
timer_irq[NUM_TIMERS-1:0] | legacy_irq0 | legacy_irq8
Key
flow
heavy
logic
registered
FSM
interface

APB HPET Block Diagram

2.1.3 Key Features

- **Multiple Independent Timers**: 2, 3, or 8 configurable hardware timers per instance
- **64-bit Main Counter**: High-resolution timestamp with configurable clock source
- **64-bit Comparators**: Long-duration timing support (up to 2^64-1 clock cycles)
- **Dual Operating Modes**:
  - **One-shot**: Timer fires once when counter reaches comparator value
  - **Periodic**: Timer auto-reloads and fires repeatedly at fixed intervals
- **Dynamic Mode Switching**: Switch between one-shot and periodic modes without reset
- **APB Interface**: Standard AMBA APB4 compliant register interface
- **Clock Domain Crossing**: Optional CDC support for independent APB and timer clocks
- **PeakRDL Integration**: Register map generated from SystemRDL specification
- **Per-Timer Write Data Buses**: Dedicated data paths prevent timer corruption
- **Individual Interrupts**: Separate interrupt output per timer with W1C status clearing

2.1.4 Applications

Real-Time Operating Systems: - System tick generation for RTOS schedulers - Watchdog timer implementation - Task deadline enforcement - Periodic interrupt generation

Performance Profiling: - High-resolution timestamp source - Code execution timing - Cache miss profiling - Inter-event timing measurement

Multi-Rate Timing: - Multiple simultaneous timing domains - Independent periodic tasks - Asynchronous event generation - Programmable pulse generation

Industrial Control: - PWM generation base timer - Motor control timing - Sensor sampling intervals - Control loop timing

---

2.2 Timing

2.2.1 Timing Accuracy

- Counter increment: Every HPET clock cycle (deterministic)
- Timer fire latency: 1 HPET clock cycle from counter match
- Interrupt assertion: Registered, one HPET clock after the fire event (`timer_irq` is a flop in `hpet_core`, gated by `timer_int_enable` sampled at fire time)

2.2.2 Register Access Latency

- No CDC: 2 APB clock cycles (APB protocol minimum)
- With CDC: 4-6 APB clock cycles (handshake synchronization overhead)

2.3 Design Notes

2.3.1 Design Philosophy

Every block makes trade-offs; here's where this one landed.

Configurability: The HPET component prioritizes configurability to support diverse use cases. Timer count and CDC enablement are parameterizable at synthesis time, so you can tailor an instance for your application without touching the RTL. (The `VENDOR_ID`/`REVISION_ID` parameters reach HPET_ID through the register block's hardware interface. `vendor_id` is the published spec's full 16 bits at GCAP_ID[31:16], so a PCI-style value such as 0x4747 reads back complete; `rev_id` is 8 bits at GCAP_ID[7:0]. See Chapter 5.)

Reliability: Extensive testing (5/6 configurations at 100% pass rate) validates core functionality. The design includes per-timer data buses to prevent corruption. (Note: the register block never raises PSLVERR – unmapped addresses alias or read 0.)

Standards Compliance: - **APB Protocol**: Full AMBA APB4 specification compliance - **PeakRDL**: Industry-standard SystemRDL for register generation - **Reset Convention**: Consistent active-low asynchronous reset (`presetn`)

Reusability: Clean module hierarchy and well-defined interfaces enable easy integration. Optional CDC support allows flexible clock domain configuration without design changes.

2.3.2 Comparison with IA-PC HPET

The APB HPET draws architectural inspiration from the IA-PC HPET specification but is **not** a drop-in replacement. Key differences:

Feature	IA-PC HPET	APB HPET
Interface	Memory-mapped	AMBA APB4
Timer Count	Up to 256	2, 3, or 8 (configurable)
FSB Delivery	Supported	Not supported
Legacy Replacement	PIT/RTC emulation	Supported: timer 0 -> IRQ0, timer 1 -> IRQ8 (`leg_rt_cap` = 1)
Counter Size	64-bit mandatory	64-bit
Comparator Size	64-bit or 32-bit	64-bit or 32-bit (per-timer `timer_size`)
Clock Source	10 MHz minimum	User-configurable
Vendor ID	16-bit, read from capability	16-bit at GCAP_ID[31:16], from the `VENDOR_ID` parameter

Retained Concepts: - 64-bit free-running counter - One-shot and periodic timer modes - Write-1-to-clear interrupt status - Capability register for hardware discovery

Removed Features: - FSB interrupt delivery (use dedicated IRQ signals) - Legacy PIT/RTC replacement (not needed in modern designs) - Main counter period configuration (use clock divider instead)

2.3.3 Resource Utilization (Post-Synthesis Estimates)

- 2-timer (no CDC): ~500 LUTs, ~300 flip-flops
- 3-timer (no CDC): ~650 LUTs, ~400 flip-flops
- 8-timer (with CDC): ~1200 LUTs, ~800 flip-flops

2.3.4 Scalability

The design scales linearly with timer count. Each additional timer adds approximately: - 150 LUTs (comparator, control logic, interrupt generation) - 100 flip-flops (timer state, configuration registers) - Minimal timing impact (no critical path through timer array)

2.3.5 Development Status

Status: RTL Functional - the issue #46 register-side defects were fixed on 2026-09-08; see the index

Completed Features: - One-shot timer mode - Periodic timer mode - Timer mode switching - 64-bit counter read/write - 64-bit comparators - Multiple independent timers - Clock domain crossing (optional) - PeakRDL register generation - Per-timer write data buses (corruption fix) - Comprehensive test suite (3-level hierarchy) - Live comparator readback – reads return hpet_core's working comparator, including the periodic advance (RLB/hpet TASK-002) - Published-spec register interface: spec offsets (GEN_CONF 0x010, GINTR_STA 0x020, MAIN_CNT 0x0F0, timer N at 0x100+0x20N with the comparator at +0x08), GCAP_ID and TN_CONF field positions, a 16-bit vendor field, and HPET_PERIOD reporting the counter tick period in femtoseconds (RLB/hpet TASK-006)

Outstanding Items: - 8-timer stress test timeout (minor, likely test configuration)

Future Enhancements (Not Planned): - FSB interrupt delivery (use dedicated IRQ signals) - Legacy mode emulation (not needed in modern designs) - 64-bit atomic counter reads – NOT a shortfall and not planned: the counter is 64 bits behind a 32-bit APB, so two reads are structural, and a non-atomic read is the IA-PC HPET behaviour, and a deliberate design ruling here. Software uses the hi/lo/hi retry loop; see ch01_overview/03_clocks_and_reset.md

2.4 Related Modules

Related Documentation: - `../../PRD.md` - Product Requirements Document - `../../CLAUDE.md` - AI integration guide - `vault/tracking/projects/components/legacy-ip/retro_legacy_blocks/hpet/` - HPET task tracking (`vault/tracking/projects/components/legacy-ip/retro_legacy_blocks/` for cross-block) - `../IMPLEMENTATION_STATUS.md` - Test results and validation status

2.5 Testing

2.5.1 Verification Status

Test Coverage: 5 of 6 configurations achieve 100% test pass rate

Configuration	Basic	Medium	Full	Overall
2-timer v4747 (no CDC)	4/4	5/5	3/3	12/12
3-timer AMD-like (no CDC)	4/4	5/5	3/3	12/12
8-timer custom (no CDC)	4/4	5/5	2/3 (known issue)	11/12 (known issue)
2-timer v4747 (CDC)	4/4	5/5	3/3	12/12
3-timer AMD-like (CDC)	4/4	5/5	3/3	12/12
8-timer custom (CDC)	4/4	5/5	3/3	12/12

Known Issue: 8-timer non-CDC “All Timers Stress” test has timeout issue (minor, likely test configuration)

Test Levels: - **Basic (4 tests)**: Register access, enable/disable, counter operation, interrupt generation - **Medium (5 tests)**: Periodic mode, multiple timers, 64-bit features, mode switching - **Full (3 tests)**: All timers stress, CDC validation, edge case coverage

See: `IMPLEMENTATION_STATUS.md` for complete test results

2.6 Navigation

This specification document is organized as follows:

- **Chapter 1 (this chapter)**: Overview, features, applications
- **Chapter 2**: Detailed block specifications (hpet_core, config_regs, PeakRDL integration)
- **Chapter 3**: Interface specifications (planned, not yet written)
- **Chapter 4**: Programming model (planned, not yet written)
- **Chapter 5**: Register definitions (address map, field descriptions)

Next: Chapter 1.2 - Architecture

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

3 APB HPET Architecture

3.1 Overview

Three blocks do the work: an APB slave (with or without CDC), a register wrapper that adapts the PeakRDL register file to the core, and the timer core itself. Figure 1.1 in the overview shows how they fit together.

3.1.1 High-Level Block Diagram

The whole unit is Figure 1.1 (APB HPET Block Diagram). Read top to bottom:

1. **APB slave.** `apb4_slave`, or `apb4_slave_cdc` when `CDC_ENABLE = 1`, turns APB transfers into the cmd/rsp handshake. With CDC the crossing is inside this block (Figure 1.3); there is no separate CDC stage after it.
2. **Register wrapper.** `hpet_config_regs` adapts cmd/rsp to the PeakRDL `hpet_regs` file through `peakrdl_to_cmdrsp`, and turns register writes into the one-cycle strobes and per-timer configuration the core consumes.
3. **Timer core.** `hpet_core` holds the 64-bit main counter and, per timer, the comparator, the period and the fire logic. Its outputs are `timer_irq[NUM_TIMERS-1:0]` and, for the LegacyReplacement Route, `legacy_irq0` and `legacy_irq8`.

3.1.2 Module Hierarchy

```
apb4_hpet (Top Level)
+-- apb4_slave (OR apb4_slave_cdc if CDC_ENABLE=1)
|   +-- APB protocol handling
|   +-- Read/write transaction management
|   +-- Optional clock domain crossing
|
+-- hpet_config_regs (Register Wrapper)
|   +-- hpet_regs (PeakRDL Generated)
|   |   +-- HPET_CONFIG register
|   |   +-- HPET_STATUS register (W1C)
|   |   +-- HPET_COUNTER_LO/HI registers
|   |   +-- HPET_ID register (RO, capabilities/identification)
|   |   +-- TIMER[i]_* registers (per-timer)
|   |
|   +-- Write-strobe alignment (swmod rising edge, one per 32-bit
half)
|   +-- HPET_STATUS W1C decode (per-bit clear mask from the write)
|   +-- Per-timer data bus routing (corruption prevention)
|
+-- hpet_core (Timer Logic)
+-- 64-bit main counter (r_main_counter)
+-- Timer array [NUM_TIMERS-1:0]
|   +-- 64-bit comparator (r_timer_comparator[i])
|   +-- 64-bit period storage (r_timer_period[i])
|   +-- Timer control FSM (one-shot vs periodic)
|   +-- Armed latch and fire pulse (r_timer_armed[i])
|   +-- Next-epoch hold bit (r_comp_next_epoch[i])
+-- Counter increment logic
+-- Comparator match detection
+-- Interrupt generation
```

3.2 Parameters

3.2.1 Compile-Time Parameters

Parameter	Type	Default	Range	Description
`NUM_TIMERS`	int	2	2, 3, 8	Number of independent timers
`VENDOR_ID`	int	1	0-65535	Drives HPET_ID[31:16] – GCAP_ID's full 16-bit vendor field, so 0x4747 reads back complete
`REVISION_ID`	int	1	0-255	Drives HPET_ID[7:0] – GCAP_ID's revision field
`COUNTER_CLK_PERIOD_FS`	int	10000000	non-zero, <= 0x05F5E100	Femtoseconds per main-counter tick, published at HPET_PERIOD (0x004). MUST match the clock the counter ticks on: `CDC_ENABLE[0] ? hpet_clk : pclk`
`CDC_ENABLE`	int	0	0, 1	Enable clock domain crossing

Parameter	Type	Default	Range	Description
`USE_JOHNSON`	int	0	0, 1	CDC FIFO pointer encoding (0 = Gray, 1 = Johnson)

The APB address bus is fixed at 12 bits and the data bus at 32 bits; they are not parameters.

Address-map constants (values from the generated decode in hpet_regs.sv – there are no such localparams in the RTL):

- Timer stride: 0x20 bytes per timer
- Timer register base: 0x100

3.3 Ports

3.3.1 Interface Summary

APB Interface: Standard AMBA APB4 - Address width: Fixed 12-bit (4KB space) - Data width: Fixed 32-bit - Protocol: APB4 (with PREADY support)

HPET Clock Interface: Separate timer clock domain - Independent from APB clock (if CDC enabled) - Free-running 64-bit counter - Configurable clock frequency

Interrupt Interface: Per-timer dedicated outputs - `timer_irq[NUM_TIMERS-1:0]` - Active-high interrupt signals - Registered output from core state (one hpet_clk after fire) - W1C clearing via HPET_STATUS register

See: Chapter 3 - Interface Specifications for detailed signal descriptions

3.4 Functional Description

3.4.1 Data Flow

3.4.1.1 Write Transaction Flow (APB -> HPET Core)

A write walks Figure 1.1 top to bottom:

1. **APB master** issues the write.
2. **APB slave** (`apb4_slave`, or `apb4_slave_cdc`) handles the protocol and, with `CDC_ENABLE = 1`, carries the command across to `hpet_clk` (Figure 1.3).
3. **`hpet_regs` (PeakRDL)** decodes the register, updates the field, and raises the software-access flags (`swacc`, `swmod`).

4. **`hpet_config_regs`**:
    - detects the rising edge of each `swmod` level, aligned to the written value;
    - generates the per-half write strobes (`counter_write_lo/hi`, `timer_comp_write_lo/hi[i]`);
    - decodes an HPET_STATUS write into a per-bit clear mask;
    - routes the per-timer data buses.
5. **`hpet_core`**:
    - loads one counter half (HPET_COUNTER_LO or _HI write);
    - loads one comparator half, re-arming only if the counter is halted (TIMER_COMPARATOR write);
    - updates control (TIMER_CONFIG write);
    - clears the status bits written with 1 (HPET_STATUS write).

3.4.1.2 Read Transaction Flow (HPET Core -> APB)

A read walks the same path and returns:

1. **APB master** issues the read.
2. **APB slave** handles the protocol; with CDC the response crosses back to `pclk`.
3. **`hpet_regs` (PeakRDL)** decodes the address and multiplexes the read data from the hardware interface (hwif).
4. **`hpet_config_regs`** connects the `hpet_core` signals to the hwif read ports.
5. **`hpet_core`** provides the counter value, the timer configuration and the status flags.
6. **APB slave** returns PRDATA to the master.

3.4.1.3 Timer Operation Flow

1. **Counter increment**, every tick of the core clock: `r_main_counter <= r_main_counter + 1`.
2. **Comparator match detection**, for each timer i: `timer_match[i] = (r_main_counter >= r_timer_comparator[i]) && !r_comp_next_epoch[i]`, at the compare width. The hold bit forces the match off while the comparator sits in the counter's next epoch.
3. **Timer fire logic.** A match fires only while the timer is ARMED, and firing clears the armed latch. The latch sets when the match falls (or is held off by

the epoch bit), when a comparator half is written on a STOPPED timer, or when a periodic catch-up step lands at or ahead of the counter.

- **One-shot mode:** fire once on the armed match, then stay un-armed until the comparator is written with the counter halted, or the match falls (comparator written above the counter, or the counter written back).
- **Periodic mode:** fire on the armed match and auto-increment the comparator, `r_timer_comparator[i] <= r_timer_comparator[i] + r_timer_period[i]`; the match drops, which re-arms the timer.
  - If the advanced comparator is still at or below the counter, keep advancing one boundary per cycle WITHOUT firing until it is at or ahead, then re-arm. Missed periods are skipped, never burst; at period 1 the step is to counter + 1.
  - If an advance carries out of the compare width, set the next-epoch hold bit: no fire and no catch-up until the counter wraps at that width, or software writes the comparator or the counter half the compare reads, or changes `timer_size`.

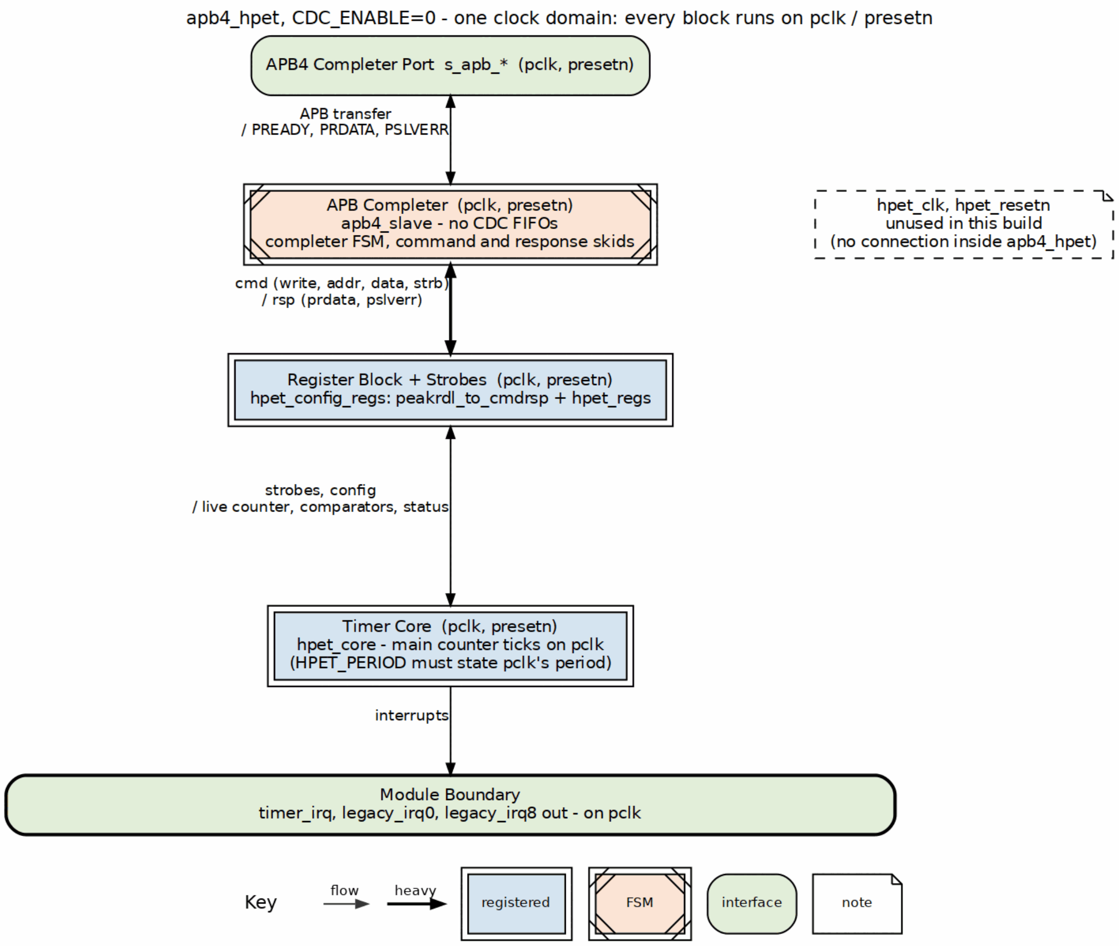
4. **Interrupt status update.** `r_interrupt_status[i] <= 1` in `hpet_core`, sticky until software clears it by W1C; HPET_STATUS[i] mirrors it.
5. **Interrupt output.** `timer_irq[i]` is a flop in `hpet_core`, set one core clock after the fire when `timer_int_enable[i]` was set at fire time, and cleared by the same W1C that clears the status. It is core state, not a combinational copy of HPET_STATUS.

3.4.2 Clock Domains

`CDC_ENABLE` selects one of two clockings at build time. In both, the register block and the timer core share a single clock and reset; only the APB front-end changes.

Synchronous mode (CDC_ENABLE = 0). The front-end is the plain `apb4_slave`, and the register block and core run on `pclk` / `presetn`. `hpet_clk` and `hpet_resetn` are not connected to anything, so the main counter ticks on `pclk` and `HPET_PERIOD` must state `pclk`'s period.

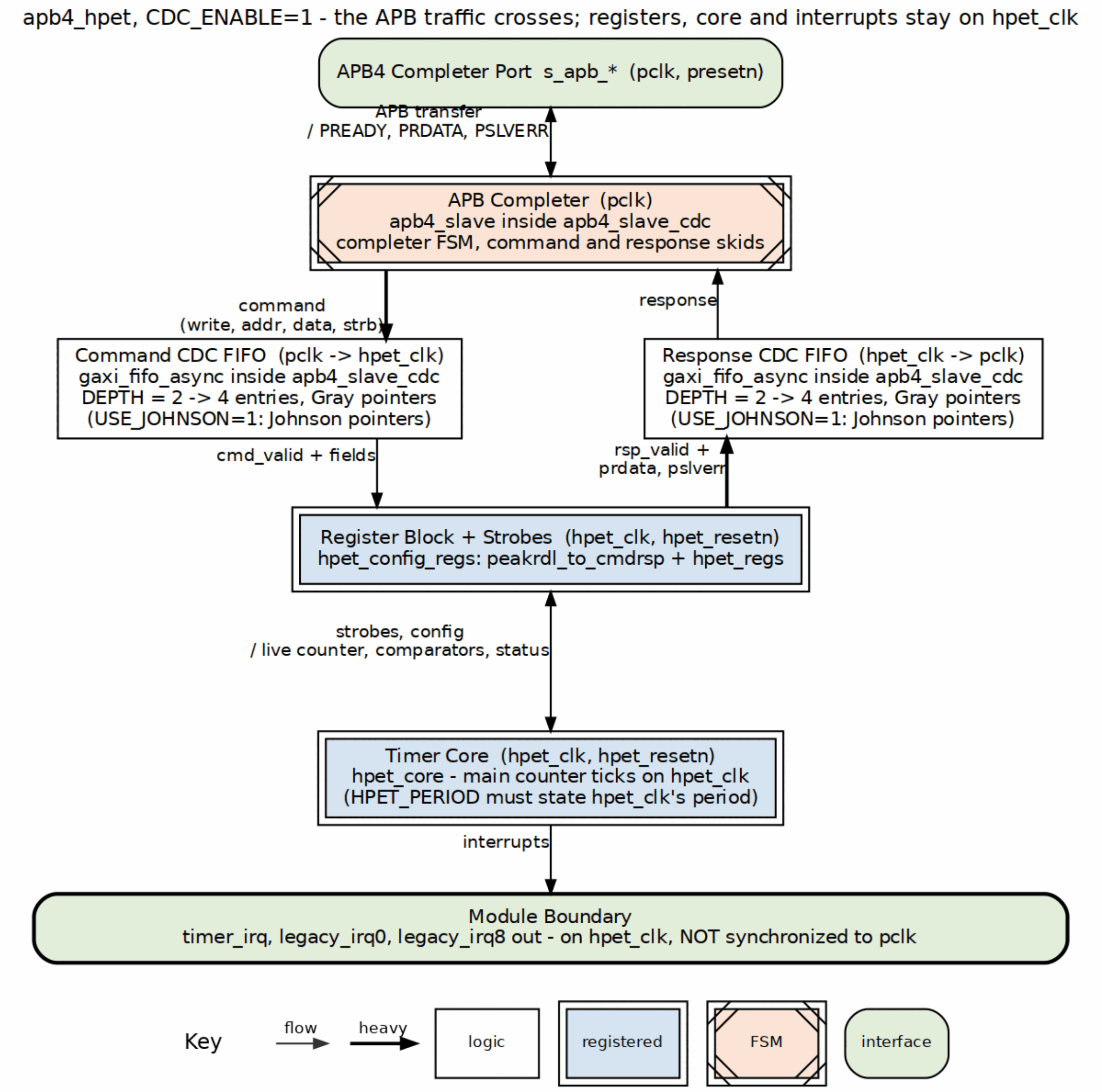
3.4.3 Figure 1.2: Clocking with CDC_ENABLE = 0


apb4_hpet, CDC_ENABLE=0 - one clock domain: every block runs on pclk / presetn
APB4 Completer Port s_apb_* (pclk, presetn)
APB transfer / PREADY, PRDATA, PSLVERR
APB Completer (pclk, presetn) apb4_slave - no CDC FIFOs completer FSM, command and response skids
hpet_clk, hpet_resetn unused in this build (no connection inside apb4_hpet)
cmd (write, addr, data, strb) / rsp (prdata, pslverr)
Register Block + Strobes (pclk, presetn) hpet_config_regs: peakrdl_to_cmdrsp + hpet_regs
strobes, config / live counter, comparators, status
Timer Core (pclk, presetn) hpet_core - main counter ticks on pclk (HPET_PERIOD must state pclk's period)
interrupts
Module Boundary timer_irq, legacy_irq0, legacy_irq8 out - on pclk
Key
flow
heavy
registered
FSM
interface
note

Clocking with CDC_ENABLE = 0

Asynchronous mode (CDC_ENABLE = 1). The front-end is `apb4_slave_cdc`: the APB completer stays on `pclk`, and the command and response cross through a pair of asynchronous FIFOs. The register block, core and interrupt outputs run on `hpet_clk` / `hpet_resetn`; the interrupts are not synchronized back to `pclk`.

3.4.4 Figure 1.3: Clocking with CDC_ENABLE = 1


apb4_hpet, CDC_ENABLE=1 - the APB traffic crosses; registers, core and interrupts stay on hpet_clk
APB4 Completer Port s_apb_* (pclk, presetn)
APB transfer
/ PREADY, PRDATA, PSLVERR
APB Completer (pclk)
apb4_slave inside apb4_slave_cdc
completer FSM, command and response skids
command
(write, addr, data, strb)
response
Command CDC FIFO (pclk -> hpet_clk)
gaxi_fifo_async inside apb4_slave_cdc
DEPTH = 2 -> 4 entries, Gray pointers
(USE_JOHNSON=1: Johnson pointers)
Response CDC FIFO (hpet_clk -> pclk)
gaxi_fifo_async inside apb4_slave_cdc
DEPTH = 2 -> 4 entries, Gray pointers
(USE_JOHNSON=1: Johnson pointers)
cmd_valid + fields
rsp_valid +
prdata, pslverr
Register Block + Strobes (hpet_clk, hpet_resetn)
hpet_config_regs: peakrdl_to_cmdrsp + hpet_regs
strobes, config
/ live counter, comparators, status
Timer Core (hpet_clk, hpet_resetn)
hpet_core - main counter ticks on hpet_clk
(HPET_PERIOD must state hpet_clk's period)
interrupts
Module Boundary
timer_irq, legacy_irq0, legacy_irq8 out - on hpet_clk, NOT synchronized to pclk
Key
flow
heavy
logic
registered
FSM
interface

Clocking with CDC_ENABLE = 1

3.4.5 Reset Domains

Reset Signals: - `presetn` - APB reset (active-low, asynchronous) - `hpet_resetn` - HPET reset (active-low, asynchronous)

Reset Behavior:

The register file and the timer core always share ONE clock/reset domain, selected by `CDC_ENABLE`: with `CDC_ENABLE=0` both run on `pclk`/`presetn`; with `CDC_ENABLE=1` both run on `hpet_clk`/`hpet_resetn` and only the APB front-end stays on `pclk`. There is no configuration in which the registers and the core reset from different domains.

Signal	Reset Value	Notes
`r_main_counter`	64’h0	Counter reset to zero
`r_timer_comparator[i]`	64’h0	Comparators reset to zero
`r_timer_period[i]`	64’h0	Period storage reset
`HPET_CONFIG`	32’h0	Global enable cleared
`HPET_STATUS`	8’h0	Mirror of the core’s `r_interrupt_status`, which also resets to 0
`TIMER[i]_CONFIG`	32’h0	All timers disabled

Reset Sequence:

```
// Illustrative only. clk/rst_n are the selected domain:
// CDC_ENABLE=0 -> pclk/presetn; CDC_ENABLE=1 -> hpet_clk/hpet_resetn.
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        // Register file
        HPET_CONFIG <= '0;
        HPET_STATUS <= '0;   // mirror field; the state lives in the
core
        for (int i = 0; i < NUM_TIMERS; i++) begin
            TIMER_CONFIG[i] <= '0;
        end
        // Timer core
        r_main_counter <= 64'h0;
        for (int i = 0; i < NUM_TIMERS; i++) begin
            r_timer_comparator[i] <= 64'h0;
            r_timer_period[i] <= 64'h0;
            r_timer_armed[i] <= 1'b1;      // counter == comparator ==
0 already matches
            r_interrupt_status[i] <= 1'b0;
        end
    end
end
```

CDC Reset Coordination: When CDC is enabled, both reset signals must be properly synchronized and coordinated to prevent metastability and ensure clean initialization.

3.5 Usage Example

3.5.1 Configuration Examples

2-Timer Configuration (synchronous clocks):

```
apb4_hpet #(
    .NUM_TIMERS(2),
    .CDC_ENABLE(0)
) u_hpet_2t (...);
```

8-Timer Configuration with CDC:

```
apb4_hpet #(
    .NUM_TIMERS(8),
    .CDC_ENABLE(1)          // Asynchronous clocks
) u_hpet_8t (...);
```

`VENDOR_ID`/`REVISION_ID` reach HPET_ID through the register block's hardware interface – the generated block is built once for the maximum configuration, so nothing about it is per-instance. `vendor_id` is the published spec's full 16 bits at GCAP_ID[31:16] and `rev_id` is 8 bits at GCAP_ID[7:0], so a PCI-style `VENDOR_ID(16'h4747)` reads back complete (see Chapter 5). The same path carries `COUNTER_CLK_PERIOD_FS` to HPET_PERIOD, which is GCAP_ID[63:32].

3.6 Design Notes

3.6.1 Per-Timer Data Bus Architecture

Problem: Initial implementation had timer corruption due to shared data bus

Root Cause:

```
// WRONG: Shared data bus for all timers
wire [63:0] timer_comp_wdata;  // Single 64-bit bus

// Multiple timers try to sample from same bus
always_ff @(posedge hpet_clk) begin
    if (timer_comp_write[0]) r_timer_comparator[0] <=
timer_comp_wdata;
    if (timer_comp_write[1]) r_timer_comparator[1] <=
timer_comp_wdata;
```

```
    if (timer_comp_write[2]) r_timer_comparator[2] <=
timer_comp_wdata;
    // If write strobes overlap, wrong timer gets wrong data!
end
```

Solution: Per-timer dedicated data buses

```
// CORRECT: Dedicated data bus per timer
wire [63:0] timer_comp_wdata [NUM_TIMERS-1:0];  // Array of 64-bit
buses

// Each timer has dedicated data path
always_ff @(posedge hpet_clk) begin
    if (timer_comp_write[0]) r_timer_comparator[0] <=
timer_comp_wdata[0];
    if (timer_comp_write[1]) r_timer_comparator[1] <=
timer_comp_wdata[1];
    if (timer_comp_write[2]) r_timer_comparator[2] <=
timer_comp_wdata[2];
    // Each timer reads from its own dedicated bus - no corruption
possible
end
```

Implementation in hpet_config_regs.sv:

```
// Dedicated data buses prevent corruption
assign timer_comp_wdata[0] = {hwif.timer0_comparator_hi.value,
                              hwif.timer0_comparator_lo.value};
assign timer_comp_wdata[1] = {hwif.timer1_comparator_hi.value,
                              hwif.timer1_comparator_lo.value};
assign timer_comp_wdata[2] = {hwif.timer2_comparator_hi.value,
                              hwif.timer2_comparator_lo.value};
// ... one data bus per timer
```

Verification: All timer corruption issues resolved after per-timer bus implementation

---

3.7 Navigation

Next: Chapter 1.3 - Clocks and Reset

---

4 APB HPET Clocks and Reset

4.1 Overview

The APB HPET operates in one or two clock domains depending on CDC configuration. Pick the mode that matches your system; the trade is latency versus clock freedom.

4.1.1 Single Clock Domain (CDC_ENABLE = 0)

Configuration: - `pclk = hpet_clk` (same physical clock) - No clock domain crossing required - Lower latency (2 APB clock cycles for register access) - Simpler timing analysis

Use Cases: - System where APB and timer clocks are guaranteed synchronous - Resource-constrained designs (CDC overhead not needed) - Minimal latency requirements

4.1.2 Dual Clock Domains (CDC_ENABLE = 1)

Configuration: - `pclk` and `hpet_clk` are independent, asynchronous clocks - CDC synchronization required - Higher latency (4-6 APB clock cycles for register access) - More complex timing analysis

Use Cases: - System where APB runs at different frequency than timer clock - HPET clock derived from external crystal/oscillator - Power management scenarios (clock gating one domain)

4.2 Functional Description

4.2.1 Clock Specifications

4.2.1.1 APB Clock (`pclk`)

Purpose: APB interface protocol clock

Constraints: - Frequency: Typically 10-200 MHz (application-dependent) - Duty cycle: 50% ±10% - Jitter: < 5% of period - No specific minimum/maximum frequency enforced in RTL

Driven Blocks: - APB slave (or the pclk side of the APB CDC wrapper) - PeakRDL register file and register configuration logic ONLY when CDC_ENABLE=0 (with CDC_ENABLE=1 they run on hpet_clk)

4.2.1.2 HPET Clock (`hpet_clk`)

Purpose: Timer counter increment and comparator evaluation

Constraints: - Frequency: User-configurable (typically 1-100 MHz) - Duty cycle: 50% ±10% - Jitter: < 2% of period (affects timer accuracy) - Must be stable and continuous when HPET enabled

Driven Blocks: - Main counter increment - Comparator match detection - Timer control FSMs - Interrupt generation logic

Timer Accuracy: Directly proportional to `hpet_clk` frequency and stability - 10 MHz -> 100ns resolution - 1 MHz -> 1μs resolution - 1 kHz -> 1ms resolution

4.2.2 Reset Domains

4.2.2.1 APB Reset (`presetn`)

Type: Asynchronous active-low reset

Scope: The APB front-end always; with `CDC_ENABLE=0` also the register file and timer core (everything runs on `pclk` in that mode)

Reset Behavior (illustrative; with `CDC_ENABLE=1` the register file resets from `hpet_clk`/`hpet_resetn` instead):

```
always_ff @(posedge pclk or negedge presetn) begin
    if (!presetn) begin
        // Global configuration
        HPET_CONFIG <= 32'h0;         // HPET disabled
        HPET_STATUS <= 32'h0;         // mirror of the core's status
(also reset there)

        // Per-timer configuration
        for (int i = 0; i < NUM_TIMERS; i++) begin
            TIMER_CONFIG[i] <= 32'h0;  // writable bits; reads 0x30
(RO size_cap/per_int_cap = 1)
        end
    end
end
```

Reset Values:

Register	Reset Value	Description
`HPET_CONFIG`	32’h0	Global disable, no legacy mapping
`HPET_STATUS`	32’h0	Mirror of the core’s `r_interrupt_status`; both reset to 0
`HPET_COUNTER_LO`	32’h0	Read/write; reads return the live counter
`HPET_COUNTER_HI`	32’h0	Read/write; reads return the live counter

Register	Reset Value	Description
`HPET_ID`	Constant	RO GCAP_ID[31:0]: `vendor_id` = VENDOR_ID (16 bits, [31:16]), `rev_id` = REVISION_ID ([7:0]), `num_tim_cap` = NUM_TIMERS-1 ([12:8]), `count_size_cap` = 1 ([13]), `leg_rt_cap` = 1 ([15])
`HPET_PERIOD`	Constant	RO GCAP_ID[63:32]: femtoseconds per counter tick, from `COUNTER_CLK_PERIOD_FS`
`TIMER[i]_CONFIG`	32’h30	Interrupt disabled, one-shot, 64-bit; the RO `per_int_cap` [4] and `size_cap` [5] read 1
`TIMER[i]_COMPARATOR_LO`	32’h0	Read/write; reads return the live comparator (includes the periodic advance)
`TIMER[i]_COMPARATOR_HI`	32’h0	Read/write; reads return the live comparator (includes the periodic advance)

4.2.2.2 HPET Reset (`hpet_resetn`)

Type: Asynchronous active-low reset

Scope: With `CDC_ENABLE=1`, the register file and timer core (both run on `hpet_clk` in that mode). With `CDC_ENABLE=0` this reset is unused by the core logic, which runs on `pclk`/`presetn`.

Reset Behavior:

```
always_ff @(posedge hpet_clk or negedge hpet_resetn) begin
    if (!hpet_resetn) begin
        // Main counter
        r_main_counter <= 64'h0;

        // Per-timer state
        for (int i = 0; i < NUM_TIMERS; i++) begin
            r_timer_comparator[i] <= 64'h0;
            r_timer_period[i] <= 64'h0;
            r_timer_armed[i] <= 1'b1;
            r_comp_next_epoch[i] <= 1'b0;
            r_interrupt_status[i] <= 1'b0;
        end
    end
end
```

Reset Values:

Signal	Reset Value	Description
`r_main_counter`	64'h0	Counter starts at zero
`r_timer_comparator[i]`	64'h0	Comparators cleared
`r_timer_period[i]`	64'h0	Period storage cleared
`r_timer_armed[i]`	1'b1	Armed: counter == comparator == 0 is already a match, and the enables gate it until software is ready
`r_comp_next_epoch[i]`	1'b0	Next-epoch hold clear: the comparator is in the counter's current epoch, so the match is not held off
`r_interrupt_status[i]`	1'b0	Interrupt status cleared

4.2.3 Clock Domain Crossing Details

4.2.3.1 CDC Synchronization

When `CDC_ENABLE = 1`, the `apb4_slave_cdc` module handles all clock domain crossing:

Write Path (pclk -> hpet_clk):

```
1. APB write on pclk
2. Command written to APB-side holding registers
3. Command crosses to the hpet_clk domain through the async command
FIFO
   (gaxi_fifo_async inside apb4_slave_cdc)
4. hpet_clk-side logic applies write to timer registers
5. Response crosses back through the async response FIFO
6. APB PREADY asserted (transaction complete)
```

```
Latency: 4-6 pclk cycles
```

Read Path (hpet_clk -> pclk):

```
1. APB read on pclk
2. Read request synchronized to hpet_clk
3. hpet_clk-side logic captures register data
4. Data synchronized back to pclk domain
5. APB PRDATA driven
6. APB PREADY asserted (transaction complete)

Latency: 4-6 pclk cycles
```

Metastability Protection: - The crossing is a pair of async FIFOs (Gray/Johnson-coded pointers); only the FIFO pointers are synchronized – data words never cross combinationally - No toggle handshake exists; ordering and stability come from the FIFO

4.2.4 Counter Read Atomicity

Problem: 64-bit counter spans two 32-bit APB registers

Non-Atomic Read Sequence:

```
1. Read HPET_COUNTER_LO -> captures lower 32 bits
2. Counter increments (may overflow from 0xFFFFFFFF to 0x00000000)
3. Read HPET_COUNTER_HI -> captures upper 32 bits (now incremented!)
4. Result: Lower 32 bits from time T, upper 32 bits from time T+1
```

Software Workaround (Overflow Detection):

```c
uint64_t read_hpet_counter(void) {
    uint32_t hi1, hi2, lo;

    do {
        hi1 = read_reg(HPET_COUNTER_HI);
        lo  = read_reg(HPET_COUNTER_LO);
        hi2 = read_reg(HPET_COUNTER_HI);
    } while (hi1 != hi2);  // Retry if overflow detected

    return ((uint64_t)hi2 << 32) | lo;
}
```

Note: A hardware atomic read is deliberately NOT implemented. Two reads are structural for a 64-bit counter on a 32-bit APB, and the retry loop above is the IA-PC HPET software contract – latching the counter on a LO read would depart from a real HPET rather than improve on it. (RLB/hpet TASK-004, dropped 2026-09-27.)

4.2.5 Clock Gating Considerations

APB Clock Gating: - Safe to gate `pclk` when no APB transactions pending - Must ensure APB master deasserts PSEL before gating - Gating has no effect on HPET timer operation (hpet_clk independent)

HPET Clock Gating: - **DO NOT gate `hpet_clk` while HPET enabled** (HPET_CONFIG[0] = 1) - Counter will stop incrementing -> timers will not fire - Safe to gate only when HPET_CONFIG[0] = 0 (disabled state)

Power Saving Strategy:

```
1. Disable HPET: Write HPET_CONFIG[0] = 0
2. Wait for any pending timer operations to complete
3. Gate hpet_clk
4. APB registers remain accessible (pclk still running)
5. To resume: Ungate hpet_clk, then write HPET_CONFIG[0] = 1
```

Re-enabling does not re-fire expired timers. The fire pulse is a raw `counter >= comparator` match qualified by a per-timer armed latch that clears when the timer fires and sets again only when the match falls (or is held off by the next-epoch bit), when a comparator half is written with the counter halted, or when a periodic catch-up step lands at or ahead of the counter. A one-shot that completed before step 1 is still un-armed when step 5 runs, so the 0->1 of HPET_CONFIG[0] (or of a timer's own enable) while counter >= comparator produces nothing. The one case that does fire on enable is the intended one: a timer whose comparator was written while everything was disabled – even to a value the counter has already passed – is armed, and fires once as soon as both enables are on. A periodic timer left behind by a counter write during the halt catches up silently at step 5: one fire at the next boundary still ahead, the missed periods skipped rather than burst. The silence has a length. A catch-up advances the comparator once per cycle and closes the deficit by period - 1 counts each time (period 1 jumps to counter + 1 in one), so it costs cycles in proportion to the deficit – a period-2 timer left 2^52 counts behind catches up for ~2^52 cycles with no interrupt. That bound is a contract, not a defect, and it is why the HPET specification and this book require the counter halted (`HPET_CONFIG[0] = 0`) before it is written – a torn or stale half on a running counter is exactly how a 2^52 deficit appears – and a periodic comparator programmed near or ahead of the counter.

4.3 Timing

4.3.1 Reset Coordination

4.3.1.1 Synchronous Mode (CDC_ENABLE = 0)

Requirement: `presetn` and `hpet_resetn` should be asserted/deasserted together

Recommended Connection:

```
assign hpet_resetn = presetn;  // Same reset for both domains
```

Reset Sequence:

```
1. Assert presetn = 0 (also asserts hpet_resetn = 0)
2. Hold for >= 10 clock cycles
3. Deassert presetn = 1 (also deasserts hpet_resetn = 1)
4. Wait >= 5 clock cycles before first register access
```

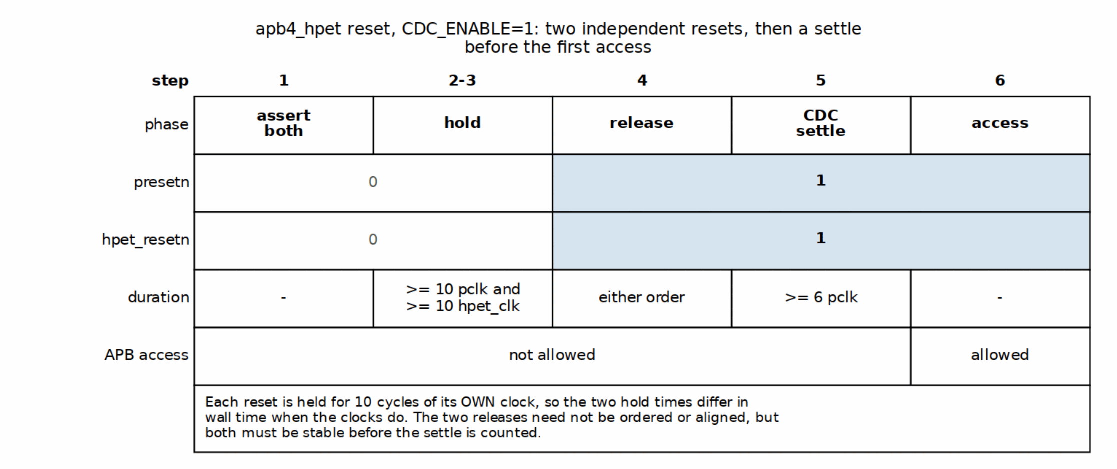
4.3.1.2 Asynchronous Mode (CDC_ENABLE = 1)

Requirement: Both resets can be independent but must overlap during power-on

Recommended Sequence:

```
1. Assert both presetn = 0 and hpet_resetn = 0
2. Hold presetn for >= 10 pclk cycles
3. Hold hpet_resetn for >= 10 hpet_clk cycles
4. Deassert resets (order not critical, but both must be stable)
5. Wait for CDC handshake to stabilize (>= 6 pclk cycles)
6. Begin register accesses
```

4.3.2 Figure 1.4: Reset Sequence with CDC_ENABLE = 1



apb4_hpet reset, CDC_ENABLE=1: two independent resets, then a settle before the first access

step	1	2-3	4	5	6
phase	**assert both**	**hold**	**release**	**CDC settle**	**access**
presetn	0		**1**		
hpet_resetn	0		**1**		
duration	-	>= 10 pclk and >= 10 hpet_clk	either order	>= 6 pclk	-
APB access	not allowed				allowed
	Each reset is held for 10 cycles of its OWN clock, so the two hold times differ in wall time when the clocks do. The two releases need not be ordered or aligned, but both must be stable before the settle is counted.				

Reset Sequence with CDC_ENABLE = 1

4.3.3 Timing Constraints

4.3.3.1 Setup/Hold Requirements

APB Interface (Synchronous):

```
Setup time:  2ns typical (technology-dependent)
Hold time:   1ns typical (technology-dependent)
```

HPET Clock (Asynchronous with CDC):

```
No setup/hold requirements between pclk and hpet_clk
CDC synchronizers handle all timing
```

4.3.3.2 Maximum Operating Frequencies

Technology-Dependent Estimates (Post-Synthesis): - APB clock: 200+ MHz (typical modern process) - HPET clock: 100+ MHz (limited by counter/comparator logic) - Clock domain crossing: Synchronizers support arbitrary frequency ratios

Recommended Operating Points: - APB clock: 10-100 MHz (typical SoC bus speeds) - HPET clock: 1-50 MHz (sufficient for most timing applications)

4.4 Navigation

Next: Chapter 1.4 - Acronyms and Terminology

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 APB HPET Acronyms and Terminology

5.1 Overview

The vocabulary used across this MAS, collected in one place so the rest of the chapters can stay terse.

Acronym	Full Term	Description
AMBA	Advanced Microcontroller Bus Architecture	ARM's on-chip interconnect specification
APB	Advanced Peripheral Bus	AMBA low-complexity peripheral bus protocol
CDC	Clock Domain Crossing	Synchronization between asynchronous clock domains
FSB	Front Side Bus	Legacy PC architecture bus (not supported in APB HPET)
FSM	Finite State Machine	Sequential logic controller
HPET	High Precision Event Timer	Multi-timer peripheral for precise timing
IA-PC	Industry-standard PC architecture	PC platform specification (architectural reference)
IRQ	Interrupt Request	Hardware interrupt signal
PIT	Programmable Interval Timer	Legacy PC timer (8254-compatible)
RO	Read-Only	Register field cannot be written by software
RTC	Real-Time Clock	Calendar/time-of-day clock (not emulated by HPET)
RW	Read-Write	Register field can be read and written
SystemRDL	System Register Description Language	Industry-standard register specification language
W1C	Write-1-to-Clear	Register field cleared by writing 1, writing 0 has no

Acronym	Full Term	Description
		effect
WO	Write-Only	Register field can only be written, reads return undefined

5.2 Functional Description

5.2.1 Terminology

64-bit Counter: The main free-running counter that increments on every HPET clock cycle. Provides high-resolution timestamp and comparison base for all timers.

Comparator: Per-timer 64-bit value that defines when a timer should fire. Timer fires when main counter value becomes greater than or equal to comparator value.

Fire / Fired: Event when a timer's comparator matches the main counter value. In one-shot mode, timer fires once. In periodic mode, timer fires repeatedly.

One-Shot Mode: Timer operating mode where the timer fires once when the counter reaches the comparator value, then remains idle until reconfigured.

Periodic Mode: Timer operating mode where the timer fires repeatedly at fixed intervals. After each fire event, the comparator is automatically incremented by the period value.

Period: In periodic mode, the interval (in HPET clock cycles) between timer fires. Stored internally and used for auto-incrementing the comparator.

PeakRDL: Industry-standard toolchain for generating register files from SystemRDL specifications. Used to generate `hpet_regs.sv` from `hpet_regs.rdl`.

Per-Timer Data Bus: Dedicated 64-bit data path for each timer to prevent corruption when multiple timer registers are written in rapid succession.

Timer Corruption: Historical bug where shared data bus allowed one timer's configuration to overwrite another timer's configuration. Fixed by implementing per-timer dedicated data buses.

Write Strobe: Edge-detected pulse generated when software writes to a timer configuration register. Used to sample comparator and configuration data atomically.

5.2.2 Register Field Access Types

RO (Read-Only): - Software can read the field - Software writes are ignored - Hardware controls the value - Example: `HPET_ID` register

RW (Read-Write): - Software can read and write the field - Hardware may also update the value - Example: `HPET_CONFIG[0]` (enable bit)

WO (Write-Only): - Software can write the field - Software reads return undefined value - Hardware uses written value internally - Example: none in this block – HPET_COUNTER_LO/HI are RW with live readback; the HPET has no write-only registers

W1C (Write-1-to-Clear): - Software writes 1 to clear the bit - Software writes 0 have no effect - Hardware can set the bit - Example: `HPET_STATUS` interrupt flags

---

5.3 Design Notes

5.3.1 Signal Naming Conventions

APB Signals: All APB signals use standard AMBA naming with `p` prefix: - `pclk` - APB clock - `presetn` - APB reset (active-low) - `paddr` - APB address bus - `psel` - APB select - `penable` - APB enable - `pwrite` - APB write enable - `pwdata` - APB write data - `pready` - APB ready - `prdata` - APB read data - `pslverr` - APB slave error

HPET Domain Signals: Timer-related signals use descriptive names: - `hpet_clk` - HPET timer clock - `hpet_resetn` - HPET reset (active-low) - `timer_irq[N]` - Timer interrupt outputs - `r_main_counter` - Internal 64-bit counter - `r_timer_comparator[i]` - Per-timer comparator value - `r_timer_period[i]` - Per-timer period value

Prefix Conventions: - `r_` - Registered (flip-flop) signal - `w_` - Wire (combinational) signal - `cfg_` - Configuration input - `hwif_` - PeakRDL hardware interface signal

5.3.2 Common Abbreviations in Code

Abbreviation	Meaning	Example
`cfg`	Configuration	`cfg_initial_credit`
`cmp`	Comparator	`timer_cmp_data`
`wr`	Write	`timer_comp_write_lo`
`rd`	Read	`counter_rd_data`
`hi`	High (upper 32 bits)	`HPET_COUNTER_HI`
`lo`	Low (lower 32 bits)	`HPET_COUNTER_LO`
`en`	Enable	`timer_en`
`irq`	Interrupt Request	`timer_irq`
`clr`	Clear	`status_clr`

5.4 Navigation

Next: Chapter 1.5 - References

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

6 APB HPET References

Everything this MAS leaned on, plus the pointers you'll want when you need to go deeper than the spec itself.

6.1 References

6.1.1 External Standards and Specifications

AMBA Protocol Specifications: - **AMBA APB Protocol Specification v2.0** - Publisher: ARM Limited - Document ID: IHI 0024C - URL: https://developer.arm.com/documentation/ihi0024/latest - Relevance: APB interface protocol specification

SystemRDL: - **SystemRDL 2.0 Specification** - Publisher: Accellera Systems Initiative - URL: https://www.accellera.org/downloads/standards/systemrdl - Relevance: Register description language for `hpet_regs.rdl`

- **PeakRDL Documentation**
    - Project: PeakRDL Register Description Language Compiler
    - URL: https://peakrdl.readthedocs.io/
    - Relevance: SystemRDL to SystemVerilog compiler tool

Architectural Reference (Not Specification Compliant): - **IA-PC HPET Specification 1.0a** - Publisher: (see specification) - Date: October 2004 - Relevance: Architectural inspiration (APB HPET is NOT IA-PC HPET compliant) - **Note:** Used as reference for timer concepts only. APB HPET uses APB interface (not memory-mapped), different register layout, and does not support legacy modes or FSB delivery.

6.1.2 Internal Project Documentation

Component-Specific Documentation: - PRD.md - Product Requirements Document - Complete functional requirements - Parameter specifications - Verification status

- CLAUDE.md - AI Integration Guide
    - Component architecture overview
    - Known issues and workarounds
    - Test methodology
- RLB/hpet tasks - Development Task Tracking
    - Active work items
    - Completed milestones
    - Future enhancements
- IMPLEMENTATION_STATUS.md - Test Results

- Detailed test results per configuration
- Pass/fail statistics
- Root cause analysis

RTL Source Files: - `rtl/hpet/apb4_hpet.sv` - Top-level wrapper module - `rtl/hpet/hpet_core.sv` - Core timer logic - `rtl/hpet/hpet_config_regs.sv` - Register wrapper - `rtl/hpet/hpet_regs.sv` - PeakRDL generated register file (from hpet_regs.rdl) - `rtl/hpet/hpet_regs_pkg.sv` - PeakRDL generated package

SystemRDL Specification: - `rdl/hpet/hpet_regs.rdl` - Register description - `rdl/hpet/README.md` - PeakRDL generation instructions

Testbench Files: - `dv/tbclasses/hpet/hpet_tb.py` - Main testbench class - `dv/tbclasses/hpet/hpet_tests_basic.py` - Basic test suite - `dv/tbclasses/hpet/hpet_tests_medium.py` - Medium test suite - `dv/tbclasses/hpet/hpet_tests_full.py` - Full test suite - `dv/tests/test_apb4_hpet.py` - Test runner with pytest integration

Known Issues Documentation: - `known_issues/README.md` - Issue tracking overview - Timer counter-cleanup fix details: see TASK-001 in RLB/hpet tasks (the old standalone `known_issues/resolved/timer_cleanup_issue.md` page was not carried over into retro_legacy_blocks)

6.1.3 Repository-Wide Documentation

Root Documentation: - `/README.md` - Repository overview and setup - `/PRD.md` - Master project requirements - `/CLAUDE.md` - Repository-wide AI guidance

Framework Documentation: - `bin/TBClasses/` - In-repo shared testbench utilities (import as `from TBClasses. ...`; APB helpers under `bin/TBClasses/apb/`) - RTLDesignSherpa-DV repository - Full CocoTBFramework (components, BFMs, tbclasses), editable-installed into the venv

Verification Architecture: - `docs/user-guides/VERIFICATION_ARCHITECTURE_GUIDE.md` - Complete verification patterns - Three-layer architecture (TB + Scoreboard + Test) - Queue-based vs memory model verification - Mandatory testbench methods

6.1.4 Related RTL Components

APB Infrastructure: - `rtl/amba/apb4/apb4_slave.sv` - Standard APB slave - `rtl/amba/apb4/apb4_slave_cdc.sv` - APB slave with clock domain crossing - `projects/components/utility-ip/converters/rtl/peakrdl_to_cmdrsp.sv` - PeakRDL adapter

Clock Domain Crossing: - `rtl/amba/cdc/cdc_4_phase_handshake.sv` - CDC handshake synchronizer (renamed from `cdc_handshake.sv`) - `rtl/amba/cdc/cdc_synchronizer.sv` - Multi-stage bit synchronizer (replaces the old `sync_2ff.sv`) - `rtl/cdc/sync_pulse.sv` - Pulse synchronizer

Common Utilities: - Edge detection for write strobes is implemented inline in `rtl/hpet/hpet_config_regs.sv` (the old standalone `rtl/common/edge_detect.sv` module no longer exists) - `rtl/common/counter_bin.sv` - Binary counter (similar to HPET main counter)

6.1.5 Design Tools

Simulation: - Verilator 5.0+ - RTL simulator - CocoTB 1.9+ - Python testbench framework - pytest 7.0+ - Test runner and parametrization

Register Generation: - PeakRDL-regblock 0.17+ - SystemRDL to SystemVerilog compiler - PeakRDL 1.0+ - SystemRDL front-end

Waveform Viewing: - GTKWave - VCD waveform viewer - GTKW files available in `dv/GTKW/` directory

6.1.6 Industry Best Practices References

RTL Coding: - *Synthesis and Simulation Design Guide* - Xilinx UG901 - Best practices for RTL coding style - Clock domain crossing guidelines - Reset strategies

- *RTL Modeling with SystemVerilog for Simulation and Synthesis* - Stuart Sutherland
  - SystemVerilog coding guidelines
  - Finite state machine design patterns

Verification: - *Writing Testbenches using SystemVerilog* - Janick Bergeron - Testbench architecture patterns - Functional coverage methodology

- *Verification Methodology Manual for SystemVerilog* - Janick Bergeron et al.
  - UVM-like verification patterns
  - Coverage-driven verification

AMBA Protocols: - *AMBA Design Kit (ADK)* - ARM - Reference implementations - Protocol checkers - Example testbenches

6.1.7 Version Control and Issue Tracking

Git Repository: - Main branch: Production-ready code - Feature branches: Active development - Commit history: Detailed change log

Issue Labels: - `bug` - Functional defects - `enhancement` - New features - `documentation` - Documentation updates - `testing` - Test infrastructure improvements

6.1.8 Related Projects

RTL Design Sherpa Components: - APB HPET (this component) - AMBA AXI4 Monitors (`rtl/amba/`) - RAPIDS DMA Engine (`projects/components/dma-ip/rapids/`) - Delta Network Arbiter (`projects/components/fabric-gen-ip/delta/`)

External Dependencies: - None - APB HPET is fully self-contained within RTL Design Sherpa

---

6.2 Navigation

Next: Chapter 2 - Blocks

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

7 APB HPET Blocks

7.1 Overview

The APB HPET component consists of four primary SystemVerilog modules organized in a hierarchical structure. This chapter walks through what each one owns and how they talk to each other.

7.1.1 Block Hierarchy

```
apb4_hpet (Top Level)
+-- APB Slave Interface
|   +-- apb4_slave.sv (CDC_ENABLE=0) OR
|   +-- apb4_slave_cdc.sv (CDC_ENABLE=1)
|
+-- hpet_config_regs (Register Wrapper)
|   +-- hpet_regs (PeakRDL Generated)
|   |   +-- Register File Logic
|   |
|   +-- Mapping Logic
|       +-- Per-Timer Data Buses
|       +-- Write-Strobe Alignment (swmod rising edge, per half)
|       +-- HPET_STATUS Mirror and W1C Decode
|
+-- hpet_core (Timer Logic)
    +-- 64-bit Free-Running Counter
    +-- Per-Timer Comparators [NUM_TIMERS]
    +-- Armed Latch and Fire Pulse [NUM_TIMERS]
    +-- Next-Epoch Hold Bit [NUM_TIMERS]
    +-- Interrupt Generation [NUM_TIMERS]
```

7.2 Functional Description

7.2.1 Module Responsibilities

7.2.1.1 1. apb4_hpet (Top Level Integration)

File: `rtl/hpet/apb4_hpet.sv` **Purpose:** System integration and CDC selection

Responsibilities: - Instantiates APB slave with or without CDC based on `CDC_ENABLE` parameter - Routes signals between APB interface and configuration registers - Exposes timer interrupts to system - Provides unified external interface

Key Features: - Conditional CDC instantiation (generate block) - Clock domain management - Parameter propagation to child modules - Single-point configuration

7.2.1.2 2. hpet_config_regs (Register Wrapper)

File: `rtl/hpet/hpet_config_regs.sv` **Purpose:** Bridge between PeakRDL registers and HPET core

Responsibilities: - Instantiates PeakRDL-generated register file - Maps PeakRDL hardware interface to HPET core signals - Implements per-timer dedicated data buses (corruption fix) - Turns the register block’s `swmod` levels into aligned one-cycle write strobes, one per 32-bit half - Combines the LO/HI fields into 64-bit data buses - Mirrors the core’s interrupt status into HPET_STATUS and decodes the W1C write into a per-bit clear - Drives HPET_ID’s vendor, revision and timer-count fields from the parameters

Key Features: - Per-timer data buses prevent configuration corruption - Strobe-driven loads: a write is the event, not a change in the stored value, so rewriting the same comparator still reloads it (whether it also re-arms the timer is the core’s call: only while the counter is halted) - Per-bit W1C clear mask from the write data and byte enables - One clear pulse per write, so a same-cycle fire is never undone

7.2.1.3 3. hpet_regs (PeakRDL Generated)

File: `rtl/hpet/hpet_regs.sv` **Purpose:** Auto-generated register file from SystemRDL specification

Responsibilities: - Implements all HPET registers from RDL specification - Provides CPU interface (passthrough protocol) - Generates hardware interface structs - Handles field access types (RO, RW, W1C)

Key Features: - Single source of truth (hpet_regs.rdl) - Regeneratable from specification - Comprehensive field control - Standard passthrough CPU interface

7.2.1.4 4. hpet_core (Timer Logic)

File: `rtl/hpet/hpet_core.sv` **Purpose:** Core timer functionality and comparison logic

Responsibilities: - Implements 64-bit free-running counter - Manages per-timer comparators and periods - Detects counter match conditions - Generates timer fire events and interrupts - Handles one-shot vs periodic mode differences

Key Features: - Fully synchronous timer logic - Per-timer FSM (conceptual) - Automatic period reload (periodic mode) - Armed-latch fire detection: one fire per arm, no re-fire on enable, comparator writes re-arm only while the counter is halted - Periodic catch-up: missed periods are skipped, never burst (period 1 steps to counter + 1) - Next-epoch hold: an advance that carries out of the compare width is held off until the counter wraps at that width - Configurable timer count (2, 3, or 8 timers)

7.2.2 Data Flow Overview

7.2.2.1 APB Write Transaction Flow

```
APB Master
    ↓ PSEL, PENABLE, PADDR, PWDATA
APB Slave (or APB Slave CDC)
    ↓ cmd_valid, cmd_pwrite, cmd_paddr, cmd_pwdata
peakrdl_to_cmdrsp Adapter
    ↓ regblk_req, regblk_req_is_wr, regblk_addr, regblk_wr_data
hpet_regs (PeakRDL)
    ↓ hwif_out (register values)
hpet_config_regs (Mapping)
    ↓ timer_int_enable/type/size, timer_comp_write_lo/hi,
timer_comp_wdata[i]
hpet_core (Timer Logic)
    -> Counter/Comparator update
```

7.2.2.2 APB Read Transaction Flow

```
APB Master
    ↓ PSEL, PENABLE, PADDR, PWRITE=0
APB Slave (or APB Slave CDC)
    ↓ cmd_valid, cmd_pwrite=0, cmd_paddr
peakrdl_to_cmdrsp Adapter
    ↓ regblk_req, regblk_req_is_wr=0, regblk_addr
hpet_regs (PeakRDL)
    ← hwif_in (live counter, status)
    ↓ regblk_rd_data
peakrdl_to_cmdrsp Adapter
    ↓ rsp_prdata
APB Slave (or APB Slave CDC)
    ↓ PRDATA
APB Master
```

7.2.2.3 Timer Fire Flow

1. **`hpet_core`:** the counter increments. A raw match (counter >= comparator) while the timer is armed fires it: `timer_int_status[i]` asserts (sticky, owned here) and the armed latch clears. `timer_irq[i]` asserts one clock later if `int_enable` was set at the fire.
2. **`hpet_config_regs`** drives `hwif_in.HPET_STATUS.timer_int_status.next` from that live level every cycle.
3. **`hpet_regs` (PeakRDL):** HPET_STATUS reads the mirrored level.
4. **Software** reads HPET_STATUS, then writes 1 to bit i.

5. **`hpet_config_regs`** pulses `timer_int_clear[i]` for ONE cycle. The mask is write data AND byte enables: bits written 0 are untouched, and a write of 0x0 clears nothing.
6. **`hpet_core`:** `timer_int_status[i]` clears (a fire of the same bit in that cycle wins) and `timer_irq[i]` deasserts.

7.2.3 Clock Domain Organization

7.2.3.1 Synchronous Mode (CDC_ENABLE=0)

```
APB Clock Domain (pclk)
+-- apb4_slave
+-- hpet_config_regs
+-- hpet_regs
+-- hpet_core

All modules use pclk
No clock domain crossing required
```

7.2.3.2 Asynchronous Mode (CDC_ENABLE=1)

```
APB Clock Domain (pclk)
+-- apb4_slave_cdc (pclk side)
+-- [CDC boundary]

HPET Clock Domain (hpet_clk)
+-- apb4_slave_cdc (hpet_clk side)
+-- hpet_config_regs
+-- hpet_regs
+-- hpet_core

CDC synchronization between pclk and hpet_clk
```

7.2.4 Module Communication

7.2.4.1 hpet_config_regs -> hpet_core Interface

Control Signals (hpet_config_regs -> hpet_core):

```
output logic                    hpet_enable;            // Global
enable
output logic                    counter_write_lo;       //
Counter[31:0] write strobe
output logic                    counter_write_hi;       //
Counter[63:32] write strobe
output logic [63:0]             counter_wdata;          // Counter
write data
output logic                    legacy_replacement;     //
```

```
HPET_CONFIG[1]
output logic [NUM_TIMERS-1:0]   timer_int_enable;       // TN_CONF[2];
gates the irq only (no per-timer run enable)
output logic [NUM_TIMERS-1:0]   timer_type;             // Per-timer
mode (0=one-shot, 1=periodic)
output logic [NUM_TIMERS-1:0]   timer_size;             // ~TN_CONF[8]
(32MODE): 0=32-bit, 1=64-bit
output logic [NUM_TIMERS-1:0]   timer_comp_write_lo;    // Per-timer
comparator[31:0] write strobe
output logic [NUM_TIMERS-1:0]   timer_comp_write_hi;    // Per-timer
comparator[63:32] write strobe
output logic [63:0]             timer_comp_wdata[NUM_TIMERS];  // Per-
timer data buses
```

Status Signals (hpet_core -> hpet_config_regs):

```
input  logic [63:0]             counter_rdata;          // Live
counter value
input  logic [NUM_TIMERS-1:0]   timer_int_status;       // Per-timer
fire status
```

Interrupt Clearing (hpet_config_regs -> hpet_core):

```
output logic [NUM_TIMERS-1:0]   timer_int_clear;        // Per-bit
clear of the sticky status
```

7.2.4.2 hpet_config_regs -> hpet_regs Interface

Uses PeakRDL-generated structs:

```
// From config regs to PeakRDL
input  hpet_regs_pkg::hpet_regs__in_t  hwif_in;

// From PeakRDL to config regs
output hpet_regs_pkg::hpet_regs__out_t hwif_out;
```

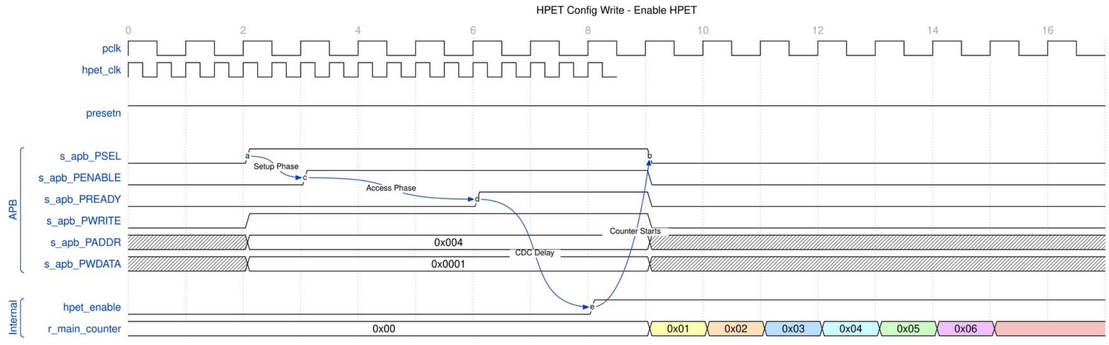
7.3 Waveforms

7.3.1 Timer Operation Waveforms

7.3.1.1 Configuration Write

When software writes to HPET_CONFIG to start the counter, the enable signal propagates through the register file to the core.

7.3.2 Waveform 2.1: HPET Config Write


HPET Config Write - Enable HPET
0
2
4
6
8
10
12
14
16
pclk
hpet_clk
presetn
APB
s_apb_PSEL
s_apb_PENABLE
s_apb_PREADY
s_apb_PWRITE
s_apb_PADDR
s_apb_PWDATA
Internal
hpet_enable
r_main_counter
a
b
c
d
e
Setup Phase
Access Phase
Counter Starts
CDC Delay
0x004
0x0001
0x00
0x01
0x02
0x03
0x04
0x05
0x06

APB write to HPET_CONFIG enables counter incrementing

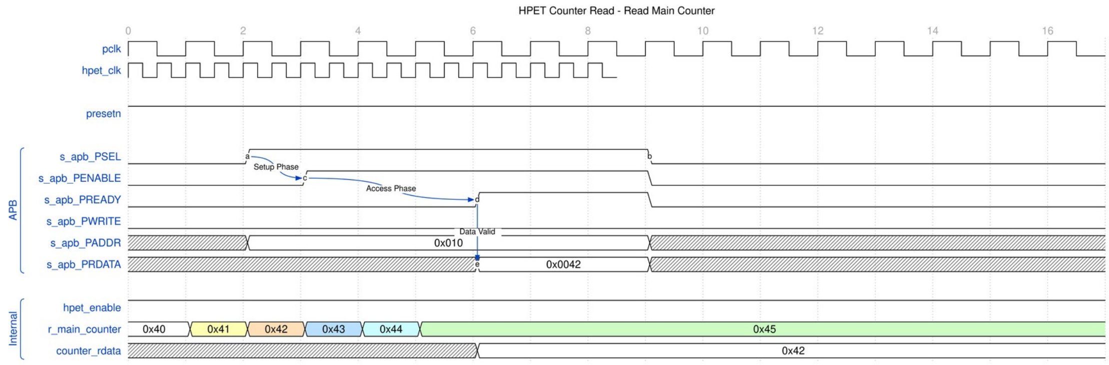
HPET Config Write

The APB write to address 0x010 (HPET_CONFIG) sets `hpet_enable`, which starts the main counter incrementing.

7.3.2.1 Counter Read

Reading the main counter returns the current 64-bit counter value.

7.3.3 Waveform 2.2: HPET Counter Read


HPET Counter Read - Read Main Counter
0
2
4
6
8
10
12
14
16
pclk
hpet_clk
presetn
APB
s_apb_PSEL
s_apb_PENABLE
s_apb_PREADY
s_apb_PWRITE
s_apb_PADDR
s_apb_PRDATA
Internal
hpet_enable
r_main_counter
counter_rdata
a
b
c
d
e
Setup Phase
Access Phase
Data Valid
0x010
0x0042
0x40
0x41
0x42
0x43
0x44
0x45
0x42

APB read of HPET_COUNTER_LO captures current counter value

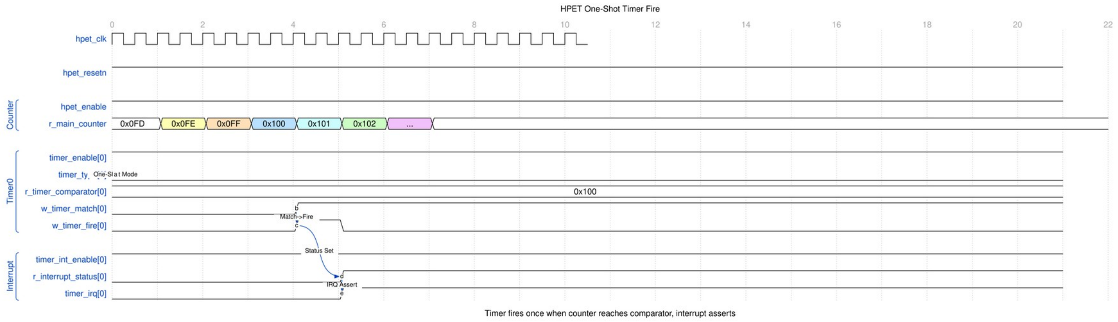
HPET Counter Read

The counter value is captured during the APB read transaction and returned on PRDATA.

7.3.3.1 One-Shot Timer Fire

In one-shot mode, the timer fires once when the counter reaches the comparator value.

7.3.4 Waveform 2.3: HPET One-Shot Timer Fire


HPET One-Shot Timer Fire
0
2
4
6
8
10
12
14
16
18
20
22
hpet_clk
hpet_resetn
Counter
hpet_enable
r_main_counter
0x0FD
0x0FE
0x0FF
0x100
0x101
0x102
...
Timer0
timer_enable[0]
timer_ty
One-Shot Mode
r_timer_comparator[0]
0x100
w_timer_match[0]
w_timer_fire[0]
b
Match->Fire
c
Status Set
Interrupt
timer_int_enable[0]
r_interrupt_status[0]
timer_irq[0]
d
IRQ Assert
e
Timer fires once when counter reaches comparator, interrupt asserts

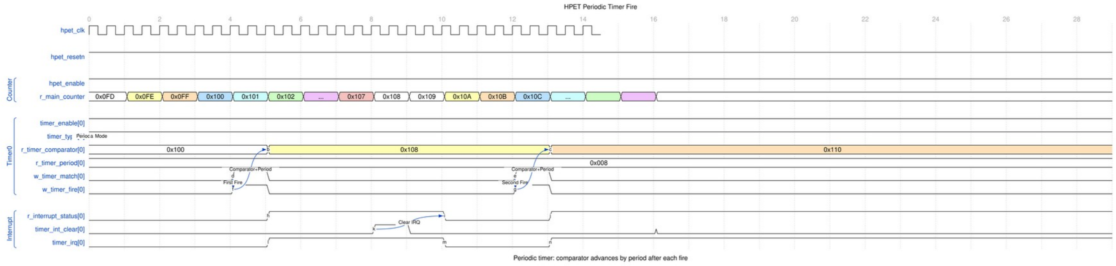
HPET One-Shot Timer Fire

When `r_main_counter` equals `r_timer_comparator[0]`, the match signal asserts, triggering `w_timer_fire[0]`. The interrupt output `timer_irq[0]` asserts and remains active until software clears it.

7.3.4.1 Periodic Timer Fire

In periodic mode, the timer fires repeatedly, automatically adding the period to the comparator.

7.3.5 Waveform 2.4: HPET Periodic Timer Fire


HPET Periodic Timer Fire
0
2
4
6
8
10
12
14
16
18
20
22
24
26
28
hpet_clk
hpet_resetn
Counter
hpet_enable
r_main_counter
0x0FD
0x0FE
0x0FF
0x100
0x101
0x102
...
0x107
0x108
0x109
0x10A
0x10B
0x10C
...
Timer0
timer_enable[0]
timer_typ
Periodic Mode
r_timer_comparator[0]
0x100
0x108
0x110
r_timer_period[0]
0x008
w_timer_match[0]
w_timer_fire[0]
Comparator+Period
First Fire
Comparator+Period
Second Fire
Interrupt
r_interrupt_status[0]
timer_int_clear[0]
timer_irq[0]
Clear IRQ
Periodic timer: comparator advances by period after each fire

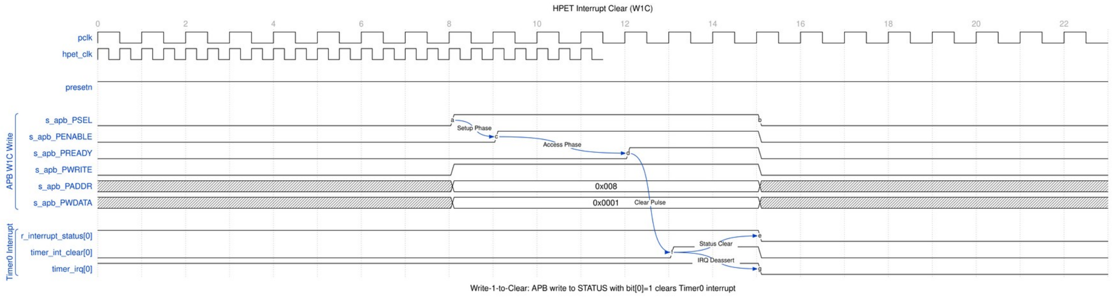
HPET Periodic Timer Fire

After each fire event, the comparator is updated: `comparator += period`. This allows continuous periodic interrupts without software intervention.

7.3.5.1 Interrupt Clear (W1C)

Software clears timer interrupts by writing 1 to the corresponding bit in HPET_STATUS.

7.3.6 Waveform 2.5: HPET Interrupt Clear


HPET Interrupt Clear (W1C)
0
2
4
6
8
10
12
14
16
18
20
22
pclk
hpet_clk
presetn
APB W1C Write
s_apb_PSEL
s_apb_PENABLE
s_apb_PREADY
s_apb_PWRITE
s_apb_PADDR
s_apb_PWDATA
Timer0 Interrupt
r_interrupt_status[0]
timer_int_clear[0]
timer_irq[0]
a
b
c
d
e
f
g
Setup Phase
Access Phase
0x008
0x0001
Clear Pulse
Status Clear
IRQ Deassert
Write-1-to-Clear: APB write to STATUS with bit[0]=1 clears Timer0 interrupt

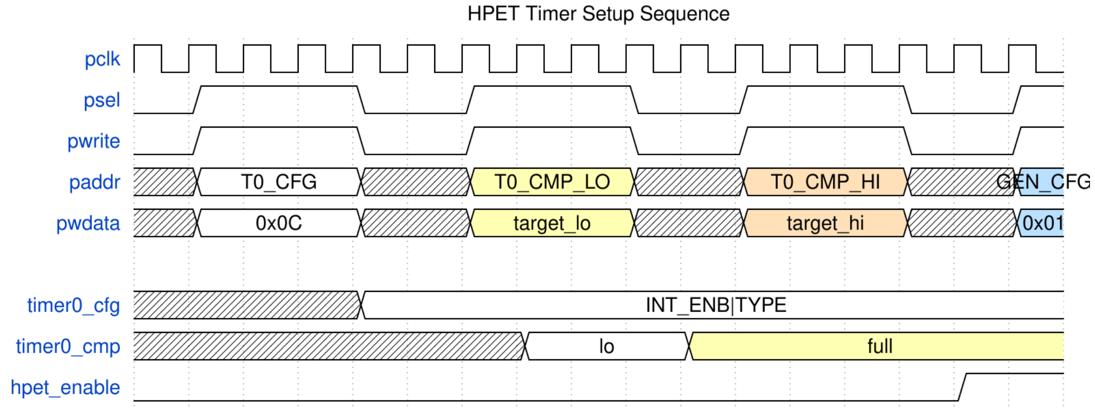
HPET Interrupt Clear

The W1C (Write-1-to-Clear) write clears only the bits written with 1: the wrapper decodes the write data and byte enables into a per-bit mask, pulses `timer_int_clear` for one cycle, and hpet_core drops just those bits. Writing 0 – to a bit or to the whole register – changes nothing.

7.3.6.1 Timer Setup Sequence

Configuring a timer requires multiple APB writes: config register, then comparator low/high words.

7.3.7 Waveform 2.6: HPET Timer Setup


HPET Timer Setup Sequence
pclk
psel
pwrite
paddr
pwdata
timer0_cfg
timer0_cmp
hpet_enable
T0_CFG
T0_CMP_LO
T0_CMP_HI
GEN_CFG
0x0C
target_lo
target_hi
0x01
INT_ENB|TYPE
lo
full

HPET Timer Setup

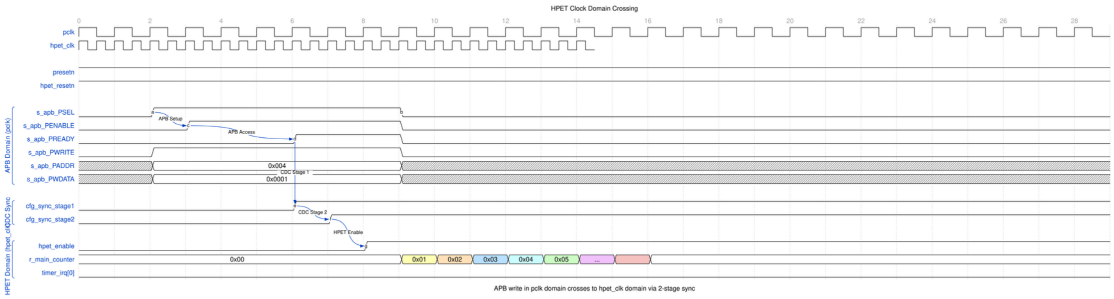
The sequence shows three consecutive writes: 1. TIMER_CONFIG (0x100): interrupt enable (bit 2), periodic mode (bit 3) 2. TIMER_COMPARATOR_LO (0x108): Lower 32 bits of comparator 3. TIMER_COMPARATOR_HI (0x10C): Upper 32 bits of comparator

Each comparator half loads hpet_core on its own write strobe, so between writes 2 and 3 the core holds {old HI, new LO}. A write re-arms the timer only while the counter is halted, so that torn value can never fire on a running timer – but a running timer then re-arms only when the completed value is above the counter. Write the comparator before starting the counter, or halt it (`HPET_CONFIG.hpet_enable = 0`) around the pair, whenever the new value may already be behind the counter; a write with the counter halted re-arms the timer to any value. There is no per-timer stop.

7.3.7.1 Clock Domain Crossing (CDC Mode)

When CDC_ENABLE=1, APB transactions cross from pclk to hpet_clk through a pair of async FIFOs inside apb4_slave_cdc (Gray/Johnson pointers per USE_JOHNSON) – not per-signal synchronizers.

7.3.8 Waveform 2.7: HPET CDC Crossing


HPET Clock Domain Crossing
0
2
4
6
8
10
12
14
16
18
20
22
24
26
28
pclk
hpet_clk
presetn
hpet_resetn
APB Domain (pclk)
s_apb_PSEL
s_apb_PENABLE
s_apb_PREADY
s_apb_PWRITE
s_apb_PADDR
s_apb_PWDATA
APB Setup
APB Access
0x004
CDC Stage 1
0x0001
CDC Sync
cfg_sync_stage1
cfg_sync_stage2
CDC Stage 2
HPET Enable
HPET Domain (hpet_clk)
hpet_enable
r_main_counter
timer_irq[0]
0x00
0x01
0x02
0x03
0x04
0x05
...
APB write in pclk domain crosses to hpet_clk domain via 2-stage sync

HPET CDC Crossing

The diagram shows the latency introduced by CDC synchronization. Configuration changes in the APB domain take 2-3 hpet_clk cycles to affect the timer core

---

7.4 Design Notes

7.4.1 Resource Allocation

Per-Configuration Estimates (Post-Synthesis):

Component	NUM_TIMERS=2	NUM_TIMERS=3	NUM_TIMERS=8
hpet_core			
- Main counter	64 FF, 70 LUTs	(same)	(same)
- Per-timer logic	256 FF, 170 LUTs	384 FF, 255 LUTs	1024 FF, 680 LUTs
- Subtotal	320 FF, 240 LUTs	448 FF, 325 LUTs	1088 FF, 750 LUTs
hpet_config_regs			
- Mapping logic	~50 FF, ~100 LUTs	~75 FF, ~150 LUTs	~150 FF, ~300 LUTs

Component	NUM_TIMERS=2	NUM_TIMERS=3	NUM_TIMERS=8
- Edge detect	~10 FF, ~20 LUTs	~15 FF, ~30 LUTs	~30 FF, ~60 LUTs
- Subtotal	60 FF, 120 LUTs	90 FF, 180 LUTs	180 FF, 360 LUTs
hpet_regs			
- Register storage	~128 FF, ~100 LUTs	~160 FF, ~125 LUTs	~256 FF, ~200 LUTs
apb4_slave (no CDC)			
- APB protocol	~20 FF, ~50 LUTs	(same)	(same)
apb4_slave_cdc (with CDC)			
- CDC logic	~100 FF, ~150 LUTs	(same)	(same)
Total (no CDC)	~528 FF, ~510 LUTs	~718 FF, ~680 LUTs	~1544 FF, ~1360 LUTs
Total (with CDC)	~608 FF, ~610 LUTs	~798 FF, ~780 LUTs	~1624 FF, ~1460 LUTs

Scaling: Resource usage is primarily driven by `NUM_TIMERS` parameter. Each additional timer adds ~128 FF and ~85 LUTs.

7.5 Testing

7.5.1 Integration Checklist

When integrating APB HPET:

1. Parameter Selection: - [ ] `NUM_TIMERS`: 2, 3, or 8 timers - [ ] `VENDOR_ID`: reported in HPET_ID[31:16]; the spec's full 16-bit field - [ ] `REVISION_ID`: reported in HPET_ID[7:0] - [ ] `COUNTER_CLK_PERIOD_FS`: femtoseconds per counter tick, at HPET_PERIOD (0x004). Must match `CDC_ENABLE[0] ? hpet_clk : pclk` - [ ] `CDC_ENABLE`: 0 for synchronous, 1 for asynchronous clocks

2. Clock Configuration: - [ ] Connect `pclk` (APB clock domain) - [ ] Connect `hpet_clk` (timer clock domain) - [ ] If `CDC_ENABLE=0`: Ensure `pclk = hpet_clk` - [ ] If `CDC_ENABLE=1`: Clocks can be asynchronous

3. Reset Coordination: - [ ] Assert `presetn` (APB reset, active-low) - [ ] Assert `hpet_resetn` (HPET reset, active-low) - [ ] If `CDC_ENABLE=1`: Ensure both resets overlap at power-on - [ ] Hold resets for >=10 clock cycles

4. APB Interface: - [ ] Connect all APB signals (PSEL, PENABLE, PADDR, etc.) - [ ] PADDR width = 12 bits (supports up to 4KB address space) - [ ] PWDATA/PRDATA width = 32 bits (fixed)

5. Interrupt Outputs: - [ ] Connect `timer_irq[NUM_TIMERS-1:0]` to interrupt controller - [ ] Each timer has independent interrupt output - [ ] Interrupts are active-high, level-sensitive

6. Verification: - [ ] Test register access via APB - [ ] Verify timer operation (one-shot and periodic modes) - [ ] Test interrupt generation and clearing - [ ] Validate CDC if enabled

---

7.6 Navigation

Next: Chapter 2.2 - hpet_config_regs

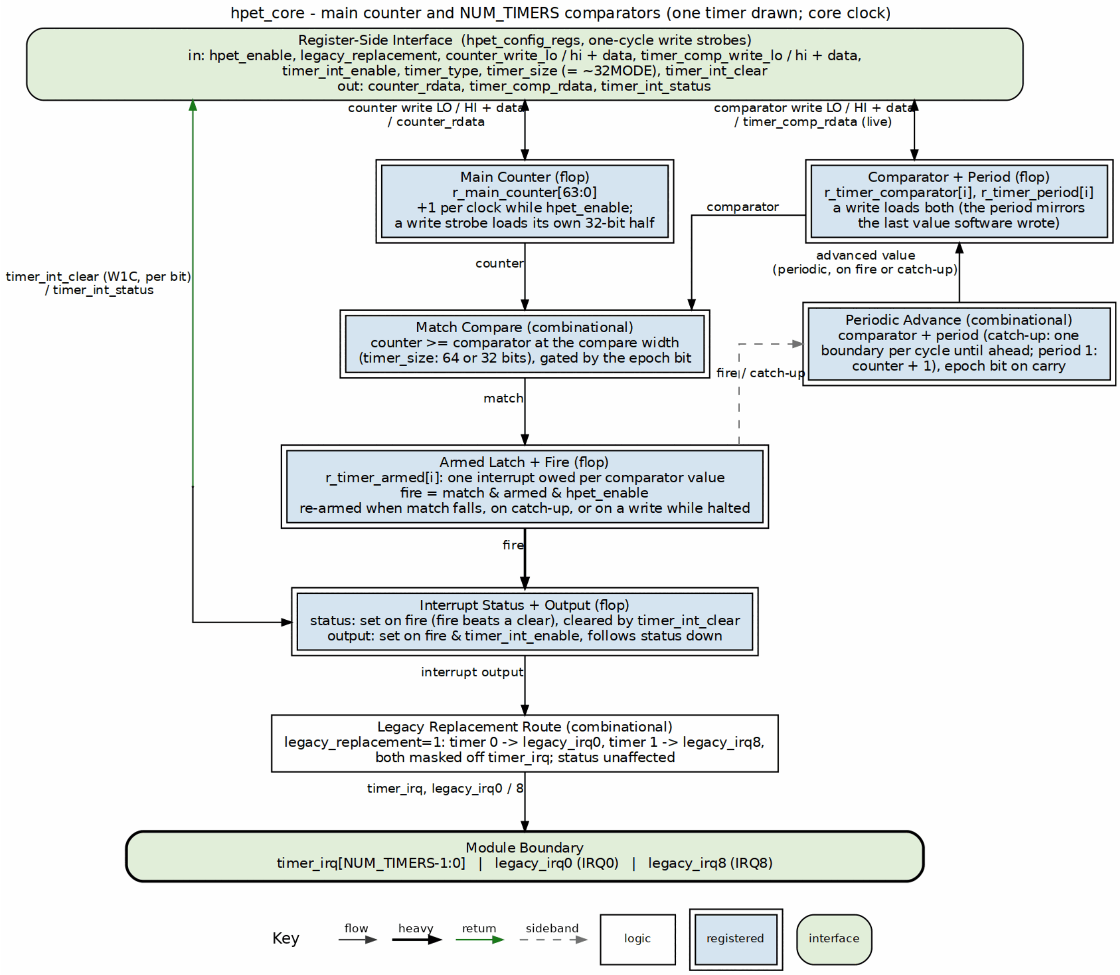
RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

8 hpet_core

8.1 Overview

The HPET core (`hpet_core.sv`) is where the timing actually happens: a 64-bit free-running counter, per-timer comparators, and interrupt generation. This module runs entirely on the core clock (`hpet_clk` when CDC_ENABLE=1, `pclk` otherwise) and contains all timing-critical logic.

8.1.1 Figure 2.8: HPET Core Block Diagram



hpet_core - main counter and NUM_TIMERS comparators (one timer drawn; core clock)
Register-Side Interface (hpet_config_regs, one-cycle write strobes)
in: hpet_enable, legacy_replacement, counter_write_lo / hi + data, timer_comp_write_lo / hi + data,
timer_int_enable, timer_type, timer_size (= ~32MODE), timer_int_clear
out: counter_rdata, timer_comp_rdata, timer_int_status
counter write LO / HI + data
/ counter_rdata
comparator write LO / HI + data
/ timer_comp_rdata (live)
Main Counter (flop)
r_main_counter[63:0]
+1 per clock while hpet_enable;
a write strobe loads its own 32-bit half
Comparator + Period (flop)
r_timer_comparator[i], r_timer_period[i]
a write loads both (the period mirrors
the last value software wrote)
comparator
timer_int_clear (W1C, per bit)
/ timer_int_status
counter
advanced value
(periodic, on fire or catch-up)
Match Compare (combinational)
counter >= comparator at the compare width
(timer_size: 64 or 32 bits), gated by the epoch bit
Periodic Advance (combinational)
comparator + period (catch-up: one
boundary per cycle until ahead; period 1:
counter + 1), epoch bit on carry
fire / catch-up
match
Armed Latch + Fire (flop)
r_timer_armed[i]: one interrupt owed per comparator value
fire = match & armed & hpet_enable
re-armed when match falls, on catch-up, or on a write while halted
fire
Interrupt Status + Output (flop)
status: set on fire (fire beats a clear), cleared by timer_int_clear
output: set on fire & timer_int_enable, follows status down
interrupt output
Legacy Replacement Route (combinational)
legacy_replacement=1: timer 0 -> legacy_irq0, timer 1 -> legacy_irq8,
both masked off timer_irq; status unaffected
timer_irq, legacy_irq0 / 8
Module Boundary
timer_irq[NUM_TIMERS-1:0] | legacy_irq0 (IRQ0) | legacy_irq8 (IRQ8)
Key
flow
heavy
return
sideband
logic
registered
interface

HPET Core Block Diagram

HPET Core architecture showing main counter, timer comparators, match detection, and interrupt generation.

8.1.2 Key Features

- **64-bit Free-Running Counter**: Increments every HPET clock cycle, provides timestamp base
- **Configurable Timer Array**: 2, 3, or 8 independent timers (compile-time parameter)
- **64-bit Comparators**: Per-timer comparison values with full counter range
- **Dual Operating Modes**: One-shot and periodic modes per timer
- **Automatic Period Reload**: Periodic mode auto-increments comparator after each fire
- **Individual Interrupts**: Separate fire flag and interrupt output per timer
- **Counter Read/Write Access**: Software can read and write the counter via the config registers, one 32-bit half per write

---

8.2 Parameters

Parameter	Type	Default	Range	Description
`NUM_TIMERS`	int	3	2, 3, 8	Number of independent timers in array (the `apb4_hpet` top-level default is 2)

8.3 Ports

8.3.1 Clock and Reset

Signal Name	Type	Width	Direction	Description
clk	logic	1	Input	Core clock (hpet_clk or pclk, selected by CDC_ENABLE at the top level)
rst_n	logic	1	Input	Active-low asynchronous reset

8.3.2 Configuration Interface (from hpet_config_regs)

Signal Name	Type	Width	Direction	Description
hpet_enable	logic	1	Input	Global HPET enable (from HPET_CONFIG[0])
counter_write_lo	logic	1	Input	One-cycle strobe: load counter[31:0] from counter_wdata[31:0]
counter_write_hi	logic	1	Input	One-cycle strobe: load counter[63:32] from counter_wdata[63:32]
counter_wdata	logic	64	Input	Counter write data (from HPET_COUNTER_LO/HI; only the strobed half is used)
legacy_replacement	logic	1	Input	LegacyReplacement Route (from HPET_CONFIG[1]): timers 0/1 leave on legacy_irq0/legacy_irq8 instead of timer_irq
timer_int_enable[NUM_TIMERS-1:0]	logic	NUM_TIMERS	Input	Per-timer interrupt enable (from TIMER_CONFIG[2], TN_CONF.INT_ENB). Gates only the interrupt output; there is NO per-

Signal Name	Type	Width	Direction	Description
				timer run enable
timer_type[NUM_TIMERS-1:0]	logic	NUM_TIMERS	Input	Per-timer mode: 0=One-shot, 1=Periodic (from TIMER_CONFIG[3])
timer_size[NUM_TIMERS-1:0]	logic	NUM_TIMERS	Input	Per-timer compare width: 0=32-bit, 1=64-bit. Driven as `~TIMER_CONFIG[8]` (32MODE forces 32-bit, so the polarity is inverted); a change clears the timer's next-epoch hold bit – change it only with the counter halted, then rewrite the comparator
timer_comp_write_lo[NUM_TIMERS-1:0]	logic	NUM_TIMERS	Input	Per-timer strobe: load comparator[31:0]; re-arms only while the counter is halted
timer_comp_write_hi[NUM_TIMERS-1:0]	logic	NUM_TIMERS	Input	Per-timer strobe: load comparator[63:32]; re-arms only while the counter is halted
timer_comp_wdata[NUM_TIMERS]	logic [63:0]	NUM_TIMERS x 64	Input	Per-timer comparator write data

8.3.3 Status Interface (to hpet_config_regs)

Signal Name	Type	Width	Direction	Description
counter_rdata	logic	64	Output	Current main counter value (to HPET_COUNTER_LO/HI)
timer_comp_rdata[NUM_TIMERS]	logic [63:0]	NUM_TIMERS x 64	Output	Live comparator, periodic advance included (to TIMER_COMPARATOR_LO/HI readback)
timer_int_status[NUM_TIMERS-1:0]	logic	NUM_TIMERS	Output	Per-timer sticky interrupt status, owned here (mirrored into HPET_STATUS)
timer_int_clear[NUM_TIMERS-1:0]	logic	NUM_TIMERS	Input	Per-bit one-cycle clear pulses from the register wrapper (W1C)

8.3.4 Interrupt Interface

Signal Name	Type	Width	Direction	Description
timer_irq[NUM_TIMERS-1:0]	logic	NUM_TIMERS	Output	Per-timer interrupt outputs (active-high); timers 0/1 suppressed while legacy_replacement is set
legacy_irq0	logic	1	Output	Timer 0 on the LegacyReplacement Route (IRQ0); 0 unless legacy_replacement is set

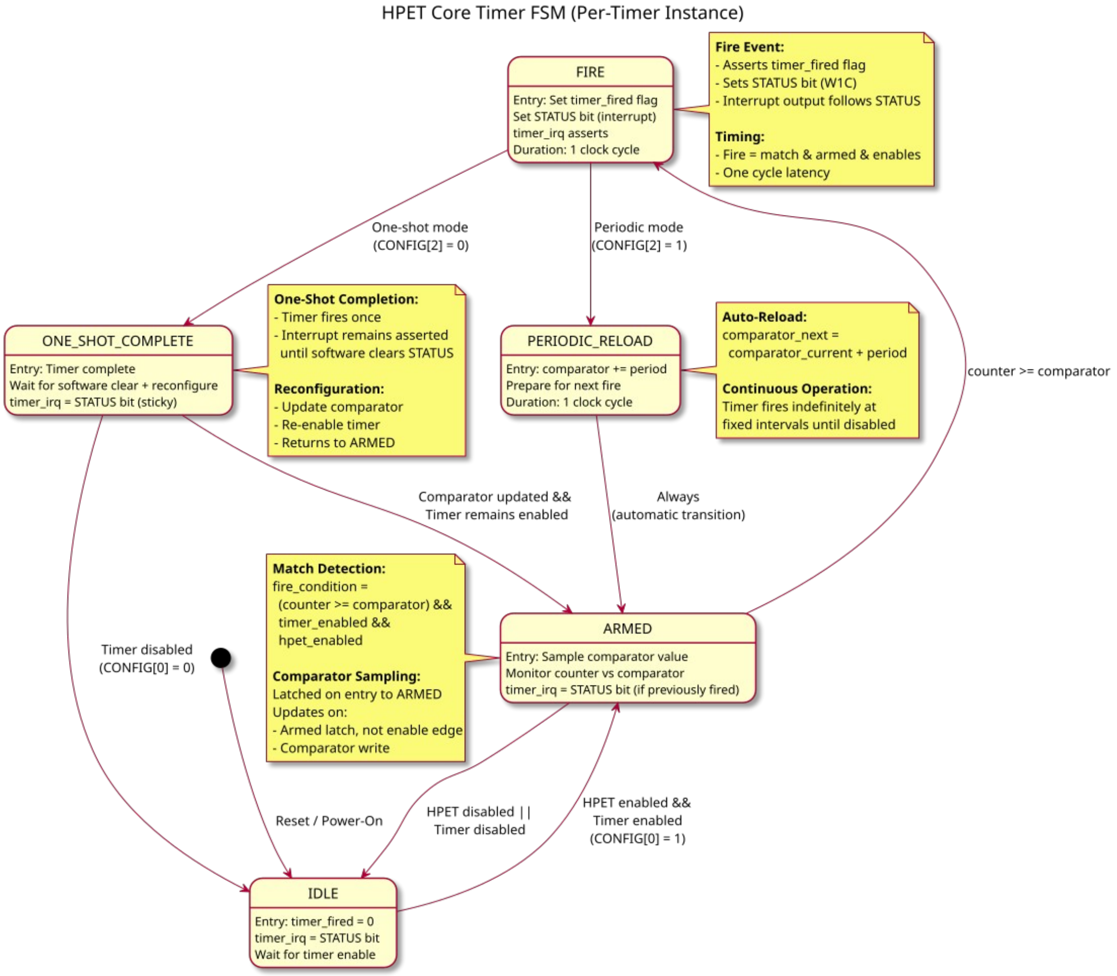
Signal Name	Type	Width	Direction	Description
legacy_irq8	logic	1	Output	Timer 1 on the LegacyReplacement Route (IRQ8); 0 unless legacy_replacement is set

8.4 Functional Description

8.4.1 Per-Timer State Machine

Each timer instance implements an identical FSM controlling its operation:

8.4.2 Figure 2.9: Timer FSM


HPET Core Timer FSM (Per-Timer Instance)
FIRE
Entry: Set timer_fired flag
Set STATUS bit (interrupt)
timer_irq asserts
Duration: 1 clock cycle
Fire Event:
- Asserts timer_fired flag
- Sets STATUS bit (W1C)
- Interrupt output follows STATUS
Timing:
- Fire = match & armed & enables
- One cycle latency
One-shot mode
(CONFIG[2] = 0)
Periodic mode
(CONFIG[2] = 1)
ONE_SHOT_COMPLETE
Entry: Timer complete
Wait for software clear + reconfigure
timer_irq = STATUS bit (sticky)
One-Shot Completion:
- Timer fires once
- Interrupt remains asserted
until software clears STATUS
Reconfiguration:
- Update comparator
- Re-enable timer
- Returns to ARMED
PERIODIC_RELOAD
Entry: comparator += period
Prepare for next fire
Duration: 1 clock cycle
Auto-Reload:
comparator_next =
comparator_current + period
Continuous Operation:
Timer fires indefinitely at
fixed intervals until disabled
counter >= comparator
Comparator updated &&
Timer remains enabled
Always
(automatic transition)
Match Detection:
fire_condition =
(counter >= comparator) &&
timer_enabled &&
hpet_enabled
Comparator Sampling:
Latched on entry to ARMED
Updates on:
- Armed latch, not enable edge
- Comparator write
ARMED
Entry: Sample comparator value
Monitor counter vs comparator
timer_irq = STATUS bit (if previously fired)
Timer disabled
(CONFIG[0] = 0)
Reset / Power-On
HPET disabled ||
Timer disabled
HPET enabled &&
Timer enabled
(CONFIG[0] = 1)
IDLE
Entry: timer_fired = 0
timer_irq = STATUS bit
Wait for timer enable

Timer FSM

8.4.2.1 FSM States

State	Encoding	Description
IDLE	Default	Counter halted (`hpet_enable = 0`), waiting for it to run
ARMED	Active	Timer enabled, monitoring counter vs comparator
FIRE	Transient	Timer match detected, asserting interrupt (1 cycle)
PERIODIC_RELOAD	Transient	Periodic mode: auto-increment comparator (1 cycle; one boundary per cycle while

State	Encoding	Description
		catching up, counter + 1 at period 1)
ONE_SHOT_COMPLETE	Sticky	One-shot mode: fired, armed latch clear, waiting for a comparator write with the counter halted or for the match to fall

Note: FSM is **conceptual** - the implementation is a raw comparator plus two state bits per timer, the armed latch (`r_timer_armed[i]`) and the next-epoch hold bit (`r_comp_next_epoch[i]`), rather than an explicit state register. A timer whose advance carried out of the compare width is epoch-held: it looks ARMED-but-not-matching (the latch is set, the match is forced off) until the counter wraps at that width, and none of the transitions below happen for it in the meantime.

8.4.2.2 State Transitions

IDLE -> ARMED: - Condition: `hpet_enable` (block-wide – the published spec defines no per-timer run enable, so every timer runs whenever the counter does) - Action: Latch current comparator value - Duration: Immediate (next clock cycle)

ARMED -> FIRE: - Condition: `counter_value >= timer_comparator[i]` with the armed latch set - Action: Assert `timer_int_status[i]` (sticky) on EVERY fire, whatever `timer_int_enable[i]` says; clear the armed latch - Duration: 1 clock cycle (the fire is a one-cycle pulse)

FIRE -> PERIODIC_RELOAD: - Condition: `timer_type[i] == 1` (periodic mode) - Action: `timer_comparator[i] <= timer_comparator[i] + timer_period[i]` - Duration: 1 clock cycle

FIRE -> ONE_SHOT_COMPLETE: - Condition: `timer_type[i] == 0` (one-shot mode) - Action: Hold `timer_int_status[i]` until software clears - Duration: Until STATUS cleared or the counter halted

PERIODIC_RELOAD -> ARMED: - Condition: Always (automatic) - Action: Resume monitoring with new comparator value - Duration: Immediate

ONE_SHOT_COMPLETE -> ARMED: - Condition: a comparator write, LO or HI half (`timer_comp_write_lo/hi[i]`), while the counter is HALTED (`hpet_enable = 0`), even with the value it already holds; or the raw match falling (a comparator written above the counter, or the counter written back below the comparator) - Action: Set the armed latch; resume monitoring - Duration: Immediate on the write strobe or on the match falling - A comparator written with the counter halted does NOT have to exceed the counter. A value at or below the counter fires on the first cycle the counter runs – that is the explicit re-arm, and it is how software fires a timer “now”. - A comparator write on a RUNNING timer loads the half but does not re-arm. Written above the counter, the match falls and the timer re-arms naturally, firing at the new value. Anything else is outside the contract: a torn 64-bit value above the counter re-arms through that same path and fires at the final value once it is written, and a same-value rewrite on a running periodic timer drags the comparator back from its advanced position and can add one off-lattice fire. To reprogram a running timer, halt the counter first.

ARMED/ONE_SHOT_COMPLETE -> IDLE: - Condition: `!hpet_enable` (clearing `timer_int_enable[i]` does NOT stop a timer; it only masks the interrupt) - Action: Stop firing. A pending interrupt status/output is NOT cleared by halting – it holds until software W1C or reset. - Duration: Immediate - The enable does not touch the armed latch, so a completed one-shot stays complete across a halt/restart cycle: restarting with an expired comparator does not re-fire it.

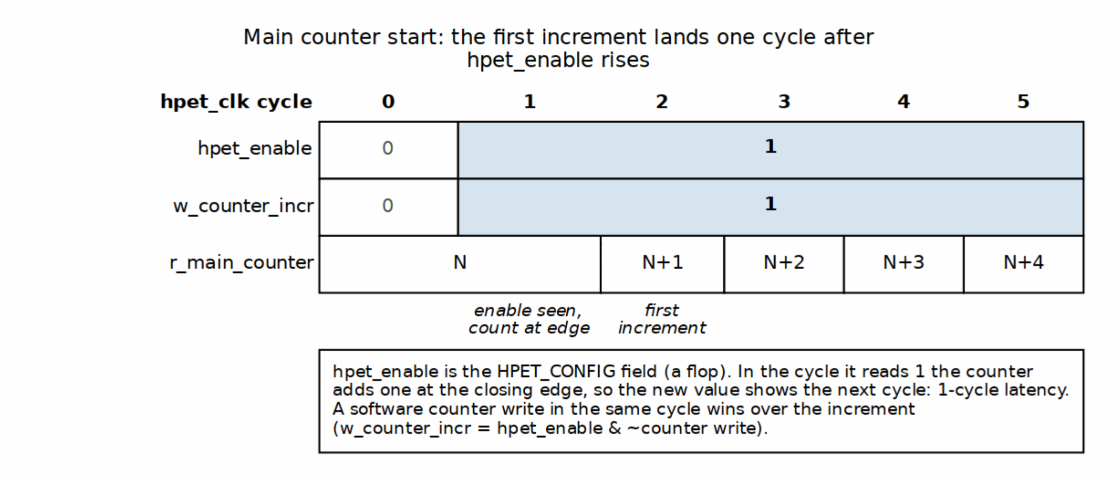
8.4.3 Main Counter Logic

8.4.3.1 Counter Increment

```
// 64-bit free-running counter
logic [63:0] r_main_counter;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_main_counter <= 64'h0;
    end else if (counter_write_lo || counter_write_hi) begin
        // Software write: each half loads on its own strobe, the
other
        // half holds, and the write cycle does not increment
        if (counter_write_lo) r_main_counter[31:0]  <=
counter_wdata[31:0];
        if (counter_write_hi) r_main_counter[63:32] <=
counter_wdata[63:32];
    end else if (hpet_enable) begin
        // Continuous increment when HPET enabled
        r_main_counter <= r_main_counter + 64'd1;
    end
    // else: Hold value when HPET disabled
end

// Output current counter value
assign counter_rdata = r_main_counter;
```

Key Behavior: - **Reset**: Counter initializes to 0 - **Software Write**: each HPET_COUNTER_LO/HI write loads its own half on the write itself; the halves are independent, as on a real HPET, so a 64-bit load is two writes and the intermediate state is visible – write with the counter halted (`hpet_enable = 0`) - **Increment**: Counter increments every clock when `hpet_enable = 1` - **Overflow**: Counter wraps from 64’hFFFF_FFFF_FFFF_FFFF to 64’h0 naturally

8.4.4 Figure 2.10: Main Counter Start Timing


Main counter start: the first increment lands one cycle after hpet_enable rises
hpet_clk cycle
0
1
2
3
4
5
hpet_enable
0
1
w_counter_incr
0
1
r_main_counter
N
N+1
N+2
N+3
N+4
enable seen, count at edge
first increment

hpet_enable is the HPET_CONFIG field (a flop). In the cycle it reads 1 the counter adds one at the closing edge, so the new value shows the next cycle: 1-cycle latency. A software counter write in the same cycle wins over the increment (w_counter_incr = hpet_enable & ~counter write).

Main Counter Start Timing

8.4.5 Timer Comparator Logic

8.4.5.1 Comparator Storage (Per-Timer)

```
// Per-timer comparator and period storage
logic [63:0] r_timer_comparator [NUM_TIMERS-1:0];
logic [63:0] r_timer_period [NUM_TIMERS-1:0];

for (genvar i = 0; i < NUM_TIMERS; i++) begin : gen_timer_advance
    // One advanced value, shared by the fire advance, the catch-up
    // advance, the ahead-of-counter test and the epoch carry. A
period-1
    // catch-up step jumps to counter + 1 (every integer is on the
    // period-1 lattice); everything else is comparator + period.
    assign w_adv_base[i]   = (w_timer_catchup[i] &&
w_timer_period_one[i]) ?
                             r_main_counter : r_timer_comparator[i];
    assign w_adv_addend[i] = (w_timer_catchup[i] &&
w_timer_period_one[i]) ?
                             64'd1 : r_timer_period[i];
    assign w_adv_sum_64[i] = {1'b0, w_adv_base[i]} + {1'b0,
w_adv_addend[i]};
    assign w_adv_sum_32[i] = {1'b0, w_adv_base[i][31:0]} + {1'b0,
w_adv_addend[i][31:0]};

    // {carry, sum} at the compare width. The carry is what the epoch
bit
    // records; in 32-bit mode the advance never touches
comparator[63:32].
    always_comb begin
```

```
        if (timer_size[i]) begin
            w_comp_advance[i]   = w_adv_sum_64[i][63:0];
            w_comp_adv_carry[i] = w_adv_sum_64[i][64];
            w_comp_adv_ahead[i] = w_adv_sum_64[i][64] ||
                                  (w_adv_sum_64[i][63:0] >=
r_main_counter);
        end else begin
            w_comp_advance[i]   = {r_timer_comparator[i][63:32],
w_adv_sum_32[i][31:0]};
            w_comp_adv_carry[i] = w_adv_sum_32[i][32];
            w_comp_adv_ahead[i] = w_adv_sum_32[i][32] ||
                                  (w_adv_sum_32[i][31:0] >=
r_main_counter[31:0]);
        end
    end
end

// An advance happens on a periodic fire or a catch-up step, never in
a
// cycle software is writing the comparator (software wins, below)
assign w_comp_advance_en = ~w_timer_comp_write &
                           ((w_timer_fire & timer_type) |
w_timer_catchup);

for (genvar i = 0; i < NUM_TIMERS; i++) begin : gen_timer_comparators
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_timer_comparator[i] <= 64'h0;
            r_timer_period[i] <= 64'h0;
        end else if (timer_comp_write_lo[i] || timer_comp_write_hi[i])
begin
            // Software write: one 32-bit half per strobe. Period
mirrors
            // the written value so periodic mode advances by the
            // programmed interval while the comparator walks ahead of
it.
            // SOFTWARE WINS: this branch outranks both advances
below, so
            // a write landing in a fire (or catch-up) cycle loads the
            // software value and the advance is skipped that cycle.
            if (timer_comp_write_lo[i]) begin
                r_timer_comparator[i][31:0]  <= timer_comp_wdata[i]
[31:0];
                r_timer_period[i][31:0]      <= timer_comp_wdata[i]
[31:0];
            end
            if (timer_comp_write_hi[i]) begin
                r_timer_comparator[i][63:32] <= timer_comp_wdata[i]
```

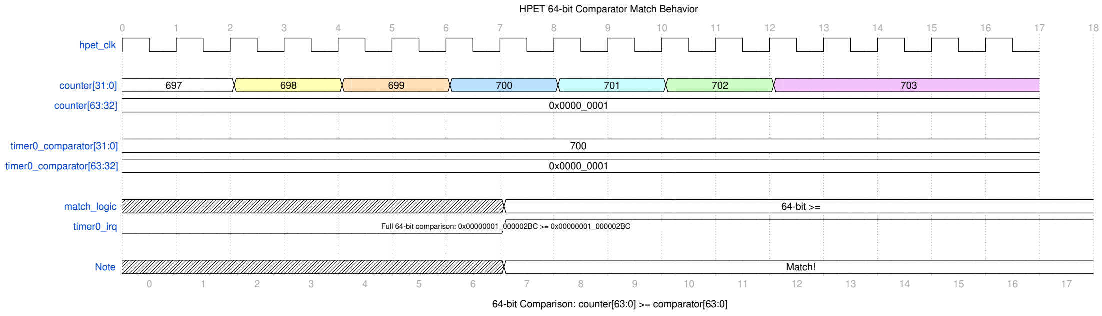
```
[63:32];
                r_timer_period[i][63:32]     <= timer_comp_wdata[i]
[63:32];
            end
        end else if (w_comp_advance_en[i]) begin
            // Periodic advance: one period per fire, or one boundary
per
            // CYCLE while catching up (see Fire Detection). Width-
masked;
            // the carry goes to the epoch bit, not the comparator
            r_timer_comparator[i] <= w_comp_advance[i];
        end
        // else: Hold value
    end
end
```

Key Behavior: - **Reset**: Comparator and period clear to 0 - **Initial Write**: Both comparator and period latched from the same write, one half per strobe - **Rewrite**: a write is the event, not a change of value – writing the comparator with the value it already holds reloads the internal comparator (discarding any periodic advance). It re-arms the timer only while the counter is halted; on a running timer the reload lands and the timer re-arms when the match next falls - **Periodic Mode**: Comparator auto-increments by period value on each fire - **Periodic catch-up**: while a periodic timer is running, un-armed and its comparator is still at or below the counter, the comparator keeps advancing by one period per cycle WITHOUT firing until it is at or ahead of the counter, then the timer re-arms (see Fire Detection) - **Period 1**: a catch-up step sets the comparator to counter + 1, not comparator + 1 – adding 1 to both sides can never close a gap, and every integer is on the period-1 lattice - **Catch-up cost**: one comparator advance per cycle, so a catch-up closes its deficit at period - 1 counts per cycle (period 1 is the special lattice jump that closes in one) and a large deficit costs a proportional catch-up – a period-2 timer left 2^52 counts behind by a counter write rewrites its comparator every cycle for ~2^52 cycles with no interrupt. This is the contract behind halting the counter before writing it, and behind programming a periodic comparator near or ahead of the counter - **Advance overflow**: an advance that carries out of the compare width keeps the wrapped low bits and sets the next-epoch hold bit; in 32-bit mode bits [63:32] are never disturbed - **Same-cycle write and fire**: exactly one fire; the software value wins and the periodic advance is skipped that cycle - **One-Shot Mode**: Comparator remains constant after initial write

8.4.5.2 Match Detection

64-bit Comparator Match Waveform:

8.4.6 Waveform 2.11: Comparator Match Behavior


HPET 64-bit Comparator Match Behavior
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
hpet_clk
counter[31:0]
697
698
699
700
701
702
703
counter[63:32]
0x0000_0001
timer0_comparator[31:0]
700
timer0_comparator[63:32]
0x0000_0001
match_logic
64-bit >=
timer0_irq
Full 64-bit comparison: 0x00000001_000002BC >= 0x00000001_000002BC
Note
Match!
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
64-bit Comparison: counter[63:0] >= comparator[63:0]

Comparator Match Behavior

Use WaveDrom Editor to view/edit, or generate SVG with `wavedrom-cli`

```
// Per-timer match detection (combinational, RAW -- no enable terms)
logic [NUM_TIMERS-1:0] w_timer_match_raw;
logic [NUM_TIMERS-1:0] w_timer_match;

for (genvar i = 0; i < NUM_TIMERS; i++) begin : gen_timer_terms
    // timer_size (= ~TIMER_CONFIG[8], 32MODE) selects 64-bit or 32-
bit compare
    assign w_timer_match_raw[i] = timer_size[i]
                                  ? (r_main_counter >=
r_timer_comparator[i])
                                  : (r_main_counter[31:0] >=
r_timer_comparator[i][31:0]);
end

// Held off while the comparator sits in the counter's NEXT epoch.
Every
// consumer (fire, catch-up, natural re-arm) reads this gated match.
assign w_timer_match = w_timer_match_raw & ~r_comp_next_epoch;
```

Match Conditions: - Counter value >= comparator value (full 64 bits when timer_size=1, low 32 bits only when timer_size=0) - And the next-epoch hold bit is clear. When an advance has carried out of the compare width the comparator belongs to the counter's next epoch, and the raw comparison says nothing useful until the counter wraps at that width; the bit forces the match off until then. - That is all. The enable (`hpet_enable`) is deliberately NOT in the match: it gates the fire pulse instead (next section), so that turning it on cannot manufacture a match edge.

8.4.7 Timer Fire Logic

8.4.7.1 Fire Detection (Armed Latch)

A per-timer armed latch decides whether the raw match may still produce a fire. It holds "this timer still owes one interrupt for this comparator" as state, which is what a plain rising-edge detect cannot do: an edge on (match && hpet_enable) re-fires every expired one-shot the moment the counter is started, while an edge on the raw match is consumed at the comparator write, before the counter runs, and then never fires at all.

```
// Per-timer armed latch and next-epoch hold bit. Reset ARMED: out of
// reset counter == comparator == 0 is already a match, and
hpet_enable
// holds the fire off until then. Reset epoch clear.
logic [NUM_TIMERS-1:0] r_timer_armed;
logic [NUM_TIMERS-1:0] r_comp_next_epoch;
logic [NUM_TIMERS-1:0] w_timer_comp_write;   // LO or HI half of timer
i written
logic [NUM_TIMERS-1:0] w_timer_running;      // = hpet_enable (block-
wide)
logic [NUM_TIMERS-1:0] w_comp_write_rearm;   // comparator write with
the counter HALTED
logic [NUM_TIMERS-1:0] w_timer_catchup;      // advance a boundary, do
NOT fire
logic [NUM_TIMERS-1:0] w_catchup_rearm;      // catch-up landed at or
ahead of the counter

assign w_timer_comp_write = timer_comp_write_lo | timer_comp_write_hi;
assign w_timer_running    = {NUM_TIMERS{hpet_enable}};
assign w_comp_write_rearm = w_timer_comp_write & ~w_timer_running;

// Periodic catch-up: periodic, NOT armed, comparator still at or
behind
// the counter (epoch-gated match), no software write this cycle, a
// period that can actually move the compare (period 0 behaves as
// one-shot), and running.
assign w_timer_catchup = timer_type & ~r_timer_armed & w_timer_match &
                         ~w_timer_comp_write & w_timer_period_nz &
                         w_timer_running;

// Finished the cycle the advance lands AT OR AHEAD of the counter
// (w_comp_adv_ahead[i] = carry || advance >= counter, at the compare
// width; see gen_timer_advance). >= not >: the comparator takes the
// advance while the counter takes counter+1, so a tie is a boundary
one
// count behind next cycle, and at period 1 the advance can only tie.
assign w_catchup_rearm = w_timer_catchup & w_comp_adv_ahead;

// Next-epoch hold: set when an advance carries out of the compare
width;
// cleared, with priority, by the counter wrapping at that width (E1),
a
// comparator write (E2), a counter write (E3) or a timer_size change
```

```
(E4).
// E2 and E3 are taken at the compare width, in the gen_timer_terms
mux
// alongside the match: with timer_size = 0 only a LO-half write
clears
// (w_epoch_clr_comp = timer_comp_write_lo, w_epoch_clr_ctr =
// counter_write_lo); a HI-half write moves no bit the 32-bit
comparison
// reads, so it leaves the hold in place. With timer_size = 1 either
half
// clears.
assign w_epoch_set = w_comp_advance_en & w_comp_adv_carry;
assign w_epoch_clr = w_counter_wrap | w_epoch_clr_comp |
w_epoch_clr_ctr |
                     w_timer_size_chg;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_comp_next_epoch <= '0;
    end else begin
        for (int t = 0; t < NUM_TIMERS; t++) begin
            if (w_epoch_clr[t])      r_comp_next_epoch[t] <= 1'b0;
            else if (w_epoch_set[t]) r_comp_next_epoch[t] <= 1'b1;
        end
    end
end

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_timer_armed <= {NUM_TIMERS{1'b1}};
    end else begin
        for (int t = 0; t < NUM_TIMERS; t++) begin
            if (w_comp_write_rearm[t] || !w_timer_match[t] ||
                w_catchup_rearm[t]) begin
                r_timer_armed[t] <= 1'b1;   // explicit, natural or
catch-up re-arm
            end else if (w_timer_fire[t]) begin
                r_timer_armed[t] <= 1'b0;   // one fire per arm
            end
        end
    end
end

// Fire: epoch-gated match, still armed, and the counter running
assign w_timer_fire = w_timer_match & r_timer_armed &
                      {NUM_TIMERS{hpet_enable}};
```

The latch is set by exactly three things and cleared only by a fire:

- **Natural re-arm** – the gated match is LOW: the counter is behind the comparator again after a periodic advance, a comparator write that moved the target forward, or a counter reload – or the next-epoch hold bit is forcing the match off.
- **Catch-up re-arm** – a periodic catch-up step (below) lands the comparator at or ahead of the counter (>=, or a carry out of the compare width). This is the only re-arm path for a period of 1, where the advance never pulls away from the counter and the match therefore never falls on its own.
- **Explicit re-arm** – a comparator half is written while the counter is HALTED (`hpet_enable = 0`; there is no per-timer stop). This is what lets software arm to a value the counter has already passed: a zero comparator, or restarting a periodic phase after a counter reset.

A comparator write on a RUNNING timer deliberately does NOT re-arm. The 64-bit comparator arrives as two 32-bit halves, so between them the register holds the torn value {old HI, new LO}; a re-arm on either strobe would fire on that value, which software never programmed. The half still loads – only the re-arm is withheld – so a running timer re-arms naturally, through the first rule, once the completed value is ahead of the counter. Written above the counter it fires at the new value.

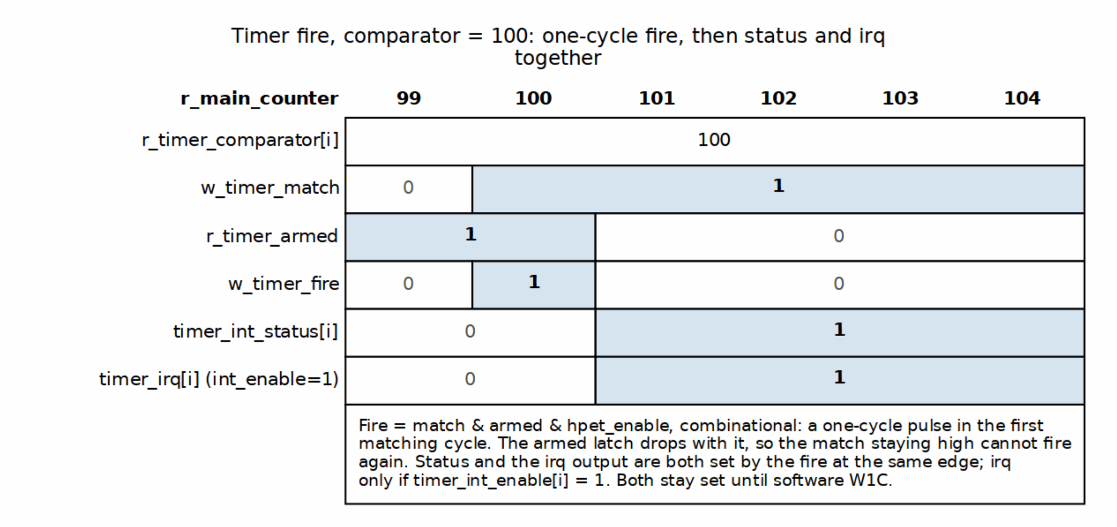
The contract that follows: to reprogram a running timer's 64-bit comparator without a spurious or a missed event, halt the counter first (`hpet_enable = 0` – there is no per-timer stop), write both halves, then restart it – it fires once, at the value programmed, and a value at or below the counter fires as soon as the counter restarts (the >= match, the same rule that makes a deficit fire rather than be missed; there is no wait for a wrap). The real HPET has the same requirement. Two consequences of reprogramming a running timer anyway are outside the contract, not defects: a torn 64-bit value ABOVE the counter re-arms the timer through the natural path and it fires at the final value once the second half is written – the completed value fires it, not the tear – and a same-value rewrite on a running periodic timer drags the comparator back from its auto-advanced position and can add one off-lattice fire.

Periodic catch-up. A fire advances a periodic comparator by one period. If the counter was already more than a period past it – programmed late, or the counter written forward – one advance is not enough: the comparator is still at or below the counter, the match never falls, and with only the natural re-arm the timer would fire once and go silent. So a periodic timer that is running, un-armed and still matching advances its comparator by the period every cycle WITHOUT firing until the advance lands at or ahead of the counter; the catch-up re-arm then sets the latch and the timer fires at that next boundary. The test is >=, not >: the comparator takes the advance in the same cycle the counter takes counter+1, so an advance that ties with the counter is one count behind it next cycle and a legal boundary to fire at. Missed periods are skipped, never burst: a large deficit yields exactly one fire, at the next boundary still in the future. Two degenerate periods are worth knowing. Period 0 behaves as one-shot – a zero period can never get ahead of the counter, so the catch-up is suppressed and the timer fires once and stays quiescent rather than rewriting its own value forever. Period 1 gains nothing on the counter per cycle – both step by exactly 1, so the deficit is an invariant and comparator + period could never close it – so a period-1 catch-up step sets the comparator to counter + 1 instead. Every integer is on the period-1 lattice, so that is a legal next boundary, and it re-arms immediately. The cadence is one fire every other tick, the fastest the one-cycle fire/re-arm loop allows, regardless of how large the deficit is; a live counter write that opens a gap recovers on the next catch-up cycle. Any other period pays for its deficit in cycles. The comparator advances once per cycle, so a catch-up closes the gap at period - 1 counts per cycle – period 1 is the special lattice jump that closes it in one – and the time to close is proportional to the deficit. A period-2 timer left 2^52 counts behind by a counter write catches up for ~2^52 cycles, the comparator rewritten every cycle and no interrupt raised. That bound is a contract, not a defect: it is why the HPET specification and this book require the main counter to be HALTED (`HPET_CONFIG.hpet_enable = 0`) before it is written, and why a periodic timer should be programmed with a comparator near or ahead of the counter.

Next-epoch hold. An advance – on a fire or a catch-up step – can carry out of the compare width: past bit 31 in 32-bit mode, past bit 63 in 64-bit mode. The comparator keeps the wrapped low bits and now belongs to the counter's NEXT epoch, where a plain `counter >= comparator` would read as an immediate match and either fire early or churn the catch-up. The core records the carry in a per-timer hold bit, `r_comp_next_epoch[i]` (reset 0), that forces the gated match to 0: no fire, no catch-up, no rewrite loop, and the armed latch simply re-sets through the natural rule and waits. The bit clears when the counter wraps at the compare width by incrementing (E1 – a software counter write is not a wrap), when software writes the comparator (E2), when software writes the counter (E3 – the write re-bases the epoch, so the ordinary `counter >= comparator` rule applies immediately and a comparator now behind the counter fires promptly), or when `timer_size[i]` changes (E4 – the carry was taken at a width that no longer applies). E2 and E3 are evaluated at the compare width, the same width the carry was taken at. In 32-bit mode (`timer_size[i] = 0`) only a write to HPET_COUNTER_LO or TIMERn_COMPARATOR_LO clears the hold; a write to either HI half changes nothing the comparison reads and leaves the hold in place. In 64-bit mode either half clears it. Clear beats set, because the two coincide exactly at the counter's own wrap and taking the set there would hold the comparator for a whole extra epoch. E4 carries a contract of its own: change `timer_size` only with the counter halted, and rewrite the comparator afterwards. The hold clears on the change, but a comparator that was advanced at the old width is not on the new width's lattice, and a live 0->1 switch can leave the timer catching up for ~2^32 cycles. What software sees: a 32-bit periodic timer whose comparator + period passes 2^32 fires at that boundary and then stays quiet until the low 32 bits of the counter wrap – up to about 2^32 ticks, though never longer than the period that caused the carry, so the next fire still lands on the programmed lattice; a 64-bit timer whose advance carries out of bit 63 waits for the 64-bit counter to wrap – up to 2^64 counts, roughly 5800 years at 100 MHz – so in practice it fires once and never again until reprogrammed. Both are the arithmetic of the register width, not defects, and in 32-bit mode the advance never disturbs comparator[63:32].

Same-cycle write and fire. A software comparator write landing in the same cycle as a fire (or a catch-up step) on a running timer produces exactly one fire; the software value wins and the periodic advance is skipped that cycle, so the comparator holds what software programmed, not that value plus a period.

8.4.8 Figure 2.12: Timer Fire Timing


Timer fire, comparator = 100: one-cycle fire, then status and irq together
r_main_counter
99
100
101
102
103
104
r_timer_comparator[i]
100
w_timer_match
0
1
r_timer_armed
1
0
w_timer_fire
0
1
0
timer_int_status[i]
0
1
timer_irq[i] (int_enable=1)
0
1

Fire = match & armed & hpet_enable, combinational: a one-cycle pulse in the first matching cycle. The armed latch drops with it, so the match staying high cannot fire again. Status and the irq output are both set by the fire at the same edge; irq only if timer_int_enable[i] = 1. Both stay set until software W1C.

Timer Fire Timing

8.4.8.1 Fire Flag Management

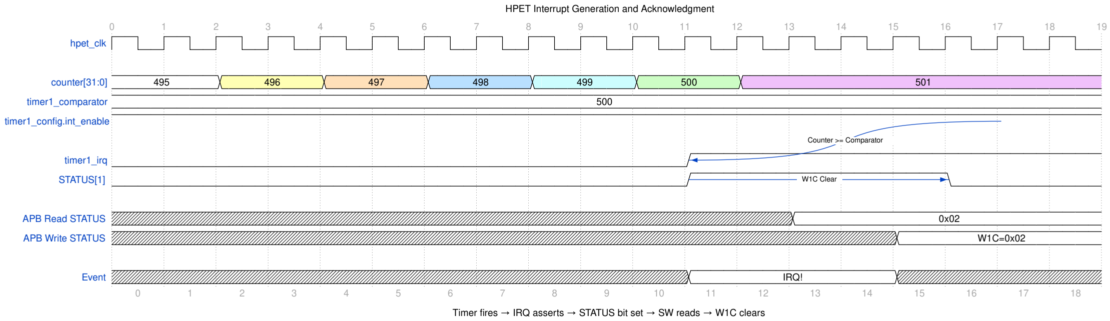
```
// Per-timer sticky interrupt status -- identical in BOTH modes
logic [NUM_TIMERS-1:0] r_interrupt_status;

for (genvar i = 0; i < NUM_TIMERS; i++) begin : gen_interrupt_logic
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_interrupt_status[i] <= 1'b0;
        end else if (w_timer_fire[i]) begin
            r_interrupt_status[i] <= 1'b1;   // Set on fire (wins over a clear)
        end else if (timer_int_clear[i]) begin
            r_interrupt_status[i] <= 1'b0;   // Software W1C, this bit only
        end
    end
end

// Sticky status to the register wrapper (mirrored into HPET_STATUS)
assign timer_int_status = r_interrupt_status;
```

Fire Flag Behavior: - **Both modes are sticky**: the status bit sets on the fire and holds until software clears it via HPET_STATUS W1C (or reset). There is no `timer_type` term in the interrupt logic – periodic mode does NOT pulse the status per period; from the first fire it stays asserted until W1C, while the comparator keeps auto-advancing in the background. - **Fire beats clear**: a fire and a software clear landing on the same bit in the same cycle leave the bit SET. A clear can be repeated; a lost fire is gone for good, so the new event takes priority. The wrapper narrows the clear to one cycle so there is no trailing cycle that could undo a fire the core just accepted. - **Per bit**: `timer_int_clear[i]` touches only timer i; the other timers’ status and irq are unaffected.

8.4.9 Interrupt Generation

Interrupt Generation and Acknowledgment Waveform:

8.4.10 Waveform 2.13: Interrupt Generation


HPET Interrupt Generation and Acknowledgment
hpet_clk
counter[31:0]
495
496
497
498
499
500
501
timer1_comparator
500
timer1_config.int_enable
Counter >= Comparator
timer1_irq
STATUS[1]
W1C Clear
APB Read STATUS
0x02
APB Write STATUS
W1C=0x02
Event
IRQ!
Timer fires → IRQ asserts → STATUS bit set → SW reads → W1C clears

Interrupt Generation

Use WaveDrom Editor to view/edit, or generate SVG with `wavedrom-cli`

8.4.10.1 Interrupt Output Logic

```
// Per-timer interrupt output -- a flop, gated by int_enable AT FIRE TIME
for (genvar i = 0; i < NUM_TIMERS; i++) begin : gen_interrupt_output
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_interrupt_output[i] <= 1'b0;
        end else if (w_timer_fire[i] && timer_int_enable[i]) begin
            r_interrupt_output[i] <= 1'b1;   // Fire wins over a same-cycle clear
        end else if (timer_int_clear[i]) begin
            r_interrupt_output[i] <= 1'b0;
        end else if (!r_interrupt_status[i]) begin
            r_interrupt_output[i] <= 1'b0;   // Follows status clear
        end
    end
end

// LegacyReplacement Route REPLACES timers 0/1 on timer_irq (TASK-003)
assign timer_irq   = r_interrupt_output &
                     ~(legacy_replacement ? LEGACY_MASK : {NUM_TIMERS{1'b0}});
assign legacy_irq0 = legacy_replacement & r_interrupt_output[0];
assign legacy_irq8 = legacy_replacement & r_interrupt_output[1];  // 0 if NUM_TIMERS < 2
```

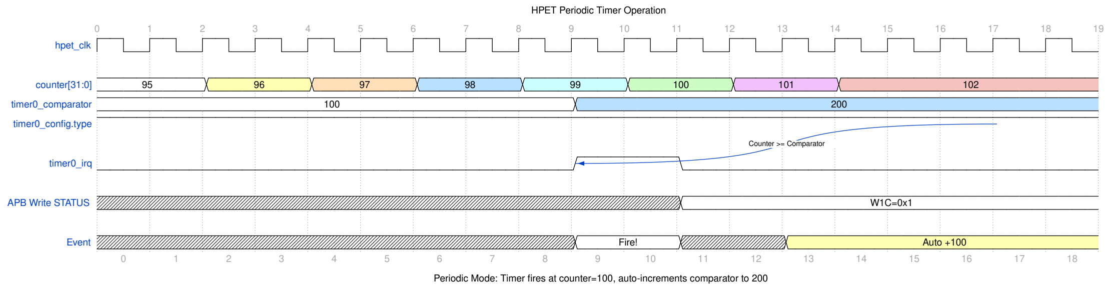
Interrupt Behavior: - **Registered**: `timer_irq` asserts one core clock after the fire event - **Gated at fire time**: `timer_int_enable[i]` is sampled only when the fire edge occurs. Enabling interrupts AFTER a timer has fired does not retroactively assert `timer_irq`, even though the status bit is set. - **Sticky in both modes**: asserted from (fire + 1 cycle) until the W1C

Interrupt Clearing: Software clears an interrupt by writing 1 to its HPET_STATUS bit (W1C). The sticky status lives in `hpet_core` (`r_interrupt_status`); the wrapper mirrors it into the PeakRDL HPET_STATUS register every cycle and turns the software write into a one-cycle `timer_int_clear[i]` pulse for exactly the bits written with 1. Bits written 0 – and a write of 0x0 – clear nothing.

8.4.11 Periodic Mode Details

Periodic Timer Waveform:

8.4.12 Waveform 2.14: Periodic Timer Operation


HPET Periodic Timer Operation
hpet_clk
counter[31:0]
95
96
97
98
99
100
101
102
timer0_comparator
100
200
timer0_config.type
Counter >= Comparator
timer0_irq
APB Write STATUS
W1C=0x1
Event
Fire!
Auto +100
Periodic Mode: Timer fires at counter=100, auto-increments comparator to 200

Periodic Timer Operation

Use WaveDrom Editor to view/edit, or generate SVG with `wavedrom-cli`

8.4.12.1 Period Storage and Auto-Reload

Initial Comparator Write:

```
Software writes: TIMER0_COMPARATOR = 1000
Result:
  r_timer_comparator[0] = 1000
  r_timer_period[0] = 1000  (also latched)
```

First Fire (at counter = 1000):

```
Armed match: timer fires
-> timer_int_status[0] asserts
-> Comparator auto-reloads:
  r_timer_comparator[0] = 1000 + 1000 = 2000
```

Second Fire (at counter = 2000):

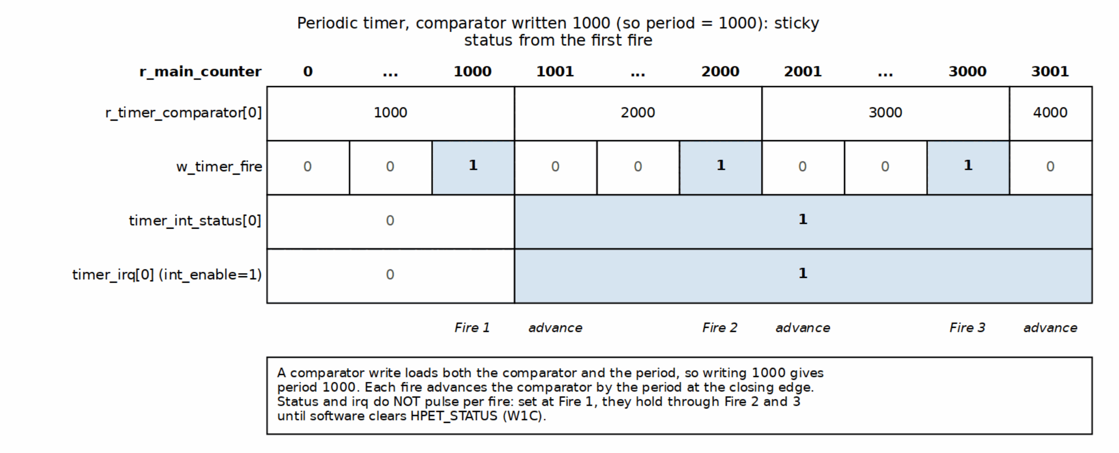
```
Armed match (re-armed when the advance dropped the match): timer fires
-> timer_int_status[0] asserts
-> Comparator auto-reloads:
  r_timer_comparator[0] = 2000 + 1000 = 3000
```

Process repeats indefinitely until the counter is halted or the timer is switched to one-shot

Late Start (counter already at 3500 when the timer is programmed):

1. An armed match with comparator = 1000 fires the timer once.
2. The comparator advances 1000 -> 2000; that is still <= 3500, so the timer stays un-armed.
3. Catch-up, no fire: 2000 -> 3000 (still behind), 3000 -> 4000 (ahead).
4. Catch-up re-arms; the next fire is at counter = 4000, then 5000, 6000, … The 2000 and 3000 boundaries are skipped, not burst.

8.4.13 Figure 2.15: Periodic Mode Timing Example



Periodic timer, comparator written 1000 (so period = 1000): sticky status from the first fire

r_main_counter	**0**	**…**	**1000**	**1001**	**…**	**2000**	**2001**	**…**	**3000**	**3001**
r_timer_comparator[0]	1000			2000			3000			4000
w_timer_fire	0	0	**1**	0	0	**1**	0	0	**1**	0
timer_int_status[0]	0			**1**						
timer_irq[0] (int_enable=1)	0			**1**						
			Fire 1	*advance*		*Fire 2*	*advance*		*Fire 3*	*advance*

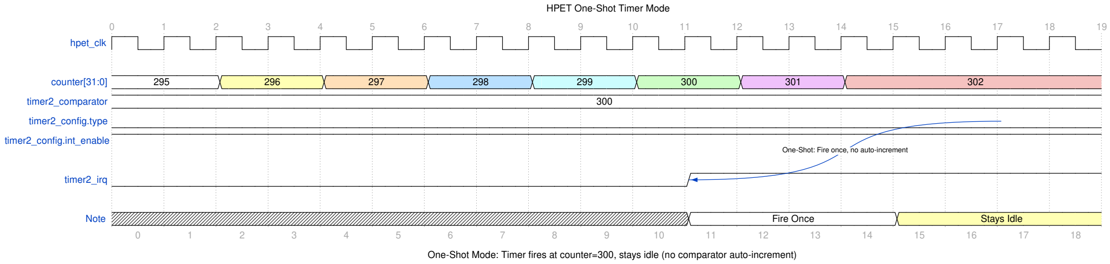
A comparator write loads both the comparator and the period, so writing 1000 gives period 1000. Each fire advances the comparator by the period at the closing edge. Status and irq do NOT pulse per fire: set at Fire 1, they hold through Fire 2 and 3 until software clears HPET_STATUS (W1C).

Periodic Mode Timing Example

8.4.14 One-Shot Mode Details

One-Shot Timer Waveform:

8.4.15 Waveform 2.16: One-Shot Mode Operation


HPET One-Shot Timer Mode
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
hpet_clk
counter[31:0]
295
296
297
298
299
300
301
302
timer2_comparator
300
timer2_config.type
timer2_config.int_enable
One-Shot: Fire once, no auto-increment
timer2_irq
Note
Fire Once
Stays Idle
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
One-Shot Mode: Timer fires at counter=300, stays idle (no comparator auto-increment)

One-Shot Mode Operation

Use WaveDrom Editor to view/edit, or generate SVG with `wavedrom-cli`

8.4.15.1 Fire-Once Behavior

Initial Comparator Write:

```
Software writes: TIMER0_COMPARATOR = 5000
Result:
  r_timer_comparator[0] = 5000
  (period not used in one-shot mode)
```

Fire Event (at counter = 5000):

- The armed match fires the timer and clears the armed latch.
- `timer_int_status[0]` asserts (sticky).
- The comparator stays at 5000 (no auto-reload).
- The status stays set until software clears it.
- Halting and restarting the counter does NOT fire it again.

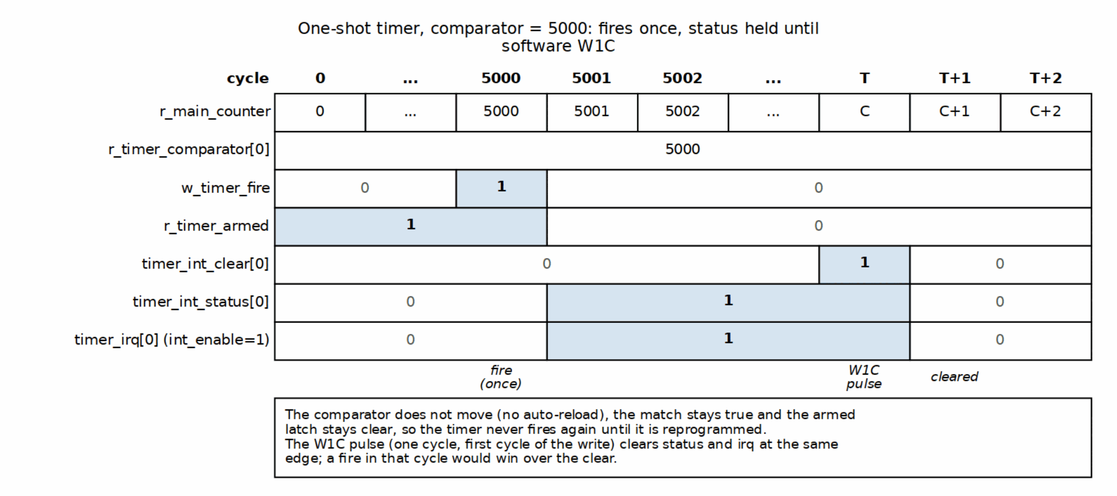
Interrupt Clearing:

```
Software writes: HPET_STATUS[0] = 1 (W1C)
Result:
  timer_int_status[0] clears
  timer_irq[0] clears
```

Reconfiguration:

```
Software writes: TIMER0_COMPARATOR = 10000
Result:
  r_timer_comparator[0] = 10000
  10000 is above the counter, so the match falls and the timer re-arms
  naturally; it waits for counter = 10000
  (to re-arm to a value the counter has already passed, disable the
  timer around the write -- a write on a running timer does not re-
arm)
```

8.4.16 Figure 2.17: One-Shot Mode Timing Example



One-shot timer, comparator = 5000: fires once, status held until software W1C

cycle	0	...	5000	5001	5002	...	T	T+1	T+2
r_main_counter	0	...	5000	5001	5002	...	C	C+1	C+2
r_timer_comparator[0]	5000								
w_timer_fire	0		1	0					
r_timer_armed	1			0					
timer_int_clear[0]	0						1	0	
timer_int_status[0]	0			1				0	
timer_irq[0] (int_enable=1)	0			1				0	

The comparator does not move (no auto-reload), the match stays true and the armed latch stays clear, so the timer never fires again until it is reprogrammed.
The W1C pulse (one cycle, first cycle of the write) clears status and irq at the same edge; a fire in that cycle would win over the clear.

One-Shot Mode Timing Example

8.5 Design Notes

8.5.1 Resource Utilization

Per-Timer Resources (Estimated): - 64-bit comparator register: 64 flip-flops - 64-bit period register: 64 flip-flops - Match comparator: 64-bit >= comparison (~80 LUTs) - Armed latch, next-epoch hold bit, catch-up and fire pulse: 3 flip-flops (armed, epoch, timer_size delay) + a second 64-bit compare (advance >= counter, with carry, ~80 LUTs) + a few gates - Total per timer: ~131 flip-flops, ~165 LUTs

Shared Resources: - 64-bit main counter: 64 flip-flops + 64-bit adder (~70 LUTs) - Global enable logic: ~10 LUTs

Total (NUM_TIMERS = 3): - Flip-flops: 64 + (131 × 3) = 457 FF - LUTs: 80 + (165 × 3) = 575 LUTs

---

8.6 Navigation

Next: Chapter 2.2 - hpet_config_regs

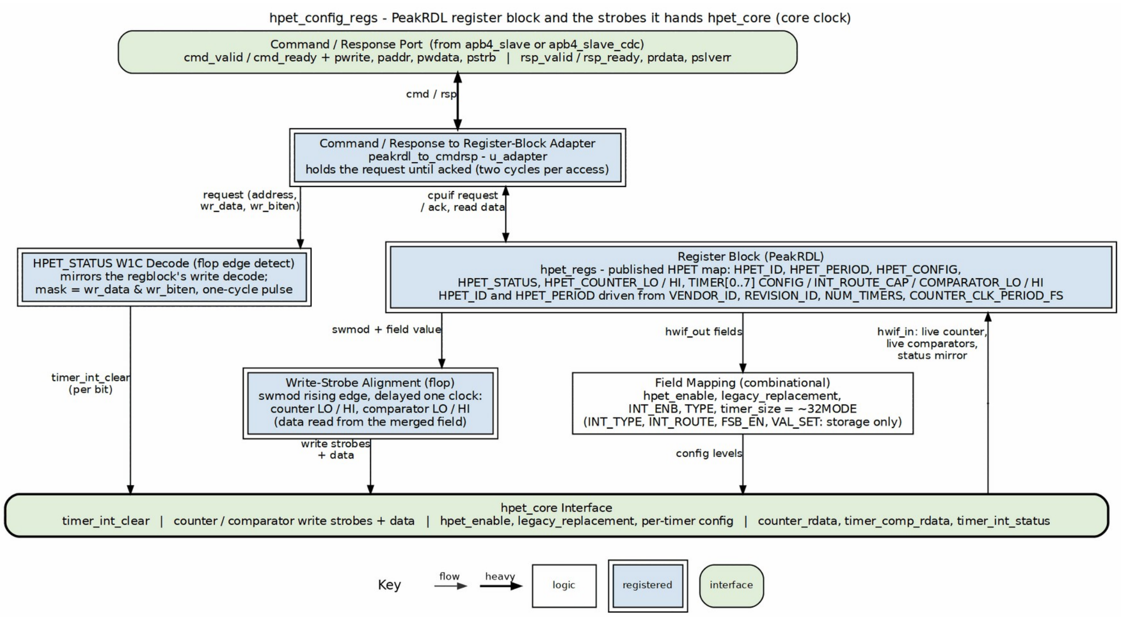
RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

9 hpet_config_regs

9.1 Overview

The `hpet_config_regs` module is the bridge between the PeakRDL-generated register file (`hpet_regs.sv`) and the HPET core timer logic (`hpet_core.sv`). The wrapper exists because the generated register interface and the core's expectations don't line up on their own: it handles interface adaptation, per-timer data bus isolation, and turning the register block's write indications into the one-cycle strobes the core expects.

9.1.1 Figure 2.18: HPET Config Registers Block Diagram


hpet_config_regs - PeakRDL register block and the strobes it hands hpet_core (core clock)
Command / Response Port (from apb4_slave or apb4_slave_cdc)
cmd_valid / cmd_ready + pwrite, paddr, pwdata, pstrb | rsp_valid / rsp_ready, prdata, pslverr
cmd / rsp
Command / Response to Register-Block Adapter
peakrdl_to_cmdrsp - u_adapter
holds the request until acked (two cycles per access)
request (address, wr_data, wr_biten)
cpuif request / ack, read data
HPET_STATUS W1C Decode (flop edge detect)
mirrors the regblock's write decode;
mask = wr_data & wr_biten, one-cycle pulse
Register Block (PeakRDL)
hpet_regs - published HPET map: HPET_ID, HPET_PERIOD, HPET_CONFIG,
HPET_STATUS, HPET_COUNTER_LO / HI, TIMER[0..7] CONFIG / INT_ROUTE_CAP / COMPARATOR_LO / HI
HPET_ID and HPET_PERIOD driven from VENDOR_ID, REVISION_ID, NUM_TIMERS, COUNTER_CLK_PERIOD_FS
swmod + field value
hwif_out fields
hwif_in: live counter, live comparators, status mirror
timer_int_clear (per bit)
Write-Strobe Alignment (flop)
swmod rising edge, delayed one clock:
counter LO / HI, comparator LO / HI
(data read from the merged field)
Field Mapping (combinational)
hpet_enable, legacy_replacement,
INT_ENB, TYPE, timer_size = ~32MODE
(INT_TYPE, INT_ROUTE, FSB_EN, VAL_SET: storage only)
write strobes + data
config levels
hpet_core Interface
timer_int_clear | counter / comparator write strobes + data | hpet_enable, legacy_replacement, per-timer config | counter_rdata, timer_comp_rdata, timer_int_status
Key
flow
heavy
logic
registered
interface

HPET Config Registers Block Diagram

Figure: HPET Config Registers architecture showing APB interface, PeakRDL registers, edge detection, per-timer data buses, and W1C logic. *Source: assets/graphviz/hpet_config_regs.dot*

9.1.2 Key Responsibilities

1. **PeakRDL Integration:** Instantiates `hpet_regs.sv` and `peakrdl_to_cmdrsp` adapter
2. **Interface Mapping:** Converts PeakRDL hardware interface to HPET core signals
3. **Per-Timer Data Buses:** Implements dedicated 64-bit data paths per timer (prevents corruption)

4. **Write Strobes:** Rising-edge detects the register block's `swmod` levels and aligns each to the cycle the field presents the written value – one strobe per 32-bit half
5. **Counter Write Handling:** Hands the just-written counter half to the core, byte strobes already merged by the register block
6. **Interrupt Management:** Mirrors the core's sticky status into HPET_STATUS and decodes the W1C write into a per-bit, one-cycle clear
7. **Identification:** Drives HPET_ID's vendor, revision and timer-count fields from the module parameters

9.2 Parameters

Parameter	Type	Default	Range	Description
`VENDOR_ID`	int	1	0-65535	Drives HPET_ID[31:16] via `hwif_in` – the spec's full 16-bit vendor field
`REVISION_ID`	int	1	0-255	Drives HPET_ID[7:0] via `hwif_in`
`COUNTER_CLK_PERIOD_FS`	int	10000000	non-zero, <= 0x05F5E100	Drives HPET_PERIOD (GCAP_ID[63:32]) via `hwif_in`, in femtoseconds
`NUM_TIMERS`	int	2	2, 3, 8	Number of independent timers in array

9.3 Ports

9.3.1 Clock and Reset

Signal Name	Type	Width	Direction	Description
clk	logic	1	Input	Configuration clock (pclk or hpet_clk

Signal Name	Type	Width	Direction	Description
				based on CDC_ENABLE)
rst_n	logic	1	Input	Active-low asynchronous reset

9.3.2 Command/Response Interface (from APB Slave)

Signal Name	Type	Width	Direction	Description
cmd_valid	logic	1	Input	Command valid
cmd_ready	logic	1	Output	Command ready
cmd_pwrite	logic	1	Input	Command write (1) or read (0)
cmd_paddr	logic	12	Input	Command address
cmd_pwdata	logic	32	Input	Command write data
cmd_pstrb	logic	4	Input	Command write byte strobes
rsp_valid	logic	1	Output	Response valid
rsp_ready	logic	1	Input	Response ready
rsp_prdata	logic	32	Output	Response read data
rsp_pslverr	logic	1	Output	Response error flag

9.3.3 HPET Core Interface (to hpet_core.sv)

Global Configuration:

Signal Name	Type	Width	Direction	Description
hpet_enable	logic	1	Output	Global HPET enable (from HPET_CONFIG[0])
legacy_replacement	logic	1	Output	LegacyReplacement Route enable (from HPET_CONFIG[1]); `hpet_core` routes timer 0 to `legacy_irq0` and timer 1 to `legacy_irq8` and suppresses both on `timer_irq`

Counter Interface:

Signal Name	Type	Width	Direction	Description
counter_write_lo	logic	1	Output	One-cycle strobe: HPET_COUNTER_LO was written
counter_write_hi	logic	1	Output	One-cycle strobe: HPET_COUNTER_HI was written
counter_wdata	logic	64	Output	Counter write data ({HI, LO} field values; the strobed half is the just-written value)
counter_rdata	logic	64	Input	Live counter value (from hpet_core)

Per-Timer Configuration:

Signal Name	Type	Width	Direction	Description
timer_int_enable[NUM_TIMERS-1:0]	logic	NUM_TIMERS	Output	Per-timer interrupt enable (from

Signal Name	Type	Width	Direction	Description
				TIMER_CONFIG[2]). The ONLY per-timer enable the spec defines – there is no per-timer RUN enable
timer_type[NUM_TIMERS-1:0]	logic	NUM_TIMERS	Output	Per-timer mode: 0=One-shot, 1=Periodic (from TIMER_CONFIG[3])
timer_size[NUM_TIMERS-1:0]	logic	NUM_TIMERS	Output	Compare WIDTH for hpet_core: 1=64-bit. Derived as `~TIMER_CONFIG[8]` – the spec's 32MODE bit FORCES 32-bit, so its polarity is the inverse
timer_value_set[NUM_TIMERS-1:0]	logic	NUM_TIMERS	Output	From TIMER_CONFIG[6]. Currently unconsumed: the wire dead-ends at the top level (hpet_core has no such input), so the bit stores and reads back with no hardware effect

Per-Timer Comparator (Dedicated Buses):

Signal Name	Type	Width	Direction	Description
timer_comp_write_lo[NUM_TIMERS-	logic	NUM_TIMERS	Output	Per-timer one-cycle strobe:

Signal Name	Type	Width	Direction	Description
1:0]				TIMER_COMPARATOR_LO written
timer_comp_write_hi[NUM_TIMERS-1:0]	logic	NUM_TIMERS	Output	Per-timer one-cycle strobe: TIMER_COMPARATOR_HI written
timer_comp_wdata[NUM_TIMERS]	logic [63:0]	NUM_TIMERS×64	Output	Per-timer comparator data (LO/HI combined)

Interrupt Status:

Signal Name	Type	Width	Direction	Description
timer_int_status[NUM_TIMERS-1:0]	logic	NUM_TIMERS	Input	Per-timer sticky status (from hpet_core; mirrored into HPET_STATUS)
timer_int_clear[NUM_TIMERS-1:0]	logic	NUM_TIMERS	Output	Per-bit one-cycle clear pulse (the bits written with 1)

9.4 Functional Description

9.4.1 Internal Architecture

9.4.1.1 Component Instantiation

1. Protocol Adapter:

```
peakrdl_to_cmdrsp #(
    .ADDR_WIDTH(12),
    .DATA_WIDTH(32)
) u_adapter (
    .aclk(clk), .aresetn(rst_n),
    // cmd/rsp interface (external)
    .cmd_valid, .cmd_ready, .cmd_pwrite, .cmd_paddr, .cmd_pwdata, .cmd_pstrb,
    .rsp_valid, .rsp_ready, .rsp_prdata, .rsp_pslverr,
    // PeakRDL passthrough interface (to register block)
    .regblk_req, .regblk_req_is_wr, .regblk_addr, .regblk_wr_data, .re
gblk_wr_biten,
    .regblk_req_stall_wr, .regblk_req_stall_rd,
    .regblk_rd_ack, .regblk_rd_err, .regblk_rd_data,
    .regblk_wr_ack, .regblk_wr_err
);
```

2. PeakRDL Register Block:

```
hpet_regs u_hpet_regs (
    .clk(clk),
    .rst(~rst_n),  // PeakRDL uses active-high reset
    // Passthrough CPU interface
    .s_cpuif_req(regblk_req),
    .s_cpuif_req_is_wr(regblk_req_is_wr),
    .s_cpuif_addr(regblk_addr[8:0]),  // 9-bit internal addressing
    .s_cpuif_wr_data(regblk_wr_data),
    .s_cpuif_wr_biten(regblk_wr_biten),
    .s_cpuif_req_stall_wr(regblk_req_stall_wr),
    .s_cpuif_req_stall_rd(regblk_req_stall_rd),
    .s_cpuif_rd_ack(regblk_rd_ack),
    .s_cpuif_rd_err(regblk_rd_err),
    .s_cpuif_rd_data(regblk_rd_data),
    .s_cpuif_wr_ack(regblk_wr_ack),
    .s_cpuif_wr_err(regblk_wr_err),
    // Hardware interface
    .hwif_in(hwif_in),
    .hwif_out(hwif_out)
);
```

9.4.2 Mapping Logic Details

9.4.2.1 Global Configuration Mapping

Direct assignment from PeakRDL outputs:

```
assign hpet_enable = hwif_out.HPET_CONFIG.hpet_enable.value;
assign legacy_replacement =
hwif_out.HPET_CONFIG.legacy_replacement.value;
```

`legacy_replacement` reaches hardware since RLB/hpet TASK-003; it is no longer storage, and `HPET_ID.leg_rt_cap` reads 1 to advertise it.

9.4.2.2 Counter Write Strobes

Everything this block hands to hpet_core is driven by the register WRITE, never by a change in a stored value. The register block exports `swmod` for HPET_COUNTER_LO/HI and every TIMER_COMPARATOR_LO/HI, and two corrections are needed before it can be used as an operation pulse:

1. **It is a level, not a pulse.** `peakrdl_to_cmdrsp` deliberately holds `regblk_req` from the accept cycle through its wait-for-ack state, so `swmod` is asserted for the whole transaction – two cycles.
2. **It leads the field by one cycle.** `swmod` is high while the field is still taking the written value.

So each strobe is a rising-edge detect on the `swmod` level (one transaction, one pulse) delayed one flop, which lands it in the cycle where the field presents the newly written value. The strobe's data is then read straight out of the field, which is what makes byte strobes work: the register block has already merged `(value & ~biten) | (wr_data & biten)`, where sampling `regblk_wr_data` raw would write whole 32-bit words on a partial-byte write. That `value` is the field's own mirror, which on a running counter is a cycle behind the live count – so a partial (`PSTRB != 0xF`) counter write is only exact with the counter halted (chapter 5, HPET_COUNTER_LO usage notes).

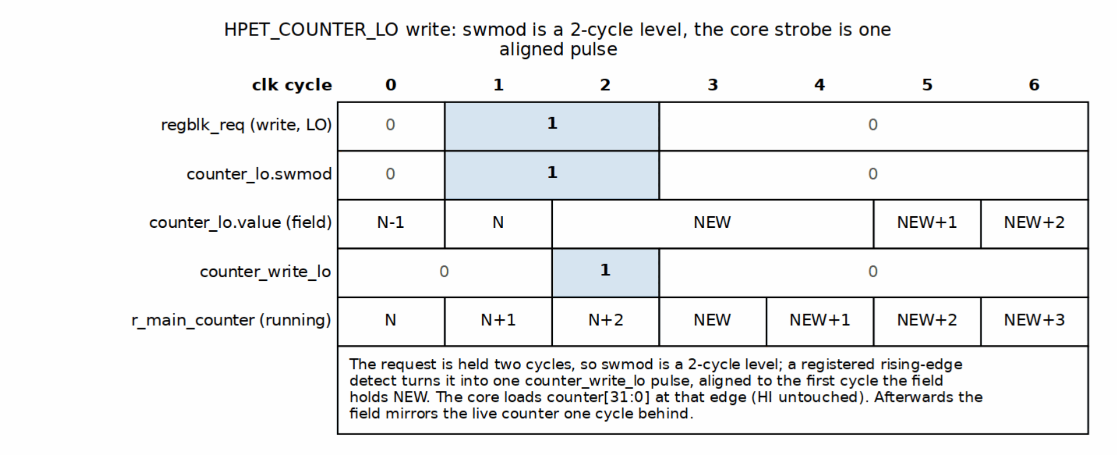
The upstream cpuif must drop `regblk_req` between transactions, or two back-to-back writes to one register would look like one long level and produce a single strobe. `apb4_slave` and `apb4_slave_cdc` are both strictly one-outstanding, so it holds; simulation-only assertions in the module trip if a `swmod` level ever spans three cycles.

Counter write strobes: each half is loaded on its OWN aligned strobe, from its OWN field, so a "write LO then write HI" sequence lands both halves and the core never sees a stale partner:

```
assign w_counter_lo_swmod = hwif_out.HPET_COUNTER_LO.counter_lo.swmod;
assign w_counter_hi_swmod = hwif_out.HPET_COUNTER_HI.counter_hi.swmod;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_counter_lo_swmod_d <= 1'b0;
        r_counter_hi_swmod_d <= 1'b0;
        counter_write_lo     <= 1'b0;
        counter_write_hi     <= 1'b0;
    end else begin
        r_counter_lo_swmod_d <= w_counter_lo_swmod;
        r_counter_hi_swmod_d <= w_counter_hi_swmod;
        counter_write_lo     <= w_counter_lo_swmod &
~r_counter_lo_swmod_d;
        counter_write_hi     <= w_counter_hi_swmod &
~r_counter_hi_swmod_d;
    end
end

// In the aligned strobe cycle the field holds the value the write
just
// committed; one cycle later the hardware write-back resumes
mirroring
// the live counter.
```

```
assign counter_wdata = {hwif_out.HPET_COUNTER_HI.counter_hi.value,
                        hwif_out.HPET_COUNTER_LO.counter_lo.value};
```

9.4.3 Figure 2.19: Counter Write Strobe Timing



HPET_COUNTER_LO write: swmod is a 2-cycle level, the core strobe is one aligned pulse

clk cycle	0	1	2	3	4	5	6
regblk_req (write, LO)	0	1		0			
counter_lo.swmod	0	1		0			
counter_lo.value (field)	N-1	N	NEW			NEW+1	NEW+2
counter_write_lo	0		1	0			
r_main_counter (running)	N	N+1	N+2	NEW	NEW+1	NEW+2	NEW+3

The request is held two cycles, so swmod is a 2-cycle level; a registered rising-edge detect turns it into one counter_write_lo pulse, aligned to the first cycle the field holds NEW. The core loads counter[31:0] at that edge (HI untouched). Afterwards the field mirrors the live counter one cycle behind.

Counter Write Strobe Timing

9.4.3.1 Timer Configuration Mapping

Per-timer array mapping:

```
generate
    for (genvar i = 0; i < NUM_TIMERS; i++) begin : g_timer_mapping
        assign timer_int_enable[i] =
hwif_out.TIMER[i].TIMER_CONFIG.timer_int_enable.value;
        assign timer_type[i]       =
hwif_out.TIMER[i].TIMER_CONFIG.timer_type.value;
        // POLARITY INVERSION: 32MODE = 1 FORCES 32-bit, timer_size =
1 means 64-bit
        assign timer_size[i]       =
~hwif_out.TIMER[i].TIMER_CONFIG.timer_32mode.value;
        assign timer_value_set[i]  =
hwif_out.TIMER[i].TIMER_CONFIG.timer_value_set.value;
    end
endgenerate
```

9.4.3.2 Per-Timer Data Bus Architecture (Corruption Fix)

The Problem: Early designs shared a single 64-bit bus for all timer comparators. Rapid writes to different timers caused corruption when one timer’s data overwrote another timer’s registers.

The Solution: Each timer gets a dedicated 64-bit data bus, preventing any possibility of cross-timer corruption:

```
// CORRECT: Per-timer dedicated data buses
generate
```

```
    for (genvar i = 0; i < NUM_TIMERS; i++) begin : g_timer_wdata
        assign timer_comp_wdata[i] = {
            hwif_out.TIMER[i].TIMER_COMPARATOR_HI.timer_comp_hi.value,
            hwif_out.TIMER[i].TIMER_COMPARATOR_LO.timer_comp_lo.value
        };
    end
endgenerate

// Per-timer write strobes: the same swmod rising-edge-plus-one-flop
// alignment as the counter, one strobe per half. The comparator
fields
// are hw = r, so the value is simply whatever software last wrote;
the
// STROBE is what tells the core to reload it, which is what makes
// rewriting the same value reload it. Whether the reload also re-arms
// the timer is the core's decision (only while the counter is
halted).
generate
    for (genvar i = 0; i < NUM_TIMERS; i++) begin : g_timer_mapping
        assign w_comp_lo_swmod[i] =
hwif_out.TIMER[i].TIMER_COMPARATOR_LO.timer_comp_lo.swmod;
        assign w_comp_hi_swmod[i] =
hwif_out.TIMER[i].TIMER_COMPARATOR_HI.timer_comp_hi.swmod;
    end
endgenerate

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_comp_lo_swmod_d   <= '0;
        r_comp_hi_swmod_d   <= '0;
        timer_comp_write_lo <= '0;
        timer_comp_write_hi <= '0;
    end else begin
        r_comp_lo_swmod_d   <= w_comp_lo_swmod;
        r_comp_hi_swmod_d   <= w_comp_hi_swmod;
        timer_comp_write_lo <= w_comp_lo_swmod & ~r_comp_lo_swmod_d;
        timer_comp_write_hi <= w_comp_hi_swmod & ~r_comp_hi_swmod_d;
    end
end
```

Architecture Benefit: each timer has its own comparator write-data bus, so a write to one timer can never land in another:

- Timer 0: `hwif.TIMER[0].COMP_LO/HI` -> `timer_comp_wdata[0]` -> `hpet_core` timer 0 only
- Timer 1: `hwif.TIMER[1].COMP_LO/HI` -> `timer_comp_wdata[1]` -> `hpet_core` timer 1 only

- Timer 2: `hwif.TIMER[2].COMP_LO/HI` -> `timer_comp_wdata[2]` -> `hpet_core` timer 2 only

9.4.3.3 Interrupt Status Handling

HPET_STATUS is a mirror. hpet_core owns the sticky interrupt status. The register's field is driven from the core's live level every cycle (`hw = w`), and the software write is turned into a per-bit clear pulse into the core. The field is a fixed 8 bits wide regardless of NUM_TIMERS; zero-extending here is what keeps the bits with no timer behind them reading 0 forever.

```
// Mirror out: the core's level, zero-extended to the 8-bit field
always_comb begin
    w_timer_int_status_reg                  = '0;
    w_timer_int_status_reg[NUM_TIMERS-1:0] = timer_int_status;
end
assign hwif_in.HPET_STATUS.timer_int_status.next =
w_timer_int_status_reg;
```

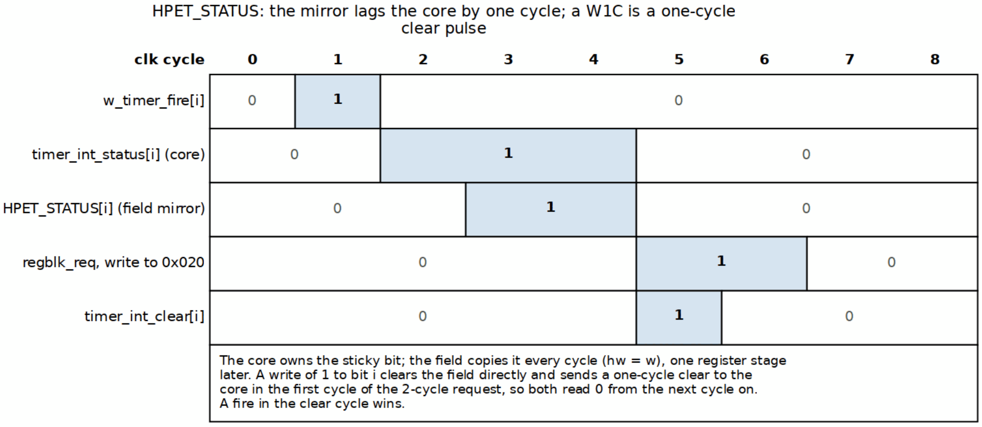
Interrupt Clearing (W1C, per bit):

The clear mask comes from the write itself – `regblk_wr_data & regblk_wr_biten`, masked to NUM_TIMERS bits – so a bit clears only if software wrote a 1 to it, and a write of 0x0 is the no-op W1C requires. The write is detected by mirroring the register block's own address decode rather than through `swmod` (whose extra `|biten` term makes it a different decode); a simulation-only assertion guards the two against drifting apart.

```
// Mirror of the regblock's decode for HPET_STATUS (offset 0x020)
assign w_status_sw_wr = regblk_req && regblk_req_is_wr &&
                        (regblk_addr[8:0] == ADDR_HPET_STATUS);

// One event per transaction: the request is held for two cycles, and
a
// clear that spanned both would undo a fire the core accepted in the
first
assign w_status_wr_event = w_status_sw_wr & ~r_status_sw_wr_d;

// Per-bit W1C mask from the write being committed
assign w_status_w1c_mask = regblk_wr_data[NUM_TIMERS-1:0] &
                           regblk_wr_biten[NUM_TIMERS-1:0];

assign timer_int_clear = w_status_wr_event ? w_status_w1c_mask : '0;
```

The clear is narrowed to ONE cycle even though the mask makes it idempotent: a two-cycle level would eat a fire that landed in its first cycle (the core sets the bit, the level's second cycle clears it again). One cycle here plus fire-over-clear priority in the core closes that window.

9.4.4 Figure 2.20: HPET_STATUS Mirror and W1C Clear Timing


HPET_STATUS: the mirror lags the core by one cycle; a W1C is a one-cycle clear pulse

clk cycle
0
1
2
3
4
5
6
7
8
w_timer_fire[i]
0
1
0
timer_int_status[i] (core)
0
1
0
HPET_STATUS[i] (field mirror)
0
1
0
regblk_req, write to 0x020
0
1
0
timer_int_clear[i]
0
1
0

The core owns the sticky bit; the field copies it every cycle (hw = w), one register stage later. A write of 1 to bit i clears the field directly and sends a one-cycle clear to the core in the first cycle of the 2-cycle request, so both read 0 from the next cycle on. A fire in the clear cycle wins.

HPET_STATUS Mirror and W1C Clear Timing

9.4.5 Register-to-Core Signal Summary

Critical Signals:

1. **hpet_enable:** Level signal, directly gates counter incrementing
2. **counter_write_lo / counter_write_hi:** Pulse (1 cycle) per counter half written
3. **counter_wdata:** The field values, valid for the strobed half in the strobe cycle
4. **timer_int_enable[i]:** Level signal per timer (no per-timer RUN enable exists)
5. **timer_comp_write_lo/hi[i]:** Pulse (1 cycle) per comparator half written
6. **timer_comp_wdata[i]:** Per-timer dedicated data bus (corruption-proof)
7. **timer_int_clear[i]:** Pulse (1 cycle) for the status bits written with 1

Signal Types: - **Level Signals:** Direct PeakRDL `.value` outputs (enable, type, size) - **Pulse Signals:** Rising-edge detects on the `swmod` write levels (strobes) or on the mirrored HPET_STATUS write decode (clear) – never on a value change - **Data Buses:** Combined register field values (counter, comparators)

9.5 Design Notes

9.5.1 Resource Utilization

Configuration Register Logic (hpet_config_regs only, excluding hpet_regs):

Component	NUM_TIMERS=2	NUM_TIMERS=3	NUM_TIMERS=8
Mapping Logic	~50 FF, ~100 LUTs	~75 FF, ~150 LUTs	~150 FF, ~300 LUTs
Edge Detect	~10 FF, ~20 LUTs	~15 FF, ~30 LUTs	~30 FF, ~60 LUTs
Interrupt Handling	~10 FF, ~20 LUTs	~15 FF, ~30 LUTs	~30 FF, ~60 LUTs
Total	~70 FF, ~140 LUTs	~105 FF, ~210 LUTs	~210 FF, ~420 LUTs

Scaling: Primarily driven by number of timers. Each additional timer adds ~35 FF and ~70 LUTs for mapping and edge detection logic.

9.6 Navigation

Next: Chapter 2.3 - hpet_regs (PeakRDL)

---

10 hpet_regs

10.1 Overview

The `hpet_regs` module is auto-generated from the SystemRDL specification (`rdl/hpet/hpet_regs.rdl`) using the PeakRDL toolchain. It implements the complete HPET register file with proper field access semantics (RO, RW, W1C), hardware interface integration, and CPU interface protocol handling.

Single Source of Truth: All register definitions, addresses, field widths, and access properties are specified in the SystemRDL file. The generated RTL is deterministic and regeneratable – which means the .rdl file is where you make changes, never the generated output.

Generation Command:

```
cd projects/components/legacy-ip/retro_legacy_blocks/rtl/hpet/peakrdl
peakrdl regblock hpet_regs.rdl --cpuif passthrough -o ../
```

Generated Files: - `hpet_regs.sv` - Register implementation - `hpet_regs_pkg.sv` - Package with structs and parameters

---

10.2 Parameters

No user-configurable parameters. All configuration is baked into the generated code from SystemRDL.

Compile-Time Constants (from SystemRDL):

```
localparam VENDOR_ID = 1;         // From RDL: vendor_id reset default
localparam REVISION_ID = 1;       // From RDL: rev_id reset default
localparam NUM_TIMERS = 8;        // From RDL: TIMER[0:7] array size
```

Note: These are fixed at generation time, but none of them decides what an instance reports. The block is generated once for 8 timer slots, and `num_tim_cap`, `vendor_id` and `rev_id` are all `hw = w` fields driven through `hwif_in` from the wrapper's `NUM_TIMERS`/`VENDOR_ID`/`REVISION_ID` parameters – so the same generated RTL serves every instantiation.

10.3 Ports

10.3.1 Clock and Reset

Signal Name	Type	Width	Direction	Description
clk	wire	1	Input	Register clock (pclk

Signal Name	Type	Width	Direction	Description
				or hpet_clk based on CDC_ENABLE)
rst	wire	1	Input	**Active-high** reset (PeakRDL convention)

Caveat: PeakRDL uses active-high reset. The wrapper (`hpet_config_regs.sv`) inverts `rst_n` before connecting.

10.3.2 CPU Interface (Passthrough Protocol)

Signal Name	Type	Width	Direction	Description
s_cpuif_req	wire	1	Input	CPU request valid
s_cpuif_req_is_wr	wire	1	Input	Request is write (1) or read (0)
s_cpuif_addr	wire	9	Input	Address (byte-aligned, bits [8:0])
s_cpuif_wr_data	wire	32	Input	Write data
s_cpuif_wr_biten	wire	32	Input	Write byte enable (bit-level)
s_cpuif_req_stall_wr	wire	1	Output	Stall write request (always 0 for HPET)
s_cpuif_req_stall_rd	wire	1	Output	Stall read request (always 0 for HPET)
s_cpuif_rd_ack	wire	1	Output	Read acknowledgment
s_cpuif_rd_err	wire	1	Output	Read error (decoding error)
s_cpuif_rd_data	wire	32	Output	Read data
s_cpuif_wr_ack	wire	1	Output	Write acknowledgment
s_cpuif_wr_err	wire	1	Output	Write error (always 0 for

Signal Name	Type	Width	Direction	Description
				HPET)

Protocol Characteristics: - **Latency:** 1 cycle for both reads and writes - **Stalls:** Never stall (HPET registers have single-cycle access) - **Errors:** None – both error outputs are tied to 0 (`cpuif_wr_err = 0`, `readback_err = 0`); no address raises PSLVERR, and addresses above 0x1FF alias the map (only addr[8:0] is decoded)

10.3.3 Hardware Interface (Structs)

```
input  hpet_regs_pkg::hpet_regs__in_t  hwif_in;   // From hardware to registers
output hpet_regs_pkg::hpet_regs__out_t hwif_out;  // From registers to hardware
```

Structure Definitions (in hpet_regs_pkg.sv):

The package defines comprehensive structs for all registers and fields. Key excerpts:

```
package hpet_regs_pkg;

    // ILLUSTRATIVE SKETCH -- the generated package uses UNPACKED structs
    // with per-field typedef names (hpet_regs__<REG>__<field>__in_t); see
    // rtl/hpet/hpet_regs_pkg.sv for the literal shapes. One notable
    // difference from this sketch: the timer array is
    // hpet_regs__timer_regfile__out_t TIMER[8].
    typedef struct {
        struct packed {
            logic [4:0] next;  // num_tim_cap field value
        } num_tim_cap;
        struct packed {
            logic [7:0] next;  // rev_id, from the REVISION_ID parameter
        } rev_id;
        struct packed {
            logic [15:0] next; // vendor_id, from the VENDOR_ID parameter (GCAP_ID[31:16])
        } vendor_id;
    } HPET_ID__in_t;

    typedef struct packed {
        logic [7:0] next;   // Live status level from hpet_core (mirror)
    } timer_int_status__in_t;

    typedef struct packed {
        logic [31:0] next;  // Next counter value
    } counter_lo__in_t;

    // ... additional field structs ...

    // Complete input struct
    typedef struct packed {
        HPET_ID__in_t HPET_ID;
        timer_int_status__in_t HPET_STATUS.timer_int_status;
        counter_lo__in_t HPET_COUNTER_LO.counter_lo;
        counter_hi__in_t HPET_COUNTER_HI.counter_hi;
        // ... additional register fields ...
    } hpet_regs__in_t;

    // Hardware output struct (registers -> hardware)
    typedef struct packed {
        struct packed {
            logic value;     // Current field value
        } hpet_enable;
        struct packed {
            logic value;
        } legacy_replacement;
    } HPET_CONFIG__out_t;

    typedef struct packed {
        logic swmod;         // Software modified (write detected)
    } timer_int_status__out_t;

    typedef struct packed {
        logic [31:0] value;  // Current register value
        logic swmod;         // Software modified
    } counter_lo__out_t;

    // ... additional field structs ...

    // Complete output struct
    typedef struct packed {
        HPET_CONFIG__out_t HPET_CONFIG;
        timer_int_status__out_t HPET_STATUS.timer_int_status;
        counter_lo__out_t HPET_COUNTER_LO.counter_lo;
        counter_hi__out_t HPET_COUNTER_HI.counter_hi;
        TIMER__out_t TIMER[7:0];  // Timer array
        // ... additional registers ...
    } hpet_regs__out_t;

endpackage
```

10.4 Functional Description

10.4.1 Register Implementation

10.4.1.1 Address Decoding

PeakRDL generates a decoded register strobe struct:

```
typedef struct {
    logic HPET_ID;
    logic HPET_CONFIG;
    logic HPET_STATUS;
    logic RESERVED_0C;
    logic HPET_COUNTER_LO;
    logic HPET_COUNTER_HI;
    struct {
        logic TIMER_CONFIG;
        logic TIMER_COMPARATOR_LO;
        logic TIMER_COMPARATOR_HI;
        logic RESERVED;
    } TIMER[8];
} decoded_reg_strb_t;

decoded_reg_strb_t decoded_reg_strb;
```

Decoding Logic:

```
always_comb begin
    decoded_reg_strb.HPET_ID = cpuif_req_masked & (cpuif_addr ==
9'h0);
    decoded_reg_strb.HPET_CONFIG = cpuif_req_masked & (cpuif_addr ==
9'h4);
    decoded_reg_strb.HPET_STATUS = cpuif_req_masked & (cpuif_addr ==
9'h8);
    decoded_reg_strb.RESERVED_0C = cpuif_req_masked & (cpuif_addr ==
9'hc);
    decoded_reg_strb.HPET_COUNTER_LO = cpuif_req_masked & (cpuif_addr
== 9'h10);
    decoded_reg_strb.HPET_COUNTER_HI = cpuif_req_masked & (cpuif_addr
== 9'h14);

    for(int i0=0; i0<8; i0++) begin
        decoded_reg_strb.TIMER[i0].TIMER_CONFIG =
            cpuif_req_masked & (cpuif_addr == 9'h100 + (9)'(i0) *
9'h20);
        decoded_reg_strb.TIMER[i0].TIMER_COMPARATOR_LO =
            cpuif_req_masked & (cpuif_addr == 9'h104 + (9)'(i0) *
9'h20);
        decoded_reg_strb.TIMER[i0].TIMER_COMPARATOR_HI =
            cpuif_req_masked & (cpuif_addr == 9'h108 + (9)'(i0) *
9'h20);
        decoded_reg_strb.TIMER[i0].RESERVED =
            cpuif_req_masked & (cpuif_addr == 9'h10c + (9)'(i0) *
9'h20);
    end
end
```

10.4.1.2 Field Logic

Each field is implemented with: - **Combo Logic:** Determines next value based on SW write, HW input, or current value - **Sequential Logic:** Stores field value in flip-flops - **Output Assignment:** Drives `hwif_out` struct

Example - HPET_CONFIG.hpet_enable Field:

```
// Field: hpet_regs.HPET_CONFIG.hpet_enable
always_comb begin
    automatic logic [0:0] next_c;
    automatic logic load_next_c;

    next_c = field_storage.HPET_CONFIG.hpet_enable.value;  // Default:
hold
    load_next_c = '0;

    if(decoded_reg_strb.HPET_CONFIG && decoded_req_is_wr) begin  // SW
write
        next_c = (field_storage.HPET_CONFIG.hpet_enable.value &
~decoded_wr_biten[0:0]) |
                (decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
        load_next_c = '1;
    end

    field_combo.HPET_CONFIG.hpet_enable.next = next_c;
    field_combo.HPET_CONFIG.hpet_enable.load_next = load_next_c;
end

always_ff @(posedge clk) begin
    if(rst) begin
        field_storage.HPET_CONFIG.hpet_enable.value <= 1'h0;  // Reset
value
    end else begin
        if(field_combo.HPET_CONFIG.hpet_enable.load_next) begin
            field_storage.HPET_CONFIG.hpet_enable.value <=
field_combo.HPET_CONFIG.hpet_enable.next;
```

```
        end
    end
end

assign hwif_out.HPET_CONFIG.hpet_enable.value =
field_storage.HPET_CONFIG.hpet_enable.value;
```

Example - HPET_STATUS.timer_int_status Field (W1C mirror of a hardware level):

```
// Field: hpet_regs.HPET_STATUS.timer_int_status
always_comb begin
    automatic logic [7:0] next_c;
    automatic logic load_next_c;

    next_c = field_storage.HPET_STATUS.timer_int_status.value;
    load_next_c = '0;

    if(decoded_reg_strb.HPET_STATUS && decoded_req_is_wr) begin  // SW
write 1 to clear
        next_c = field_storage.HPET_STATUS.timer_int_status.value &
                ~(decoded_wr_data[7:0] & decoded_wr_biten[7:0]);
        load_next_c = '1;
    end else begin  // HW write: follow the core's live level
        next_c = hwif_in.HPET_STATUS.timer_int_status.next;
        load_next_c = '1;
    end

    field_combo.HPET_STATUS.timer_int_status.next = next_c;
    field_combo.HPET_STATUS.timer_int_status.load_next = load_next_c;
end

always_ff @(posedge clk) begin
    if(rst) begin
        field_storage.HPET_STATUS.timer_int_status.value <= 8'h0;
    end else begin
        if(field_combo.HPET_STATUS.timer_int_status.load_next) begin
            field_storage.HPET_STATUS.timer_int_status.value <=
field_combo.HPET_STATUS.timer_int_status.next;
        end
    end
end

// swmod: the write decode with an extra |biten term. The wrapper
mirrors
// the decode itself for the W1C clear and uses swmod only as a drift
guard.
assign hwif_out.HPET_STATUS.timer_int_status.swmod =
    decoded_reg_strb.HPET_STATUS && decoded_req_is_wr && |
(decoded_wr_biten[7:0]);
```

There is no `sticky`/`hwset` in this field, on purpose. PeakRDL renders a multi-bit sticky field as “load `next` only while the register reads all-zero, otherwise set ALL bits on hwset”, which sets every bit whenever a second timer fires with an earlier one still pending. hpet_core owns the sticky state instead, and this field mirrors it – note the software W1C branch still wins over the mirror in its own cycle (`precedence = sw`), and the core drops the bit the cycle after, so the two agree from then on.

Example - HPET_COUNTER_LO Field (HW write with SW precedence):

```
// Field: hpet_regs.HPET_COUNTER_LO.counter_lo
always_comb begin
    automatic logic [31:0] next_c;
    automatic logic load_next_c;

    next_c = field_storage.HPET_COUNTER_LO.counter_lo.value;
    load_next_c = '0;

    if(decoded_reg_strb.HPET_COUNTER_LO && decoded_req_is_wr) begin
// SW write
        next_c = (field_storage.HPET_COUNTER_LO.counter_lo.value &
~decoded_wr_biten[31:0]) |
                (decoded_wr_data[31:0] & decoded_wr_biten[31:0]);
        load_next_c = '1;
    end else begin  // HW write (precedence=sw means HW writes unless
SW writes)
        next_c = hwif_in.HPET_COUNTER_LO.counter_lo.next;
        load_next_c = '1;
    end

    field_combo.HPET_COUNTER_LO.counter_lo.next = next_c;
    field_combo.HPET_COUNTER_LO.counter_lo.load_next = load_next_c;
end

always_ff @(posedge clk) begin
    if(rst) begin
        field_storage.HPET_COUNTER_LO.counter_lo.value <= 32'h0;
    end else begin
        if(field_combo.HPET_COUNTER_LO.counter_lo.load_next) begin
            field_storage.HPET_COUNTER_LO.counter_lo.value <=
field_combo.HPET_COUNTER_LO.counter_lo.next;
        end
    end
end
```

```
assign hwif_out.HPET_COUNTER_LO.counter_lo.value =
field_storage.HPET_COUNTER_LO.counter_lo.value;
assign hwif_out.HPET_COUNTER_LO.counter_lo.swmod =
    decoded_reg_strb.HPET_COUNTER_LO && decoded_req_is_wr && |
(decoded_wr_biten[31:0]);
```

10.4.1.3 Read Response Logic

PeakRDL generates readback arrays for all registers:

```
// Assign readback values to a flattened array
logic [31:0] readback_array[38];

// Global registers
assign readback_array[0][7:0]   = (decoded_reg_strb.HPET_ID && !
decoded_req_is_wr) ?
                                  hwif_in.HPET_ID.rev_id.next : '0;
assign readback_array[0][12:8]  = (decoded_reg_strb.HPET_ID && !
decoded_req_is_wr) ?
                                  hwif_in.HPET_ID.num_tim_cap.next :
'0;
assign readback_array[0][13:13] = (decoded_reg_strb.HPET_ID && !
decoded_req_is_wr) ? 1'h1 : '0;  // count_size_cap
assign readback_array[0][14:14] = (decoded_reg_strb.HPET_ID && !
decoded_req_is_wr) ? 1'h0 : '0;  // reserved
assign readback_array[0][15:15] = (decoded_reg_strb.HPET_ID && !
decoded_req_is_wr) ? 1'h1 : '0;  // leg_rt_cap
assign readback_array[0][31:16] = (decoded_reg_strb.HPET_ID && !
decoded_req_is_wr) ?
                                  hwif_in.HPET_ID.vendor_id.next : '0;

// Config/status registers
assign readback_array[2][0:0] = (decoded_reg_strb.HPET_CONFIG && !
decoded_req_is_wr) ?

field_storage.HPET_CONFIG.hpet_enable.value : '0;
assign readback_array[3][7:0] = (decoded_reg_strb.HPET_STATUS && !
decoded_req_is_wr) ?

field_storage.HPET_STATUS.timer_int_status.value : '0;

// Counter registers
assign readback_array[4][31:0] = (decoded_reg_strb.HPET_COUNTER_LO
&& !decoded_req_is_wr) ?

field_storage.HPET_COUNTER_LO.counter_lo.value : '0;
assign readback_array[5][31:0] = (decoded_reg_strb.HPET_COUNTER_HI
&& !decoded_req_is_wr) ?
```

```
field_storage.HPET_COUNTER_HI.counter_hi.value : '0;

// Per-timer registers
for(genvar i0=0; i0<8; i0++) begin
    assign readback_array[i0 * 4 + 6][2:2] =
(decoded_reg_strb.TIMER[i0].TIMER_CONFIG && !decoded_req_is_wr) ?

field_storage.TIMER[i0].TIMER_CONFIG.timer_int_enable.value : '0;
    // ... additional timer fields ...
end

// Reduce array via OR (only one element active at a time)
always_comb begin
    automatic logic [31:0] readback_data_var;
    readback_done = decoded_req & ~decoded_req_is_wr;
    readback_err = '0;
    readback_data_var = '0;
    for(int i=0; i<38; i++) readback_data_var |= readback_array[i];
    readback_data = readback_data_var;
end

assign cpuif_rd_ack = readback_done;
assign cpuif_rd_data = readback_data;
assign cpuif_rd_err = readback_err;
```

10.4.2 Field Access Semantics

10.4.2.1 Read-Only (RO)

Characteristics: - Software reads return hardware-driven value - Software writes are ignored (no effect) - Hardware controls value via `hwif_in`

Example: HPET_ID register

```
// RO fields: vendor_id, rev_id, num_tim_cap are hw = w, driven from
the
// wrapper's parameters; leg_rt_cap and count_size_cap are constants
// Software can read, but writes have no effect
```

10.4.2.2 Read-Write (RW)

Characteristics: - Software can read and write - Default next value is current value - Software write updates value - Reset value specified in RDL

Example: HPET_CONFIG.hpet_enable

```
// RW field: Software can enable/disable HPET
// Reset value: 0 (disabled)
```

10.4.2.3 Write-1-to-Clear (W1C)

Characteristics: - Software writes 1 to clear bit - Software writes 0 have no effect - Hardware drives the live level via `hwif_in.next`; the field mirrors it - The sticky state itself lives in hpet_core, not here

Example: HPET_STATUS.timer_int_status

```
// W1C mirror: software writes 1 to clear the bit; the wrapper
forwards
// that write to hpet_core as a per-bit clear, and hardware re-drives
the
// field from the core's level every cycle
```

10.4.2.4 Hardware Write with Software Precedence

Characteristics: - Hardware continuously writes value via `hwif_in.next` - Software write takes precedence - Used for live counter readback

Example: HPET_COUNTER_LO/HI

```
// hw=w, precedence=sw
// Hardware writes counter value every cycle
// Software write overrides hardware write
```

10.4.3 SystemRDL Specification

Source File: `rdl/hpet/hpet_regs.rdl`

Key RDL Properties Used:

```
addrmap hpet_regs {
    name = "HPET Register Block";
    desc = "High Precision Event Timer registers";

    default regwidth = 32;       // All registers 32-bit
    default accesswidth = 32;    // Single-beat access

    // Read-only identification
    reg {
        field {
            hw = w;              // Driven from the VENDOR_ID
parameter
            sw = r;              // Software read-only
        } vendor_id[31:16] = VENDOR_ID;   // spec's full 16-bit vendor
field

        field {
            hw = w; sw = r;      // Driven from the REVISION_ID
```

```
parameter
        } rev_id[7:0] = REVISION_ID;

        field {
            hw = w;              // Hardware controls value
            sw = r;              // Software can only read
        } num_tim_cap[12:8];

        field {
            hw = na; sw = r;     // Reads 1: legacy replacement IS
routed
        } leg_rt_cap[15:15] = 1'b1;

    } HPET_ID @ 0x000;

    // Read-write configuration
    reg {
        field {
            sw = rw;             // Software read-write
            hw = r;              // Hardware reads value
        } hpet_enable[0:0] = 1'b0;

        field {
            sw = rw; hw = r;
        } legacy_replacement[1:1] = 1'b0;

    } HPET_CONFIG @ 0x010;

    // Write-1-to-clear status: a MIRROR of hpet_core's sticky status
    reg {
        field {
            sw = rw;
            hw = w;              // Hardware writes the core's live
level
            precedence = sw;     // The W1C write wins in its cycle
            onwrite = woclr;     // Write 1 to clear
            swmod;               // Drift guard for the wrapper's
decode
        } timer_int_status[NUM_TIMERS-1:0] = 0;   // no sticky, no
hwset

    } HPET_STATUS @ 0x020;

    // Hardware-written counter with software override
    reg {
        field {
            sw = rw;             // Software can write
            hw = rw;             // Hardware writes every cycle, reads
it back
            precedence = sw;     // Software write takes priority
            swmod;               // Write STROBE into hpet_core
        } counter_lo[31:0] = 32'h0;

    } HPET_COUNTER_LO @ 0x0F0;

    // Per-timer array
    regfile {
        reg {
            field { sw = rw; hw = r; } timer_int_type[1:1] = 1'b0;
            field { sw = rw; hw = r; } timer_int_enable[2:2] = 1'b0;
            field { sw = rw; hw = r; } timer_type[3:3] = 1'b0;
            field { sw = r;  hw = na; } per_int_cap[4:4] = 1'b1;   //
RO capability
            field { sw = r;  hw = na; } size_cap[5:5] = 1'b1;      //
RO capability
            field { sw = rw; hw = r; } timer_value_set[6:6] = 1'b0;
            field { sw = rw; hw = r; } timer_32mode[8:8] = 1'b0;   //
1 FORCES 32-bit
            field { sw = rw; hw = r; } timer_int_route[13:9] = 5'h0;
            field { sw = rw; hw = r; } timer_fsb_en[14:14] = 1'b0;
            field { sw = r;  hw = na; } fsb_int_del_cap[15:15] = 1'b0;
// RO, no FSB
        } TIMER_CONFIG @ 0x00;

        reg {
            field { sw = rw; hw = rw; precedence = sw; swmod; }
timer_comp_lo[31:0] = 32'h0;  // live readback + write strobe
        } TIMER_COMPARATOR_LO @ 0x08;

        reg {
            field { sw = rw; hw = rw; precedence = sw; swmod; }
timer_comp_hi[31:0] = 32'h0;
        } TIMER_COMPARATOR_HI @ 0x0C;

    } TIMER[NUM_TIMERS] @ 0x100 += 0x20;  // 32-byte spacing
};
```

10.5 Usage Example

10.5.1 Regeneration Procedure

When to Regenerate: 1. Changing register addresses 2. Adding/removing fields 3. Modifying field access properties 4. Changing the constant capability bits (`leg_rt_cap`, `count_size_cap`)

NUM_TIMERS, VENDOR_ID and REVISION_ID do NOT require regeneration: the block is generated with 8 slots, and all three ID fields are driven through `hwif_in` from the wrapper's parameters at instantiation.

Steps:

```
cd projects/components/legacy-ip/retro_legacy_blocks/rtl/hpet/peakrdl

# 1. Edit SystemRDL specification
vim hpet_regs.rdl

# 2. Generate RTL
peakrdl regblock hpet_regs.rdl --cpuif passthrough -o ../

# 3. Verify generated files
ls -l ../hpet_regs.sv ../hpet_regs_pkg.sv

# 4. Review changes (if in version control)
git diff ../hpet_regs.sv ../hpet_regs_pkg.sv

# 5. Run tests to verify
pytest
projects/components/legacy-ip/retro_legacy_blocks/dv/tests/test_apb4_hpet.py -v
```

Caveat: Do not manually edit generated files! All changes must be made in `hpet_regs.rdl` and regenerated.

10.6 Navigation

Next: Chapter 2.4 - apb4_hpet (Top Level)

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

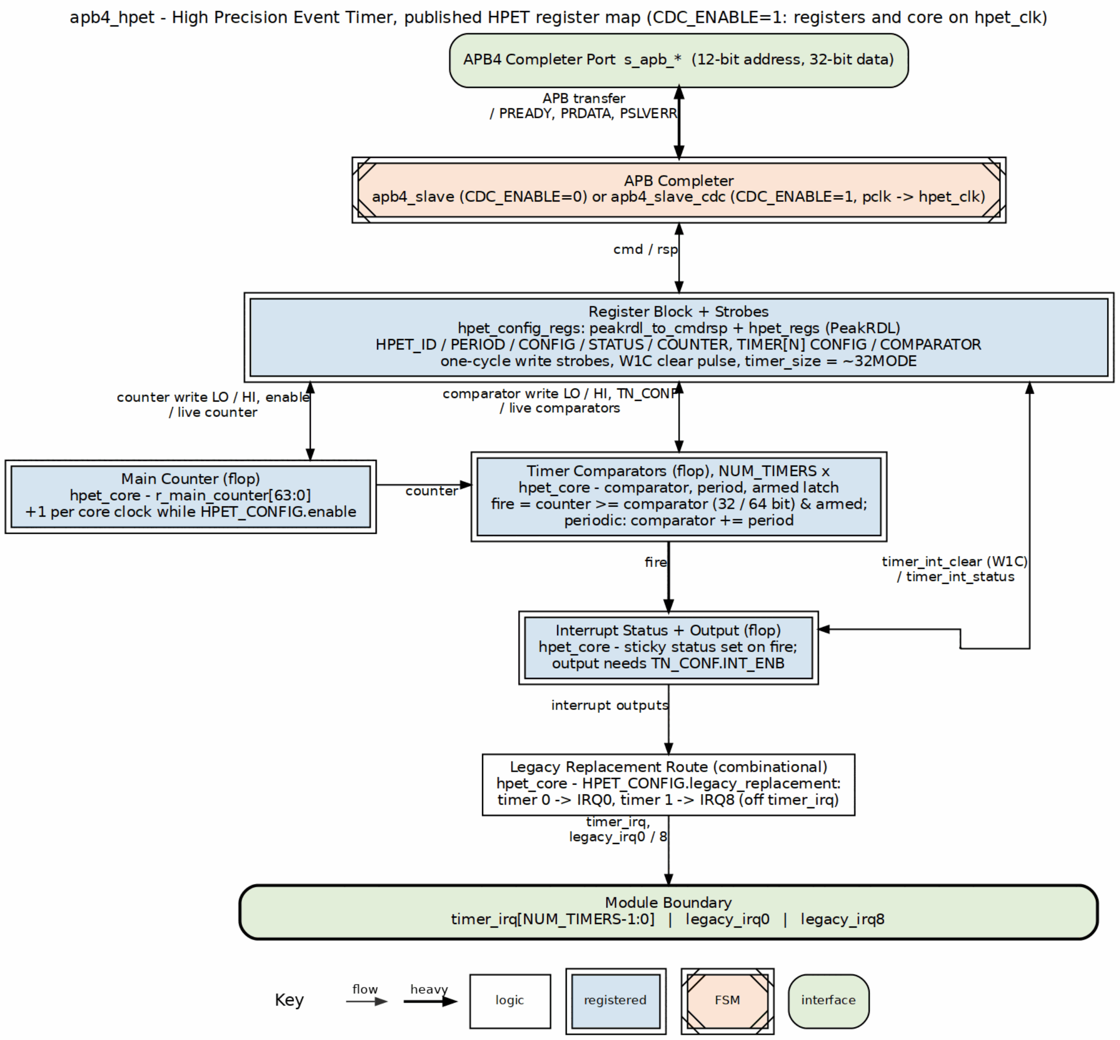
---

11 apb4_hpet

11.1 Overview

The `apb4_hpet` module is the top-level system integration point. It combines the APB slave interface, configuration registers, and timer core into a complete HPET peripheral, provides parameterized clock domain crossing (CDC) support, and exposes a unified external interface. It's mostly wiring – but wiring with opinions, particularly about which clock each block runs on.

11.1.1 Figure 2.21: APB HPET Top-Level Block Diagram


apb4_hpet - High Precision Event Timer, published HPET register map (CDC_ENABLE=1: registers and core on hpet_clk)
APB4 Completer Port s_apb_* (12-bit address, 32-bit data)
APB transfer / PREADY, PRDATA, PSLVERR
APB Completer
apb4_slave (CDC_ENABLE=0) or apb4_slave_cdc (CDC_ENABLE=1, pclk -> hpet_clk)
cmd / rsp
Register Block + Strobes
hpet_config_regs: peakrdl_to_cmdrsp + hpet_regs (PeakRDL)
HPET_ID / PERIOD / CONFIG / STATUS / COUNTER, TIMER[N] CONFIG / COMPARATOR
one-cycle write strobes, W1C clear pulse, timer_size = ~32MODE
counter write LO / HI, enable / live counter
comparator write LO / HI, TN_CONF / live comparators
Main Counter (flop)
hpet_core - r_main_counter[63:0]
+1 per core clock while HPET_CONFIG.enable
counter
Timer Comparators (flop), NUM_TIMERS x
hpet_core - comparator, period, armed latch
fire = counter >= comparator (32 / 64 bit) & armed;
periodic: comparator += period
fire
timer_int_clear (W1C) / timer_int_status
Interrupt Status + Output (flop)
hpet_core - sticky status set on fire;
output needs TN_CONF.INT_ENB
interrupt outputs
Legacy Replacement Route (combinational)
hpet_core - HPET_CONFIG.legacy_replacement:
timer 0 -> IRQ0, timer 1 -> IRQ8 (off timer_irq)
timer_irq, legacy_irq0 / 8
Module Boundary
timer_irq[NUM_TIMERS-1:0] | legacy_irq0 | legacy_irq8
Key
flow
heavy
logic
registered
FSM
interface

APB HPET Top-Level Block Diagram

Figure: APB HPET top-level integration showing dual clock domains (PCLK and HPET_CLK), optional CDC, configuration registers, HPET core, and interrupt outputs. Source:

assets/graphviz/hpet_whole_unit.dot; the CDC_ENABLE=1 crossings are in hpet_cdc_crossings

Key Integration Features: - Conditional APB slave instantiation (CDC or non-CDC) - Clock domain management - Parameter propagation to all child modules - Timer interrupt aggregation - Single-point system configuration

11.1.2 Module Hierarchy

```
apb4_hpet
+-- APB Slave Interface (conditional generation)
|   +-- apb4_slave (CDC_ENABLE=0)
|   |   +-- Synchronous APB protocol
|   +-- apb4_slave_cdc (CDC_ENABLE=1)
|       +-- APB protocol (pclk domain)
|       +-- CDC handshake (pclk ↔ hpet_clk)
|
+-- HPET Configuration Registers
|   +-- peakrdl_to_cmdrsp (protocol adapter)
|   +-- hpet_regs (PeakRDL generated)
|   +-- Interface mapping logic
|
+-- HPET Core
    +-- 64-bit counter
    +-- Timer comparators [NUM_TIMERS]
    +-- Interrupt generation [NUM_TIMERS]
```

11.2 Parameters

Parameter	Type	Default	Range	Description
VENDOR_ID	int	1	0-65535	Reported in HPET_ID[31:16] – GCAP_ID's full 16-bit vendor field, so 0x4747 reads back complete
REVISION_ID	int	1	0-255	Reported in HPET_ID[7:0] – GCAP_ID's revision field
COUNTER_CLK_PERIOD_FS	int	10000000	non-zero, <=	Femtoseconds per counter tick,

Parameter	Type	Default	Range	Description
			0x05F5E100	published at HPET_PERIOD (0x004)
NUM_TIMERS	int	2	2, 3, 8	Number of independent timers in array
USE_JOHNSON	int	0	0, 1	CDC counter encoding (forwarded to apb4_slave_cdc)
CDC_ENABLE	int	0	0, 1	Clock domain crossing: 0=synchronous, 1=asynchronous

Parameter Notes: - **VENDOR_ID** and **REVISION_ID**: forwarded to `hpet_config_regs`, which drives them into HPET_ID through the register block's hardware interface – no regeneration involved. `vendor_id` is the published spec's full 16 bits at HPET_ID[31:16] and `rev_id` is 8 bits at HPET_ID[7:0], so a PCI-style vendor reads back complete (RLB/hpet TASK-006). - **NUM_TIMERS**: Instantiation-time choice (2, 3, or 8). The generated register block is fixed at 8 timer slots and reports the instantiated count through HPET_ID.num_tim_cap – no regeneration needed - **CDC_ENABLE**: Critical for system integration - determines clock relationship

---

11.3 Ports

11.3.1 Clock and Reset - Dual Domain

Signal Name	Type	Width	Direction	Description
pclk	logic	1	Input	APB clock domain (always used for APB interface)
presetn	logic	1	Input	APB reset (active-low)
hpet_clk	logic	1	Input	HPET clock domain (used for timer logic)
hpet_resetn	logic	1	Input	HPET reset (active-

Signal Name	Type	Width	Direction	Description
				low)

Clock Constraints: - **CDC_ENABLE=0:** `pclk` and `hpet_clk` must be the same or synchronous - **CDC_ENABLE=1:** `pclk` and `hpet_clk` can be fully asynchronous

Reset Constraints: - **CDC_ENABLE=0:** `presetn` and `hpet_resetn` should be asserted/deasserted together - **CDC_ENABLE=1:** Both resets must overlap during power-on, can be independent afterward

11.3.2 APB4 Slave Interface (Low Frequency Domain)

Signal Name	Type	Width	Direction	Description
s_apb_PSEL	logic	1	Input	Peripheral select
s_apb_PENABLE	logic	1	Input	Enable signal
s_apb_PREADY	logic	1	Output	Ready signal
s_apb_PADDR	logic	12	Input	Address bus (fixed 12-bit addressing)
s_apb_PWRITE	logic	1	Input	Write enable (1=write, 0=read)
s_apb_PWDATA	logic	32	Input	Write data bus
s_apb_PSTRB	logic	4	Input	Write strobe (byte enables)
s_apb_PPROT	logic	3	Input	Protection type
s_apb_PRDATA	logic	32	Output	Read data bus
s_apb_PSLVERR	logic	1	Output	Slave error

Address Space: 12-bit addressing supports up to 4KB (0x000-0xFFF) - Global registers: 0x000-0x0FF - Timer registers: 0x100-0x1FF (32-byte spacing per timer) - 0x200-0xFFF: ALIASES of the 0x000-0x1FF map. Only address bits [8:0] reach the register block, so e.g. a read at 0x200 returns HPET_ID. No address ever raises PSLVERR – the register block ties both error outputs to 0.

11.3.3 Timer Interrupt Outputs (High Frequency Domain)

Signal Name	Type	Width	Direction	Description
timer_irq[NUM_TIMERS-1:0]	logic	NUM_TIMERS	Output	Per-timer interrupt outputs (active-high); timers 0/1 suppressed while HPET_CONFIG.lega

Signal Name	Type	Width	Direction	Description
				cy_replacement is set
legacy_irq0	logic	1	Output	Timer 0 on the LegacyReplacement Route (IRQ0); 0 unless legacy_replacement is set
legacy_irq8	logic	1	Output	Timer 1 on the LegacyReplacement Route (IRQ8); 0 unless legacy_replacement is set

Interrupt Characteristics: - **Active-high level-sensitive** - **One interrupt per timer** (independent) - Core interrupt state, NOT a copy of HPET_STATUS: set at fire time when int_enable is 1, cleared by the per-bit W1C clear or when the core status falls. HPET_STATUS mirrors the same core status, so the two agree; the one legitimate difference is a timer that fired with int_enable = 0, whose status bit reads 1 while timer_irq stays 0 - **W1C clearing** (software writes 1 to the timer's HPET_STATUS bit; other timers' bits and irq outputs are untouched)

11.4 Functional Description

11.4.1 Internal Signal Interfaces

11.4.1.1 CDC Command/Response Interface

Between APB Slave and Configuration Registers:

```
logic                    w_cmd_valid;
logic                    w_cmd_ready;
logic                    w_cmd_pwrite;
logic [11:0]             w_cmd_paddr;
logic [31:0]             w_cmd_pwdata;
logic [3:0]              w_cmd_pstrb;
logic [2:0]              w_cmd_pprot;

logic                    w_rsp_valid;
logic                    w_rsp_ready;
logic [31:0]                w_rsp_prdata;
logic                       w_rsp_pslverr;
```

Clock Domain: - **CDC_ENABLE=0:** Runs on `pclk` - **CDC_ENABLE=1:** Runs on `hpet_clk` (synchronized from pclk)

11.4.1.2 Configuration Register Interface

Between hpet_config_regs and hpet_core:

```
// Global configuration
logic                       w_hpet_enable;
logic                       w_legacy_replacement;    // -> hpet_core
(TASK-003)

// Counter interface (one strobe per 32-bit half)
logic                       w_counter_write_lo;
logic                       w_counter_write_hi;
logic [63:0]                w_counter_wdata;
logic [63:0]                w_counter_rdata;

// Per-timer configuration
logic [NUM_TIMERS-1:0]   w_timer_int_enable;      // no per-timer run
enable (TASK-006)
logic [NUM_TIMERS-1:0]   w_timer_type;
logic [NUM_TIMERS-1:0]   w_timer_size;
logic [NUM_TIMERS-1:0]   w_timer_value_set;

// Per-timer comparator (dedicated buses, one strobe per half)
logic [NUM_TIMERS-1:0]   w_timer_comp_write_lo;
logic [NUM_TIMERS-1:0]   w_timer_comp_write_hi;
logic [63:0]                w_timer_comp_wdata [NUM_TIMERS];

// Interrupt status
logic [NUM_TIMERS-1:0]   w_timer_int_status;
logic [NUM_TIMERS-1:0]   w_timer_int_clear;
```

11.4.2 APB Slave Conditional Generation

The top-level module uses a SystemVerilog `generate` block to conditionally instantiate the appropriate APB slave variant:

11.4.2.1 Non-CDC Configuration (CDC_ENABLE=0)

```
generate
    if (CDC_ENABLE != 0) begin : g_apb4_slave_cdc
        // ... CDC variant instantiation ...
    end else begin : g_apb4_slave_no_cdc
        // Non-CDC version for same clock domain (pclk == hpet_clk)
        apb4_slave #(
            .ADDR_WIDTH(12),
            .DATA_WIDTH(32),
            .STRB_WIDTH(4),
            .PROT_WIDTH(3)
        ) u_apb4_slave (
            // Single clock domain (use pclk for both APB and cmd/rsp)
            .pclk                 (pclk),
            .presetn              (presetn),

            // APB Interface
            .s_apb_PSEL           (s_apb_PSEL),
            .s_apb_PENABLE        (s_apb_PENABLE),
            .s_apb_PREADY         (s_apb_PREADY),
            .s_apb_PADDR          (s_apb_PADDR),
            .s_apb_PWRITE         (s_apb_PWRITE),
            .s_apb_PWDATA         (s_apb_PWDATA),
            .s_apb_PSTRB          (s_apb_PSTRB),
            .s_apb_PPROT          (s_apb_PPROT),
            .s_apb_PRDATA         (s_apb_PRDATA),
            .s_apb_PSLVERR        (s_apb_PSLVERR),

            // Command Interface (same pclk domain)
            .cmd_valid            (w_cmd_valid),
            .cmd_ready            (w_cmd_ready),
            .cmd_pwrite           (w_cmd_pwrite),
            .cmd_paddr            (w_cmd_paddr),
            .cmd_pwdata           (w_cmd_pwdata),
            .cmd_pstrb            (w_cmd_pstrb),
            .cmd_pprot            (w_cmd_pprot),

            // Response Interface (same pclk domain)
            .rsp_valid            (w_rsp_valid),
            .rsp_ready            (w_rsp_ready),
            .rsp_prdata           (w_rsp_prdata),
            .rsp_pslverr          (w_rsp_pslverr)
        );
    end
endgenerate
```

Characteristics: - **Latency:** 2 APB clock cycles (SETUP + ACCESS phases) - **Clock:** Single `pclk` domain - **Resources:** ~20 FF, ~50 LUTs

11.4.2.2 CDC Configuration (CDC_ENABLE=1)

```
generate
    if (CDC_ENABLE != 0) begin : g_apb4_slave_cdc
        // Clock Domain Crossing version for async clocks
        apb4_slave_cdc #(
            .ADDR_WIDTH(12),
            .DATA_WIDTH(32),
            .STRB_WIDTH(4),
            .PROT_WIDTH(3),
            .DEPTH     (2),
            .USE_JOHNSON(USE_JOHNSON)
        ) u_apb4_slave_cdc (
            // APB Clock Domain
            .pclk                 (pclk),
            .presetn              (presetn),

            // HPET Clock Domain
            .aclk                 (hpet_clk),
            .aresetn              (hpet_resetn),

            // APB Interface (pclk domain)
            .s_apb_PSEL           (s_apb_PSEL),
            .s_apb_PENABLE        (s_apb_PENABLE),
            .s_apb_PREADY         (s_apb_PREADY),
            .s_apb_PADDR          (s_apb_PADDR),
            .s_apb_PWRITE         (s_apb_PWRITE),
            .s_apb_PWDATA         (s_apb_PWDATA),
            .s_apb_PSTRB          (s_apb_PSTRB),
            .s_apb_PPROT          (s_apb_PPROT),
            .s_apb_PRDATA         (s_apb_PRDATA),
            .s_apb_PSLVERR        (s_apb_PSLVERR),

            // Command Interface (hpet_clk domain)
            .cmd_valid            (w_cmd_valid),
            .cmd_ready            (w_cmd_ready),
            .cmd_pwrite           (w_cmd_pwrite),
            .cmd_paddr            (w_cmd_paddr),
            .cmd_pwdata           (w_cmd_pwdata),
            .cmd_pstrb            (w_cmd_pstrb),
            .cmd_pprot            (w_cmd_pprot),

            // Response Interface (hpet_clk domain)
            .rsp_valid            (w_rsp_valid),
            .rsp_ready            (w_rsp_ready),
            .rsp_prdata           (w_rsp_prdata),
            .rsp_pslverr          (w_rsp_pslverr)
        );
    end else begin : g_apb4_slave_no_cdc
        // ... non-CDC variant instantiation ...
    end
endgenerate
```

Characteristics: - **Latency:** 4-6 APB clock cycles (CDC handshake overhead) - **Clocks:** Dual domains (pclk and hpet_clk) - **Resources:** ~100 FF, ~150 LUTs (additional CDC logic)

11.4.3 Clock Domain Assignment

Configuration registers and HPET core run in a clock domain determined by `CDC_ENABLE`:

```
// HPET Configuration Registers
hpet_config_regs #(
    .VENDOR_ID        (VENDOR_ID),
    .REVISION_ID      (REVISION_ID),
    .NUM_TIMERS       (NUM_TIMERS)
) u_hpet_config_regs (
    // Clock and Reset - conditional based on CDC_ENABLE
    .clk               (CDC_ENABLE[0] ? hpet_clk : pclk),
    .rst_n             (CDC_ENABLE[0] ? hpet_resetn : presetn),
    // ... interface connections ...
);

// HPET Timer Core
hpet_core #(
    .NUM_TIMERS(NUM_TIMERS)
) u_hpet_core (
    // Clock and Reset - conditional based on CDC_ENABLE
    .clk                  (CDC_ENABLE[0] ? hpet_clk : pclk),
    .rst_n                (CDC_ENABLE[0] ? hpet_resetn : presetn),
    // ... interface connections ...
);
```

Clock Assignment Logic: - **CDC_ENABLE=0:** Both use `pclk` and `presetn` - **CDC_ENABLE=1:** Both use `hpet_clk` and `hpet_resetn`

Rationale: Configuration registers and timer core must run in the same domain. APB slave handles the clock crossing (if needed).

11.5 Usage Example

11.5.1 Example 1: Synchronous Configuration (CDC_ENABLE=0)

```
apb4_hpet #(
    .NUM_TIMERS(2),            // VENDOR_ID/REVISION_ID left at their
0x01 defaults
    .CDC_ENABLE(0)              // ← Synchronous clocks
) u_hpet (
    // Use same clock for both domains
    .pclk           (system_clk),
    .presetn        (system_rst_n),
    .hpet_clk       (system_clk),      // ← Same clock as pclk
    .hpet_resetn    (system_rst_n),    // ← Same reset as presetn

    // APB Interface
    .s_apb_PSEL     (apb_psel),
    .s_apb_PENABLE  (apb_penable),
    .s_apb_PREADY   (apb_pready),
    .s_apb_PADDR    (apb_paddr[11:0]),
    .s_apb_PWRITE   (apb_pwrite),
    .s_apb_PWDATA   (apb_pwdata),
    .s_apb_PSTRB    (apb_pstrb),
    .s_apb_PPROT    (apb_pprot),
    .s_apb_PRDATA   (apb_prdata),
    .s_apb_PSLVERR  (apb_pslverr),

    // Timer Interrupts
    .timer_irq      (hpet_irq[1:0])
);

// Connect interrupts to system interrupt controller
assign irq_sources[31:30] = hpet_irq[1:0];
```

11.5.2 Example 2: Asynchronous Configuration (CDC_ENABLE=1)

```
apb4_hpet #(
    .NUM_TIMERS(3),            // VENDOR_ID/REVISION_ID left at their 0x01 defaults
    .CDC_ENABLE(1)              // ← Asynchronous clocks
) u_apb4_hpet (
    .pclk                  (pclk),
    .presetn               (presetn),
    .hpet_clk              (hpet_clk),
    .hpet_resetn           (hpet_resetn),
    .s_apb_PSEL            (s_apb_PSEL),
    .s_apb_PENABLE         (s_apb_PENABLE),
    .s_apb_PREADY          (s_apb_PREADY),
    .s_apb_PADDR           (s_apb_PADDR),
    .s_apb_PWRITE          (s_apb_PWRITE),
    .s_apb_PWDATA          (s_apb_PWDATA),
    .s_apb_PSTRB           (s_apb_PSTRB),
    .s_apb_PPROT           (s_apb_PPROT),
    .s_apb_PRDATA          (s_apb_PRDATA),
    .s_apb_PSLVERR         (s_apb_PSLVERR),
    .timer_irq             (timer_irq)
);
```

11.6 Design Notes

11.6.1 Resource Utilization Summary

Total Resource Usage by Configuration:

Configuration	NUM_TIMERS	CDC_ENABLE	Flip-Flops	LUTs	BRAM
2-timer sync	2	0	~528 FF	~510 LUTs	0
3-timer sync	3	0	~718 FF	~680 LUTs	0
8-timer sync	8	0	~1544 FF	~1360 LUTs	0
2-timer CDC	2	1	~608 FF	~610 LUTs	0
3-timer CDC	3	1	~798 FF	~780 LUTs	0
8-timer CDC	8	1	~1624 FF	~1460 LUTs	0

Resource Breakdown: - **APB Slave (no CDC):** ~20 FF, ~50 LUTs - **APB Slave CDC:** ~100 FF, ~150 LUTs - **Config Registers:** Scales with NUM_TIMERS (~35 FF + ~70 LUTs per timer) - **HPET Core:** Scales with NUM_TIMERS (~128 FF + ~85 LUTs per timer)

11.7 Testing

11.7.1 Verification Checklist

Integration Validation:

- ☐ **Clock Configuration:**
  - ☐ If CDC_ENABLE=0: Verify pclk = hpet_clk
  - ☐ If CDC_ENABLE=1: Verify independent clock sources
- ☐ **Reset Coordination:**
  - ☐ Both resets overlap at power-on
  - ☐ Both resets held for >=10 cycles
  - ☐ Reset deasserted cleanly

- ☐ **APB Interface:**
  - ☐ Read/write to all registers functional
  - ☐ Address decoding correct
  - ☐ PREADY timing appropriate (2 cycles sync, 4-6 cycles CDC)
- ☐ **Timer Operation:**
  - ☐ All NUM_TIMERS functional
  - ☐ One-shot mode works
  - ☐ Periodic mode works
  - ☐ Counter increments correctly
- ☐ **Interrupt Generation:**
  - ☐ All timer_irq outputs functional
  - ☐ W1C clearing works, per bit (a write of 0x0 clears nothing)
  - ☐ Sticky behavior correct
  - ☐ Re-enabling with an expired comparator does not re-fire
- ☐ **CDC (if enabled):**
  - ☐ No metastability issues
  - ☐ Data integrity across domains
  - ☐ Proper handshake protocol

11.8 Navigation

Next: Chapter 2.5 - FSM Summary

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

12 APB HPET FSM Summary

12.1 Overview

The APB HPET component contains multiple state machines across different modules. This chapter collects all of them – states, transitions, and interactions – in one place so you don't have to chase them across the block pages.

12.1.1 FSM Inventory

Module	FSM Name	Type	States	Purpose
apb4_slave	APB Protocol FSM	Explicit	2-3	APB handshake protocol
apb4_slave_cdc	CDC Handshake FSM	Explicit	4	Clock domain crossing protocol
hpet_core	Per-Timer FSM	Conceptual	5	Timer operation and fire control

Note: The hpet_config_regs and hpet_regs modules use combinational and sequential logic without explicit state machines.

12.2 Functional Description

12.2.1 1. APB Slave Protocol FSM

Module: `apb4_slave.sv` **Clock Domain:** `pclk` **Implementation:** Explicit state register

12.2.1.1 States

State	Encoding	Description
IDLE	2'b00	Waiting for PSEL assertion
SETUP	2'b01	PSEL asserted, waiting for PENABLE
ACCESS	2'b10	PENABLE asserted, transaction active

12.2.1.2 State Transitions

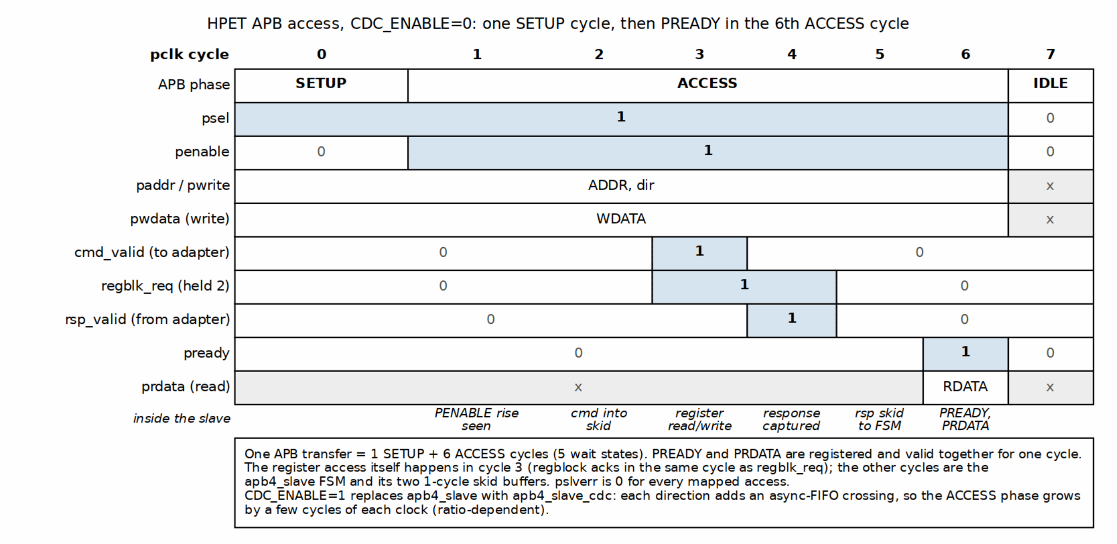
IDLE -> SETUP: - **Condition:** `PSEL = 1` - **Action:** Latch address, write data, and control signals - **Duration:** 1 clock cycle

SETUP -> ACCESS: - **Condition:** `PENABLE = 1` (always follows SETUP in next cycle) - **Action:** Assert cmd_valid to downstream, wait for rsp_valid - **Duration:** Variable (1 cycle minimum, waits for rsp_valid)

ACCESS -> IDLE: - **Condition:** `rsp_valid = 1` (response received) - **Action:** Assert PREADY, complete transaction - **Duration:** Immediate return to IDLE

ACCESS -> IDLE (Early Termination): - **Condition:** `PSEL = 0` (transaction aborted) - **Action:** Deassert cmd_valid, return to IDLE - **Duration:** Immediate

12.2.2 Figure 2.22: APB Access Timing



HPET APB access, CDC_ENABLE=0: one SETUP cycle, then PREADY in the 6th ACCESS cycle

pclk cycle	0	1	2	3	4	5	6	7
APB phase	SETUP	ACCESS						IDLE
psel	1							0
penable	0	1						0
paddr / pwrite	ADDR, dir							x
pwdata (write)	WDATA							x
cmd_valid (to adapter)	0			1	0			
regblk_req (held 2)	0			1		0		
rsp_valid (from adapter)	0				1	0		
pready	0						1	0
prdata (read)	x						RDATA	x
inside the slave		*PENABLE rise seen*	*cmd into skid*	*register read/write*	*response captured*	*rsp skid to FSM*	*PREADY, PRDATA*	

One APB transfer = 1 SETUP + 6 ACCESS cycles (5 wait states). PREADY and PRDATA are registered and valid together for one cycle. The register access itself happens in cycle 3 (regblock acks in the same cycle as regblk_req); the other cycles are the apb4_slave FSM and its two 1-cycle skid buffers. pslverr is 0 for every mapped access.
CDC_ENABLE=1 replaces apb4_slave with apb4_slave_cdc: each direction adds an async-FIFO crossing, so the ACCESS phase grows by a few cycles of each clock (ratio-dependent).

APB Access Timing

12.2.3 2. APB Slave CDC (async-FIFO crossing)

Module: `apb4_slave_cdc.sv` **Clock Domains:** `pclk` (APB side) and `aclk` (application side) **Implementation:** NOT a handshake FSM. The wrapper instantiates the plain `apb4_slave` in the pclk domain and crosses its command and response streams through two `gaxi_fifo_async` instances (`u_cmd_cdc_fifo`, `u_rsp_cdc_fifo`, DEPTH=2 here), with Gray- or Johnson-coded pointers selected by the USE_JOHNSON parameter. There are no request/acknowledge toggle FSMs and no per-signal 2-stage synchronizers on the data path – the FIFO pointer synchronizers are the only crossing logic.

Latency per access spans the apb4_slave FSM plus one FIFO traversal each way (a few cycles of each clock domain, ratio-dependent).

12.2.43. HPET Core Per-Timer FSM

Module: `hpet_core.sv` **Clock Domain:** `hpet_clk` (or `pclk` if CDC_ENABLE=0) **Implementation:** Conceptual FSM (implemented as combinational logic, not explicit state register)

Note: The HPET core uses a conceptual FSM model for specification clarity, but the actual implementation is a raw comparator plus two state bits per timer – the armed latch and the next-epoch hold bit – rather than an explicit state register. This provides simpler timing and resource usage while maintaining the same functional behavior.

12.2.4.1 States

State	Description	Duration
IDLE	Counter halted (`hpet_enable = 0`); there is no per-timer run enable	Until the counter runs
ARMED	Counter running, monitoring counter vs comparator	Until counter match
FIRE	Armed match detected, asserting interrupt	1 cycle (one-cycle pulse)
PERIODIC_RELOAD	Periodic mode: auto-increment comparator	1 cycle, or one boundary per cycle while catching up (counter + 1 at period 1)
ONE_SHOT_COMPLETE	One-shot mode: fired, armed latch clear, waiting for a comparator write with the counter halted or for the match to fall	Until the comparator is rewritten (counter halted) or written above the counter, or the counter halted

An epoch-held timer – one whose advance carried out of the compare width, setting `r_comp_next_epoch[i]` – is not a state of its own. It looks ARMED-but-not-matching: the latch is set, the match is forced off, and it sits there with no fire and no catch-up until the counter wraps at that width (or software writes the comparator or counter half the comparison reads – LO only in 32-bit mode, either half in 64-bit – or changes `timer_size`).

12.2.4.2 State Transition Conditions

IDLE -> ARMED: - **Condition:** `hpet_enable = 1` (block-wide; the published spec defines no per-timer run enable, and `timer_int_enable[i]` gates only the interrupt) - **Action:** Latch current comparator value, begin monitoring - **Trigger:** Rising edge of `hpet_enable`

ARMED -> FIRE: - **Condition:** `counter >= comparator[i]` with the armed latch set - **Action:** Assert `timer_int_status[i]` (sticky, on every fire whatever `timer_int_enable[i]` says), clear the armed latch; the interrupt output is set at the same edge, only if `timer_int_enable[i]` = 1 - **Trigger:** Counter comparison (combinational), gated by `hpet_enable` and the latch

FIRE -> PERIODIC_RELOAD: - **Condition:** `timer_type[i] = 1` (periodic mode) - **Action:** `comparator[i] <= comparator[i] + period[i]` at the compare width; a carry out of that width sets the next-epoch hold bit instead of reaching the comparator - **Trigger:** Immediate (next clock cycle after fire)

FIRE -> ONE_SHOT_COMPLETE: - **Condition:** `timer_type[i] = 0` (one-shot mode) - **Action:** Hold `timer_int_status[i]`, interrupt remains asserted - **Trigger:** Immediate (next clock cycle after fire)

PERIODIC_RELOAD -> ARMED: - **Condition:** The advanced comparator is ahead of the counter (the usual case: the match falls and the latch re-arms) - **Action:** Resume monitoring with new comparator value - **Trigger:** Immediate (next clock cycle) - **Catch-up:** if the advanced comparator is still at or below the counter, the timer stays in RELOAD, advancing one boundary per cycle WITHOUT firing, until the advance lands at or ahead of the counter (>=, or a carry); that step re-arms the latch. Missed periods are skipped, never burst. Period 0 cannot get ahead and is treated as one-shot (fires once, no churn). Period 1 gains nothing on the counter per cycle, so its catch-up step sets the comparator to counter + 1 (every integer is on the period-1 lattice) and re-arms at once: one fire every other tick, at any deficit. Any other period closes its deficit at period - 1 counts per cycle, one advance per cycle, so the catch-up is proportional to the deficit – a period-2 timer left 2^52 behind by a counter write catches up for ~2^52 cycles with no interrupt, which is why the counter is written halted - **Epoch hold:** an advance that carries out of the compare width leaves the timer ARMED-but-not-matching until the counter wraps at that width; no fire and no further catch-up in between

ONE_SHOT_COMPLETE -> ARMED: - **Condition:** `timer_comp_write_lo[i]` or `timer_comp_write_hi[i]` while the counter is HALTED (`hpet_enable = 0`; software writes the LO or HI half, even with the value it already holds), or the raw match falling (comparator written above the counter, or the counter reloaded below it) - **Action:** Set the armed latch and resume monitoring. A comparator written with the counter halted need not exceed the counter: a value already passed fires on the first cycle the counter runs - **Trigger:** Comparator write strobe with the counter halted, or the match falling

A 0->1 of `hpet_enable` while counter >= comparator does NOT re-enter FIRE: the armed latch is the fired-state memory, and the enable cannot set it. A completed one-shot stays complete across a halt/restart cycle.

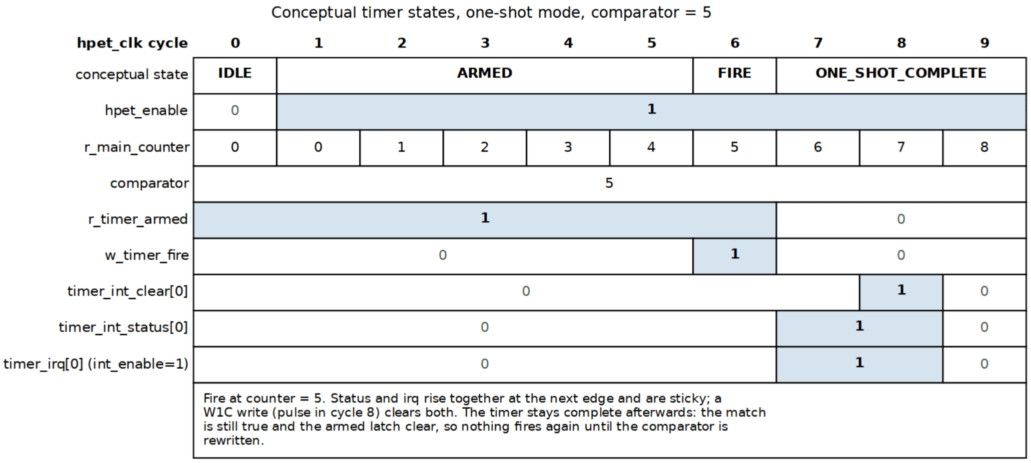
A comparator write on a RUNNING timer does not re-arm either: the half loads, and the timer re-arms only when the completed value moves the match low. That keeps the torn {old HI, new LO} between two half-writes from ever firing. The contract is to halt the counter before reprogramming it; a write with the counter HALTED always re-arms, and a value at or below the counter fires as soon as the counter restarts. Reprogramming a running timer anyway has two consequences that are outside the contract, not bugs: a torn 64-bit value above the counter re-arms through the natural path and fires at the final value once it is written, and a same-value rewrite on a running periodic timer drags the comparator back and can add one off-lattice fire.

ARMED -> IDLE: - **Condition:** `hpet_enable = 0` - **Action:** Stop match generation. A pending interrupt status/output is NOT cleared by halting – it holds until software W1C or reset. - **Trigger:** Falling edge of `hpet_enable`

ONE_SHOT_COMPLETE -> IDLE: - **Condition:** `hpet_enable = 0` - **Action:** Stop monitoring (pending status, if any, holds until W1C) - **Trigger:** Counter halt. Clearing `timer_int_enable[i]` is not a stop

12.2.4.3 FSM Timing Examples

12.2.5 Figure 2.23: One-Shot Mode State Timeline



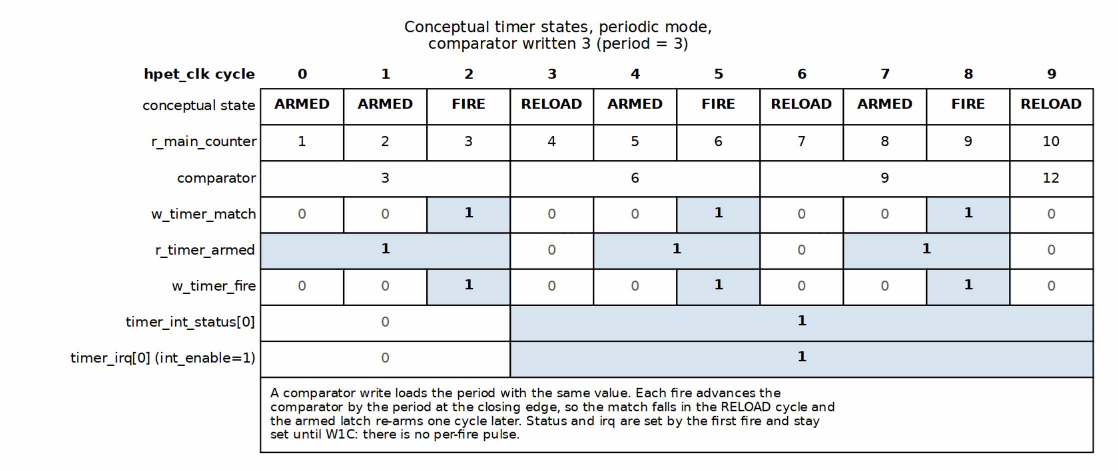
Conceptual timer states, one-shot mode, comparator = 5

hpet_clk cycle	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
conceptual state	**IDLE**	**ARMED**					**FIRE**	**ONE_SHOT_COMPLETE**		
hpet_enable	0	**1**								
r_main_counter	0	0	1	2	3	4	5	6	7	8
comparator	5									
r_timer_armed	**1**							0		
w_timer_fire	0						**1**	0		
timer_int_clear[0]	0								**1**	0
timer_int_status[0]	0							**1**		0
timer_irq[0] (int_enable=1)	0							**1**		0

Fire at counter = 5. Status and irq rise together at the next edge and are sticky; a W1C write (pulse in cycle 8) clears both. The timer stays complete afterwards: the match is still true and the armed latch clear, so nothing fires again until the comparator is rewritten.

One-Shot Mode State Timeline

12.2.6 Figure 2.24: Periodic Mode State Timeline



Conceptual timer states, periodic mode,
comparator written 3 (period = 3)

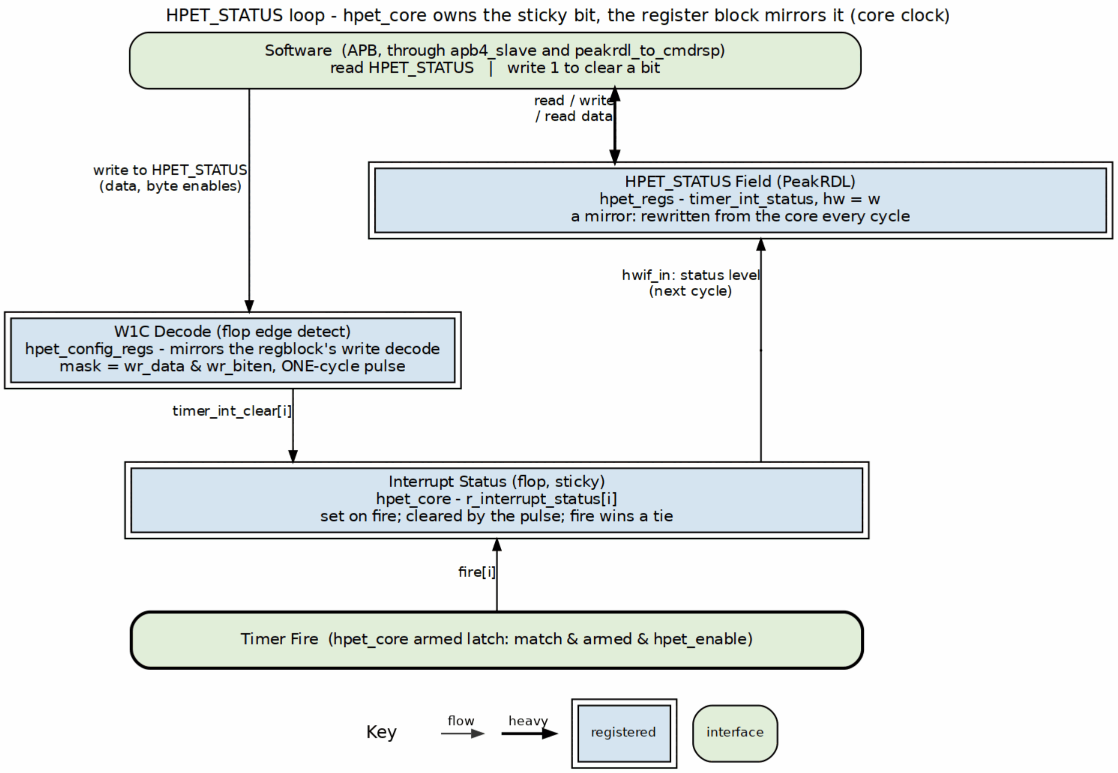
hpet_clk cycle	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
conceptual state	**ARMED**	**ARMED**	**FIRE**	**RELOAD**	**ARMED**	**FIRE**	**RELOAD**	**ARMED**	**FIRE**	**RELOAD**
r_main_counter	1	2	3	4	5	6	7	8	9	10
comparator	3			6			9			12
w_timer_match	0	0	**1**	0	0	**1**	0	0	**1**	0
r_timer_armed	**1**			0	**1**		0	**1**		0
w_timer_fire	0	0	**1**	0	0	**1**	0	0	**1**	0
timer_int_status[0]	0			**1**						
timer_irq[0] (int_enable=1)	0			**1**						

A comparator write loads the period with the same value. Each fire advances the comparator by the period at the closing edge, so the match falls in the RELOAD cycle and the armed latch re-arms one cycle later. Status and irq are set by the first fire and stay set until W1C: there is no per-fire pulse.

Periodic Mode State Timeline

12.2.7 FSM Interaction Summary

12.2.7.1 Cross-Module State Dependencies


HPET_STATUS loop - hpet_core owns the sticky bit, the register block mirrors it (core clock)
Software (APB, through apb4_slave and peakrdl_to_cmdrsp)
read HPET_STATUS | write 1 to clear a bit
read / write
/ read data
write to HPET_STATUS
(data, byte enables)
HPET_STATUS Field (PeakRDL)
hpet_regs - timer_int_status, hw = w
a mirror: rewritten from the core every cycle
hwif_in: status level
(next cycle)
W1C Decode (flop edge detect)
hpet_config_regs - mirrors the regblock's write decode
mask = wr_data & wr_biten, ONE-cycle pulse
timer_int_clear[i]
Interrupt Status (flop, sticky)
hpet_core - r_interrupt_status[i]
set on fire; cleared by the pulse; fire wins a tie
fire[i]
Timer Fire (hpet_core armed latch: match & armed & hpet_enable)
Key
flow
heavy
registered
interface

HPET_STATUS loop

12.2.7.2 Clock Domain Considerations

Synchronous Mode (CDC_ENABLE=0): - All FSMs run on `pclk` - No synchronization required - Direct signal propagation

Asynchronous Mode (CDC_ENABLE=1): - APB Slave CDC FSM bridges `pclk` and `hpet_clk` - Configuration registers and timers run on `hpet_clk` - Handshake protocol ensures data stability

---

12.3 Design Notes

12.3.1 State Machine Design Patterns

12.3.1.1 Pattern 1: Explicit State Register (APB Slave)

```
typedef enum logic [1:0] {
    IDLE   = 2'b00,
    SETUP  = 2'b01,
    ACCESS = 2'b10
} state_t;

state_t r_state, w_next_state;

always_ff @(posedge pclk or negedge presetn) begin
    if (!presetn) r_state <= IDLE;
    else          r_state <= w_next_state;
end

always_comb begin
    w_next_state = r_state;  // Default: hold state
    case (r_state)
        IDLE:   if (PSEL)               w_next_state = SETUP;
        SETUP:  if (PENABLE)            w_next_state = ACCESS;
        ACCESS: if (rsp_valid || !PSEL) w_next_state = IDLE;
    endcase
end
```

Characteristics: - Explicit state storage - Separate combo/sequential blocks - Easy to verify and debug - Standard FSM coding style

12.3.1.2 Pattern 2: Combinational Match with an Armed Latch (Timer FSM)

```
// No explicit state register - a raw comparator plus two state bits

// Raw match: deliberately NO enable terms; held off by the epoch bit
assign w_timer_match_raw[i] = (counter >= comparator[i]);   // at the
compare width
```

```
assign w_timer_match[i]     = w_timer_match_raw[i] && !
r_comp_next_epoch[i];

// Periodic catch-up: running, periodic, un-armed, still matching, no
// software write this cycle, period able to move the compare.
Advances
// the comparator a boundary per cycle without firing (counter + 1 at
// period 1); re-arms the cycle the advance lands at or ahead of the
// counter (w_comp_adv_ahead[i] = carry || advance >= counter).
assign w_comp_write_rearm[i] = w_timer_comp_write[i] && !
w_timer_running[i];
assign w_timer_catchup[i]    = timer_type[i] && !r_timer_armed[i] &&
w_timer_match[i] &&
                               !w_timer_comp_write[i] &&
w_timer_period_nz[i] &&
                               w_timer_running[i];
assign w_catchup_rearm[i]    = w_timer_catchup[i] &&
w_comp_adv_ahead[i];

// Next-epoch hold: set when an advance carries out of the compare
width;
// cleared (clear beats set) by the counter wrapping at that width, a
// comparator write, a counter write or a timer_size change. The write
// clears are taken at the compare width
(w_epoch_clr_comp/w_epoch_clr_ctr,
// built in the same mux as the match): in 32-bit mode only a LO-half
write
// clears, a HI-half write leaves the hold in place. Reset 0.
assign w_epoch_set[i] = w_comp_advance_en[i] && w_comp_adv_carry[i];
assign w_epoch_clr[i] = w_counter_wrap[i] || w_epoch_clr_comp[i] ||
                        w_epoch_clr_ctr[i] || w_timer_size_chg[i];
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)                r_comp_next_epoch[i] <= 1'b0;
    else if (w_epoch_clr[i])   r_comp_next_epoch[i] <= 1'b0;
    else if (w_epoch_set[i])   r_comp_next_epoch[i] <= 1'b1;
end

// Armed latch: set on a comparator write with the counter HALTED,
when the
// gated match drops, or when a catch-up step lands at or ahead of the
// counter; cleared when the timer fires. Reset armed.
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)                                  r_timer_armed[i] <=
1'b1;
    else if (w_comp_write_rearm[i] || !w_timer_match[i] ||
             w_catchup_rearm[i])                 r_timer_armed[i] <=
1'b1;
    else if (w_timer_fire[i])                    r_timer_armed[i] <=
1'b0;
end

// Fire: match, still armed, counter running (no per-timer run enable)
assign w_timer_fire[i] = w_timer_match[i] && r_timer_armed[i] &&
                         hpet_enable;

// Sticky interrupt status -- identical in BOTH modes (no timer_type
term)
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_interrupt_status[i] <= 1'b0;
    end else if (w_timer_fire[i]) begin
        r_interrupt_status[i] <= 1'b1;   // Set on fire (wins over a
clear)
    end else if (timer_int_clear[i]) begin
        r_interrupt_status[i] <= 1'b0;   // Software W1C, this bit
only
    end
end
```

Characteristics: - No explicit state register; the armed latch and the next-epoch hold bit are the only state bits - One fire per arm, so `hpet_enable` cannot manufacture a fire - A comparator write re-arms only with the counter halted, so a half-written 64-bit comparator cannot fire on the torn value - Periodic catch-up skips missed periods without bursting; period 1 steps to counter + 1 and keeps its every-other-tick cadence at any deficit - An advance that carries out of the compare width is held off until the counter wraps there, instead of matching early or churning - Simpler implementation - Lower resource usage - Same functional behavior as FSM

12.4 Testing

12.4.1 FSM Verification Considerations

12.4.1.1 State Coverage

APB Slave FSM: - [ ] IDLE state entry and exit - [ ] SETUP state timing (1 cycle) - [ ] ACCESS state with response wait - [ ] ACCESS state early termination (PSEL deassert)

CDC Handshake FSM: - [ ] Request synchronization (pclk -> aclk) - [ ] Response synchronization (aclk -> pclk) - [ ] Concurrent requests handling - [ ] Clock ratio corner cases (fast pclk, slow aclk and vice versa)

Timer FSM: - [ ] IDLE -> ARMED transition - [ ] ARMED -> FIRE on match - [ ] FIRE -> PERIODIC_RELOAD path - [ ] FIRE -> ONE_SHOT_COMPLETE path - [ ] PERIODIC_RELOAD ->

ARMED auto-transition - [ ] ONE_SHOT_COMPLETE -> ARMED on reconfigure - [ ] Return to IDLE on counter halt

12.4.1.2 Transition Coverage

Edge Cases: - [ ] Counter halt/restart during active timer - [ ] Comparator write during countdown - [ ] Counter write during active timer - [ ] Multiple timers firing simultaneously - [ ] Interrupt clear during fire event (the fire wins) - [ ] Counter restart with counter >= comparator (must NOT re-fire) - [ ] Comparator rewritten with the same value (reloads the comparator; re-arms only if the counter is halted or the match falls) - [ ] 64-bit comparator reprogrammed on a running timer (the torn {old HI, new LO} never fires; halt-write-restart fires once) - [ ] Comparator write in the same cycle as a fire (exactly one fire, software value wins, no periodic advance that cycle) - [ ] Periodic timer more than one period behind the counter (catch-up: exactly one fire, missed periods skipped) - [ ] Period 0 (fires once, no churn) and period 1 (every other tick) - [ ] Period-1 timer with a deficit (catch-up steps to counter + 1 and recovers within a cycle; a live counter write that opens a gap) - [ ] Comparator advance carrying out of the compare width (32- and 64-bit: epoch hold, no fire and no churn until the counter wraps; comparator[63:32] untouched in 32-bit mode; clear beats set at the wrap) - [ ] Counter write while epoch-held (re-base: a comparator now at or below the counter fires promptly; in 32-bit mode a HI-half write to counter or comparator leaves the hold in place), and a timer_size change while epoch-held (hold cleared) - [ ] Mode switch (one-shot ↔️ periodic) mid-operation

---

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

13 Claude Code Guide: Retro Legacy Blocks

Version: 2.0 **Last Updated:** 2025-10-29 **Purpose:** AI-specific guidance for working with Retro Legacy Blocks (RLB) peripheral blocks

---

13.1 Quick Context

What: Collection of production-quality retro-compatible legacy peripherals **Status:** Active Development - HPET Production Ready, 12 more blocks planned **Your Role:** Help users develop new legacy blocks, integrate existing blocks, understand RLB architecture

Complete Documentation: - `projects/components/legacy-ip/retro_legacy_blocks/PRD.md` ← Master requirements for all blocks - `projects/components/legacy-ip/retro_legacy_blocks/README.md` ← Component overview and usage guide - `docs/hpet_mas/hpet_mas_index.md` ← HPET complete specification

RLB Address Map: Single APB entry point at `0xFEC0_0000` (the `BASE_ADDR` parameter default), ten 4KB windows decoded by the generated `apbx_xbar_1to10`

13.2 Critical Rules for All Blocks

13.2.1 Rule #0.1: Reset Macro Standards - MANDATORY FOR ALL BLOCKS

Use `ALWAYS_FF_RST` / `RST_ASSERTED` from `reset_defs.svh`; a hand-written `always_ff @(posedge clk or negedge rst_n)` is rejected. Enforced repo-wide – `/GLOBAL_REQUIREMENTS.md` 1.1 is the authority, the rationale and the failures behind it are `vault/handbook/design/reset-and-clocking.md`, and `bin/update_resets.py` converts existing code.

13.2.2 Rule #0.2: FPGA Synthesis Attributes - MANDATORY

Every memory array carries vendor synthesis hints (`ram_style` / `ramstyle`) or a large memory synthesises into logic. Enforced repo-wide: `/GLOBAL_REQUIREMENTS.md` 1.2; see `vault/handbook/design/sram-and-memories.md`.

13.2.3 Rule #0.3: Testbench Architecture - MANDATORY SEPARATION

A TB class never lives in a test runner; `/GLOBAL_REQUIREMENTS.md` 2.1 is the authority. What is specific to this component is the layout and the tier names:

```
projects/components/legacy-ip/retro_legacy_blocks/dv/
├── tbclasses/{block}/            # block-specific TB classes
│   ├── {block}_tb.py
│   ├── {block}_tests_basic.py     # runs at TEST_LEVEL=gate
│   ├── {block}_tests_medium.py    # runs at TEST_LEVEL=func
│   └── {block}_tests_full.py      # runs at TEST_LEVEL=full
└── tests/                         # runners, FLAT:
test_apb4_{block}.py, conftest.py
```

The tier FILENAMES are historical; the LEVELS are gate/func/full. `{block}_tests_basic.py` defines `{Block}BasicTests` and runs at `TEST_LEVEL=gate`; `_medium.py` / `{Block}MediumTests` runs at `func`. The files and classes really are named that way on disk – do not rename them to match the level, and do not read the filename as the level.

13.2.4 Rule #0.4: Test Hierarchy - 3 Levels Required

Every block carries all three levels. The gate/func/full convention is repo-wide (`vault/handbook/dv/running-regressions.md`); the per-block targets here are:

Level	Tests	Pass rate	Per-test duration
gate	4-6	100%	<30 s
func	5-8	100%	30-90 s
full	3-5	>=95%	90 s+

13.2.5 Rule #0.5: Register Generation - Use PeakRDL

Registers come from SystemRDL via `bin/peakrdl_generate.py` only – never raw peakrdl, which skips the docs and regmap the wrapper emits in lockstep (`vault/handbook/design/generated-rtl-discipline.md`).

Run it from the block’s RDL directory and point `--copy-rtl` at the block’s RTL directory, since the two are no longer parent and child (RLB TASK-007, was RLB-007):

```
cd rdl/{block}
python $REPO_ROOT/bin/peakrdl_generate.py {block}_regs.rdl --copy-
rtl ../../rtl/{block}
```

See `rdl/hpet/` for a complete example.

13.3 Block Development Workflow

13.3.1 Adding a New Block

1. Directory structure:

```
cd projects/components/legacy-ip/retro_legacy_blocks
mkdir -p rdl/{block}                  # SystemRDL: the source of truth
mkdir -p rtl/{block}/filelists        # component + integration
filelists
mkdir -p dv/tbclasses/{block}         # TB classes; runners stay flat
in dv/tests/
mkdir -p docs/{block}_mas
```

```
rtl/{block}/
├── apb_{block}.sv          # top-level wrapper
├── {block}_core.sv         # core logic
├── {block}_config_regs.sv  # register wrapper
├── {block}_regs_pkg.sv     # PeakRDL generated
├── {block}_regs.sv         # PeakRDL generated
├── filelists/{component,integration}
└── Makefile
```

The RDL lives outside `rtl/` on purpose: every SystemRDL source in the repo belongs in an `rdl` area rather than scattered under the RTL it generates (RLB TASK-007 (was RLB-007), and projects/components/utility-ip/misc TASK-001 (was MISC-001) repo-wide). There is no README beside it – a file next to a tool restating how to run the tool is the copy nobody edits.

2. RTL, TB classes, test suites and runner take the standard shapes: Pattern B (`cocotb_test_*` functions behind pytest wrappers), the three mandatory TB methods, and the tier files named in Rule #0.3. Templates are the `test-patterns` skill and `/GLOBAL_REQUIREMENTS.md` 2.2 / 3.2. A filelist lands in the same commit as the module (`vault/handbook/design/filelists.md`).

3. Documentation: add the block to `PRD.md`, create `docs/{block}_mas/{block}_mas_index.md`, update the `README.md` status table.

13.4 HPET-Specific Guidance

13.4.1 HPET Quick Reference

Status: Production Ready (5/6 configurations 100% passing) **RTL Location:** `rtl/hpet/` **Test Location:** `dv/tests/test_apb4_hpet.py`

13.4.2 Critical HPET Rules

13.4.2.1 Rule #1: Timer Cleanup is MANDATORY

ALWAYS Reset Counter Between Tests

```
# CORRECT: Clean up at end of test
async def test_64bit_counter(self):
    await self.tb.write_register(HPET_COUNTER_LO, 0xFFFFFFFF)
    # ... test logic ...

    # MANDATORY cleanup
    await self.tb.write_register(HPET_COUNTER_LO, 0x0)
    await self.tb.write_register(HPET_COUNTER_HI, 0x0)
    return True
```

Why: Test leaves counter at high value, next test expects counter at 0. Timer 2+ won't fire if counter starts high.

13.4.2.2 Rule #2: Timer Timeout Calculations

Account for counter starting value when setting timeouts:

```
# Calculate timeout based on timer periods
timer_configs = [
    {"period": 100},  # Timer 0 fires at 100
    {"period": 200},  # Timer 1 fires at 200
    {"period": 700},  # Timer 2 fires at 700 (needs most time)
]

# 3x safety margin for latest timer
timeout_ns = max(cfg["period"] for cfg in timer_configs) * 3
timeout_us = (timeout_ns + 999) // 1000
```

13.4.2.3 Rule #3: HPET Register Map

The PUBLISHED spec map (RLB/hpet TASK-006); `rdl/hpet/hpet_regs.rdl` is the source of truth. Reserved offsets read 0 with no PSLVERR.

```
0x000: HPET_ID              GCAP_ID[31:0] RO: rev_id[7:0],
num_tim_cap[12:8] (N-1),
                            count_size_cap[13]=1, leg_rt_cap[15]=1,
vendor_id[31:16] (16 bits)
0x004: HPET_PERIOD          GCAP_ID[63:32] RO: femtoseconds per tick
(COUNTER_CLK_PERIOD_FS)
0x010: HPET_CONFIG          GEN_CONF: hpet_enable[0],
legacy_replacement[1] (T0->IRQ0, T1->IRQ8)
0x020: HPET_STATUS          GINTR_STA: timer interrupt status, W1C;
set on EVERY fire
0x0F0: HPET_COUNTER_LO      main counter bits [31:0], RW
```

```
0x0F4: HPET_COUNTER_HI      main counter bits [63:32], RW

Per-Timer Registers (i = 0 to NUM_TIMERS-1):
0x100 + i*0x20: TIMER[i]_CONFIG         TN_CONF[31:0]; reads 0x30 out
of reset
0x104 + i*0x20: TIMER[i]_INT_ROUTE_CAP   TN_CONF[63:32], RO, reads 0
0x108 + i*0x20: TIMER[i]_COMPARATOR_LO   bits [31:0], RW
0x10C + i*0x20: TIMER[i]_COMPARATOR_HI   bits [63:32], RW

TIMER_CONFIG (TN_CONF): int_type[1] (storage), int_enable[2], type[3]
(1=periodic),
  per_int_cap[4]=1 RO, size_cap[5]=1 RO, value_set[6] (storage),
32mode[8],
  int_route[13:9] (storage), fsb_en[14] (storage),
fsb_int_del_cap[15]=0 RO
```

Two traps: there is NO per-timer run enable – `int_enable` masks only the interrupt, and the only stop is `HPET_CONFIG.hpet_enable` (halt it before reprogramming a comparator). And `32mode` FORCES 32-bit, so 64-bit operation is the bit CLEAR – the inverse of the retired `timer_size`.

13.4.2.4 Rule #4: HPET Timer Modes

One-Shot: - Timer fires once when counter >= comparator - Does NOT automatically reload - Must reconfigure for next fire

Periodic: - Timer fires repeatedly - Comparator auto-increments by period value - Fires indefinitely until the counter is halted or the timer is switched to one-shot (there is no per-timer stop)

13.4.3 HPET Common Issues

Issue: Timer Not Firing 1. HPET enabled? (HPET_CONFIG bit 0 – the only run control) 2. Comparator set correctly? (LO at +0x08, HI at +0x0C) 3. Counter incrementing? 4. Counter will reach comparator? 5. Interrupt enable set? (TIMER_CONFIG bit 2) – without it the HPET_STATUS bit still sets but `timer_irq` stays low 6. Legacy replacement on? (HPET_CONFIG bit 1 moves timers 0/1 off `timer_irq`)

Issue: Tests Failing Inconsistently - Most common cause: Missing test cleanup (counter not reset) - Solution: Add cleanup at end of EVERY test

See: Complete HPET guidance in `docs/hpet_mas/hpet_mas_index.md`

13.5 Common User Questions

13.5.1 Q: “Which blocks are implemented?”

A: Current status:

Block	Priority	Status	Address	Documentation
HPET	High	Production	0xFEC0_0000 -0x0FFF	Complete
8259 PIC (master)	High	Implemented	0xFEC0_1000 -0x1FFF	MAS (docs/pic_8259_mas)
8254 PIT	High	Implemented	0xFEC0_2000 -0x2FFF	MAS (docs/pit_8254_mas)
RTC	Medium	Implemented	0xFEC0_3000 -0x3FFF	MAS (docs/rtc_mas)
SMBus	Medium	Implemented	0xFEC0_4000 -0x4FFF	MAS (docs/smbus_mas)
PM/ACPI	Medium	Implemented	0xFEC0_5000 -0x5FFF	MAS (docs/pm_acpi_mas)
IOAPIC	Medium	Implemented	0xFEC0_6000 -0x6FFF	MAS (docs/ioapic_mas)
GPIO	Medium	Implemented	0xFEC0_7000 -0x7FFF	MAS (docs/gpio_mas)
UART 16550	Medium	Implemented	0xFEC0_8000 -0x8FFF	MAS (docs/uart_16550_mas)
8259 PIC (cascade slave)	High	Implemented	0xFEC0_9000 -0x9FFF	MAS (docs/pic_8259_mas)
SPI	Low	Planned	unallocated	N/A
I2C	Low	Planned	unallocated	N/A
Watchdog	Low	Planned	unallocated	N/A
Interconnect	Low	Planned	unallocated – all ten windows are assigned	N/A

See: `PRD.md` Section 3 for planned block details and Section 4.2 for complete address map

13.5.2 Q: "How do I integrate a block in my design?"

A: Each block has APB interface, example:

```
apb4_hpet #(
    .NUM_TIMERS(3),
    .VENDOR_ID(16'h4747),   // hardware-driven into GCAP_ID[31:16]
(TASK-006)
    .REVISION_ID(8'h01),    // hardware-driven into GCAP_ID[7:0]
    .CDC_ENABLE(0)
) u_hpet (
    // APB interface
    .pclk          (apb_clk),
    .presetn       (apb_rst_n),
    .s_apb_PADDR    (paddr),
    .s_apb_PSEL     (psel_hpet),
    .s_apb_PENABLE (penable),
    .s_apb_PWRITE   (pwrite),
    .s_apb_PWDATA   (pwdata),
    .s_apb_PRDATA   (prdata_hpet),
    .s_apb_PREADY   (pready_hpet),
    .s_apb_PSLVERR (pslverr_hpet),
    // Block-specific signals
    .hpet_clk      (timer_clk),
    .hpet_resetn    (timer_rst_n),
    .timer_irq     (timer_irq[2:0]),
    // LegacyReplacement Route (TASK-003): asserted only while
    // HPET_CONFIG.legacy_replacement is set, and timers 0/1 are then
    // SUPPRESSED on timer_irq. Consuming these also means silencing
the
    // 8254 and the RTC, and timer 0 belongs on IRQ0 / IOAPIC pin 2 --
    // never PIC IRQ2, which is the cascade input.
    .legacy_irq0   (hpet_legacy_irq0),
    .legacy_irq8   (hpet_legacy_irq8)
);
```

See: `README.md` for integration examples

13.5.3 Q: "What's the RLB wrapper goal?"

A: Create unified subsystem combining all blocks with single APB entry point:

```
RLB Wrapper Architecture:

Single APB Slave → APB Decoder/Bridge → Individual Blocks
(0xFEC0_0000)    (4KB window decode)   (HPET, 8259, 8254, etc.)
```

Address Map (4KB windows): - `0xFEC0_0000-0x0FFF`: HPET - `0xFEC0_1000-0x1FFF`: 8259 PIC (master) - `0xFEC0_2000-0x2FFF`: 8254 PIT - `0xFEC0_3000-0x3FFF`: RTC - `0xFEC0_4000-0x4FFF`: SMBus - `0xFEC0_5000-0x5FFF`: PM/ACPI - `0xFEC0_6000-0x6FFF`: IOAPIC - `0xFEC0_7000-0x7FFF`: GPIO - `0xFEC0_8000-0x8FFF`: UART 16550 - `0xFEC0_9000-0x9FFF`: 8259 PIC (cascade slave) - All others → decode-miss agent, answered locally with PSLVERR

Benefits: - Single APB slave port (easy integration) - Drop-in retro-compatible peripheral subsystem - Clean power-of-2 decode (4KB = bits [15:12]) - 32KB reserved space for expansion

See: `PRD.md` Section 4.2 for complete RLB wrapper specification and decoder implementation

13.5.4 Q: "Why 'Retro Legacy Blocks'?"

A: - **Retro**: Implements proven architectures from older platforms - **Legacy**: Based on time-tested peripheral interface specifications - **Blocks**: Collection of independent peripherals

Not experimental - production-ready implementations of time-tested designs.

13.6 Quick Commands

```
# Always through the area Makefile -- a bare pytest drops the level, the derived
# worker count and the reruns, and skips clean-all.
cd projects/components/legacy-ip/retro_legacy_blocks/dv/tests
make list                           # the roots discovered by glob
make clean-all && make run-all-full-parallel
make run-apb4_hpet-gate             # one block, one level
make run-apb4_hpet-gate-waves       # same with waves (never --vcd)
```

Grammar and rationale: `vault/handbook/dv/running-regressions.md`.

13.7 PDF Generation Location

IMPORTANT: PDF files should be generated in the docs directory:

`$REPO_ROOT/projects/components/legacy-ip/retro_legacy_blocks/docs/`

Quick Command: Use the MAS script. The books live in `*_mas/` directories and `generate_mas_pdf.sh` is what knows about them:

```
cd $REPO_ROOT/projects/components/legacy-ip/retro_legacy_blocks/docs
./generate_mas_pdf.sh                       # every book
./generate_mas_pdf.sh --component smbus     # one book
```

`generate_pdf.sh` in the same directory is the OLDER script and targets `*_spec/` directories that no longer exist here. It used to answer a missing index with “generated successfully” while writing nothing; that is fixed, and it now fails loudly, but it still has nothing to build in this component.

Version: 2.0 **Last Updated:** 2025-10-29 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

14 APB HPET Register Map

Chapter: 5.1 **Title:** Complete Register Address Map **Version:** 1.1 **Last Updated:** 2026-10-13

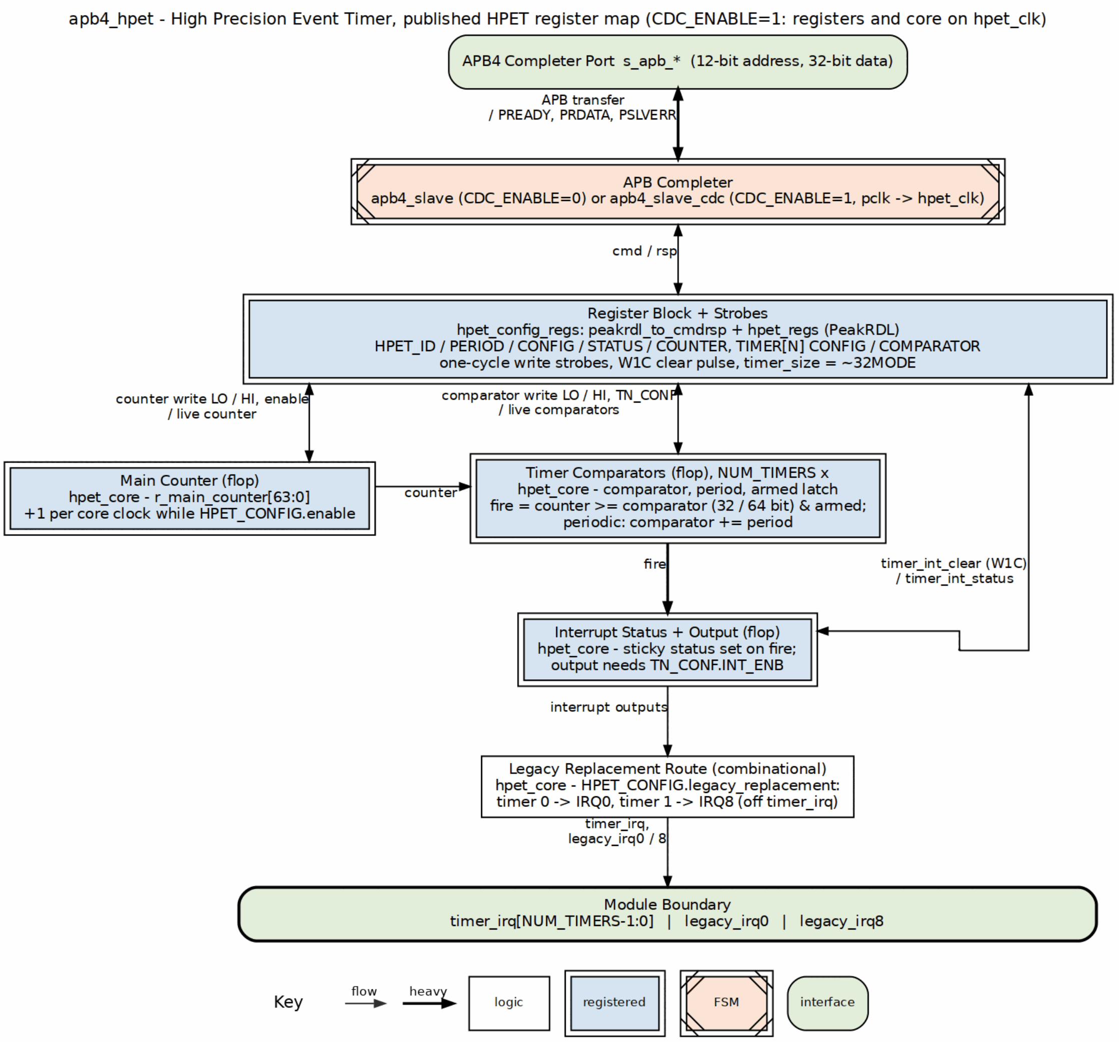
14.1 Overview

The APB HPET provides a memory-mapped register interface accessible via the APB slave port. The register space is organized into two main sections:

1. **Global Registers (0x000-0x0FF):** Configuration, status, and main counter
2. **Per-Timer Registers (0x100-0x1FF):** Timer-specific configuration and comparators

Each timer occupies a 32-byte (0x20) register block, supporting up to 8 timers.

14.1.1 Figure 5.1: APB HPET Block Diagram


apb4_hpet - High Precision Event Timer, published HPET register map (CDC_ENABLE=1: registers and core on hpet_clk)
APB4 Completer Port s_apb_* (12-bit address, 32-bit data)
APB transfer
/ PREADY, PRDATA, PSLVERR
APB Completer
apb4_slave (CDC_ENABLE=0) or apb4_slave_cdc (CDC_ENABLE=1, pclk -> hpet_clk)
cmd / rsp
Register Block + Strobes
hpet_config_regs: peakrdl_to_cmdrsp + hpet_regs (PeakRDL)
HPET_ID / PERIOD / CONFIG / STATUS / COUNTER, TIMER[N] CONFIG / COMPARATOR
one-cycle write strobes, W1C clear pulse, timer_size = ~32MODE
counter write LO / HI, enable
/ live counter
comparator write LO / HI, TN_CONF
/ live comparators
Main Counter (flop)
hpet_core - r_main_counter[63:0]
+1 per core clock while HPET_CONFIG.enable
counter
Timer Comparators (flop), NUM_TIMERS x
hpet_core - comparator, period, armed latch
fire = counter >= comparator (32 / 64 bit) & armed;
periodic: comparator += period
fire
timer_int_clear (W1C)
/ timer_int_status
Interrupt Status + Output (flop)
hpet_core - sticky status set on fire;
output needs TN_CONF.INT_ENB
interrupt outputs
Legacy Replacement Route (combinational)
hpet_core - HPET_CONFIG.legacy_replacement:
timer 0 -> IRQ0, timer 1 -> IRQ8 (off timer_irq)
timer_irq,
legacy_irq0 / 8
Module Boundary
timer_irq[NUM_TIMERS-1:0] | legacy_irq0 | legacy_irq8
Key
flow
heavy
logic
registered
FSM
interface

APB HPET Block Diagram

APB HPET top-level architecture showing APB interface, configuration registers, HPET core, and timer outputs.

14.2 Functional Description

14.2.1 Register Address Map Summary

14.2.1.1 Global Registers

Offset	Register Name	Access	Width	Description
0x000	HPET_ID	RO	32b	GCAP_ID[31:0] -

Offset	Register Name	Access	Width	Description
				identification and capabilities
0x004	HPET_PERIOD	RO	32b	GCAP_ID[63:32] - main counter period in femtoseconds
0x008-0x00F	RESERVED	RO	-	Reserved
0x010	HPET_CONFIG	RW	32b	GEN_CONF - global configuration and control
0x014-0x01F	RESERVED	RO	-	Reserved
0x020	HPET_STATUS	RW/W1C	32b	GINTR_STA - interrupt status (write-1-to-clear)
0x024-0x0EF	RESERVED	RO	-	Reserved
0x0F0	HPET_COUNTER_LO	RW	32b	MAIN_CNT bits [31:0]
0x0F4	HPET_COUNTER_HI	RW	32b	MAIN_CNT bits [63:32]
0x0F8-0x0FF	RESERVED	RO	-	Reserved

These are the PUBLISHED SPEC offsets (RLB/hpet TASK-006). Reserved space is not decoded: a read returns 0 and a write is dropped, and neither raises PSLVERR (the generated register block ties its read and write error responses to 0). See the address-aliasing note under the Memory Map Diagram below.

14.2.1.2 Per-Timer Registers

Each timer (N = 0 to NUM_TIMERS-1) has a 32-byte register block at base address `0x100 + N*0x20`.

Timer N Base Address: `0x100 + N * 0x20`

Offset	Register Name	Access	Width	Description
+0x00	TIMER_CONFIG	RW	32b	TN_CONF[31:0] - configuration and control
+0x04	TIMER_INT_ROUTE_CAP	RO	32b	TN_CONF[63:32] - legal I/O APIC inputs (reads 0: general per-timer routing is not implemented). INDEPENDENT of `leg_rt_cap` = 1 – legacy replacement OVERRIDES `timer_int_route` rather than selecting through this mask
+0x08	TIMER_COMPARATOR_LO	RW	32b	Timer comparator bits [31:0]
+0x0C	TIMER_COMPARATOR_HI	RW	32b	Timer comparator bits [63:32]
+0x10-0x1F	RESERVED	RO	-	The spec's FSB route registers live here; not implemented, and fsb_int_del_cap reads 0 to say so

Example Timer Addresses:

Timer	Base Address	CONFIG	COMPARATOR_LO	COMPARATOR_HI
0	0x100	0x100	0x108	0x10C
1	0x120	0x120	0x128	0x12C
2	0x140	0x140	0x148	0x14C

Timer	Base Address	CONFIG	COMPARATOR_LO	COMPARATOR_HI
3	0x160	0x160	0x168	0x16C
4	0x180	0x180	0x188	0x18C
5	0x1A0	0x1A0	0x1A8	0x1AC
6	0x1C0	0x1C0	0x1C8	0x1CC
7	0x1E0	0x1E0	0x1E8	0x1EC

14.2.2 Global Register Descriptions

14.2.2.1 HPET_ID (0x000) - Identification Register

Access: Read-Only **Reset Value:** `(VENDOR_ID[15:0] << 16) | 0x8000 | 0x2000 | ((NUM_TIMERS-1) << 8) | REVISION_ID[7:0]` – 0x8000 is `leg_rt_cap`, 0x2000 is `count_size_cap`

Contains capability information and identification fields. `vendor_id`, `rev_id` and `num_tim_cap` are all driven from the top-level parameters through the register block's hardware interface, so one generated block serves every instantiation. This is GCAP_ID[31:0] at the published spec field positions (RLB/hpet TASK-006); `vendor_id` is the spec's full 16 bits, so a PCI-style `VENDOR_ID(16'h4747)` now reads back complete rather than truncated to its low byte. GCAP_ID[63:32] is the counter clock period, at 0x004.

Bits	Field	Access	Reset	Description
[31:16]	vendor_id	RO	VENDOR_ID	Vendor identifier, full 16 bits per the spec
[15]	leg_rt_cap	RO	1	Legacy-replacement capability. Reads 1: the LegacyReplacement Route IS implemented (RLB/hpet TASK-003) – timer 0 leaves on `legacy_irq0` (IRQ0) and timer 1 on `legacy_irq8` (IRQ8), both suppressed on `timer_irq`. The POSITION matters – drivers gate on this bit, so a 0 here would mean legacy mode is never

Bits	Field	Access	Reset	Description
				used
[14]	reserved	RO	0	Reserved
[13]	count_size_cap	RO	1	Counter size capability (1 = 64-bit counter)
[12:8]	num_tim_cap	RO	NUM_TIMERS-1	Number of timers minus 1 (e.g., 7 for 8 timers)
[7:0]	rev_id	RO	REVISION_ID	Revision identifier

Example Values, computed for the parameters the DV grid actually passes (the previous table claimed the parameter DEFAULT of 1 while the suite has always driven 0x4747/0x1022/0xABCD, so those constants never matched a real readback):

Config	VENDOR_ID	REVISION_ID	HPET_ID reads
2 timers	0x4747	0x01	`0x4747A101`
3 timers	0x1022	0x02	`0x1022A202`
8 timers	0xABCD	0x10	`0xABCDA710`

Decoding the 8-timer value: vendor 0xABCD, leg_rt_cap 1 (bit 15), count_size_cap 1 (bit 13), num_tim_cap 7 (bits 12:8), rev_id 0x10.

These words each carry 0x8000 because `leg_rt_cap` reads 1 since RLB/hpet TASK-003; before the legacy routing existed they read `0x47472101`, `0x10222202` and `0xABCD2710`.

14.2.2.2 HPET_CONFIG (0x010) - Configuration Register

Access: Read-Write **Reset Value:** 0x00000000

Global enable and configuration control.

Bits	Field	Access	Reset	Description
[31:2]	reserved	RO	0	Reserved
[1]	legacy_replacement	RW	0	Enables the LegacyReplacement Route: timer 0 is delivered as IRQ0 and timer 1 as IRQ8, and BOTH stop being delivered on `timer_irq`.

Bits	Field	Access	Reset	Description
				Timers 2+ are unaffected. `HPET_ID.leg_rt_cap` reads 1 to advertise it
[0]	hpet_enable	RW	0	HPET main counter enable (0=stopped, 1=running)

Usage Notes: - Write `hpet_enable=1` to start the main counter - Write `hpet_enable=0` to stop the main counter (value preserved) - `legacy_replacement` REPLACES the delivery of timers 0 and 1 rather than adding to it: setting it moves timer 0 to `legacy_irq0` and timer 1 to `legacy_irq8` and holds both off `timer_irq`. `HPET_ID.leg_rt_cap` reads 1 - The sticky `HPET_STATUS` bit is deliberately NOT affected by legacy mode: only the delivery path moves, so software that polls status instead of taking the interrupt behaves identically either way - Consuming the legacy routes also means silencing the 8254 PIT tick and the RTC periodic interrupt. That is an integration duty at `rlb_top`, not this block's, and note timer 0 belongs on master IRQ0 and I/O APIC pin 2 – never PIC IRQ2, which is the 8259 cascade input - Counter must be enabled for any timer to fire

Example Configuration Sequence:

```
// Disable HPET
WRITE(HPET_CONFIG, 0x0);

// Reset counter
WRITE(HPET_COUNTER_LO, 0x0);
WRITE(HPET_COUNTER_HI, 0x0);

// Configure timers...

// Enable HPET
WRITE(HPET_CONFIG, 0x1);
```

14.2.2.3 HPET_STATUS (0x020) - Interrupt Status Register

Access: Read-Write (Write-1-to-Clear) **Reset Value:** 0x00000000

Interrupt status bits for all timers. The register is a mirror of the sticky status held in `hpet_core`: hardware re-drives it from the core's level every cycle, and a software write is forwarded to the core as a per-bit clear. Write 1 to a bit to clear the corresponding interrupt.

Bits	Field	Access	Reset	Description
[31:8]	reserved	RO	0	Hard zero

Bits	Field	Access	Reset	Description
[7:NUM_TIMERS]	(unused timer bits)	RO	0	The field is a fixed 8 bits wide; bits with no timer behind them read 0 and cannot be set
[NUM_TIMERS-1:0]	timer_int_status	RW/W1C	0	Timer interrupt status bits, mirrored from `hpet_core`

Per-Timer Status Bit: - **Bit[N]** = Timer N interrupt status - 0 = No interrupt pending - 1 = Timer N has fired, interrupt pending

Write-1-to-Clear (W1C) Behavior: - Write 1 to bit[N] to clear Timer N interrupt status (and its irq) - Write 0 to a bit leaves it alone; a write of 0x00000000 is a no-op - Clearing one timer's bit does not touch the other timers' bits or irq outputs - A timer that fires in the same cycle its own bit is being cleared keeps the bit set: the new event wins, and the clear can simply be repeated - Reading returns the current interrupt status

Example Interrupt Handling:

```
// Read interrupt status
uint32_t status = READ(HPET_STATUS);

// Check if Timer 0 fired
if (status & 0x1) {
    // Handle Timer 0 interrupt

    // Clear Timer 0 interrupt
    WRITE(HPET_STATUS, 0x1);  // Write 1 to clear bit 0 only
}
```

14.2.2.4 *HPET_COUNTER_LO (0x0F0) - Main Counter Low*

Access: Read-Write **Reset Value:** 0x00000000

Lower 32 bits of the 64-bit free-running main counter.

Bits	Field	Access	Reset	Description
[31:0]	counter_lo	RW	0	Main counter bits [31:0]

Behavior: - **Read:** Returns current counter value [31:0] - **Write:** Loads counter bits [31:0] on the write itself, byte enables respected; bits [63:32] are untouched (see the 64-bit write ordering note below) - Counter increments every `hpet_clk` cycle when `HPET_CONFIG.hpet_enable=1` - Software can write to reset or set counter to specific value

Usage Notes: - Writing counter is useful for test/debug or implementing periodic reset - Write the counter with it halted (`HPET_CONFIG.hpet_enable=0`), as on a real HPET: the two halves load independently, so a running timer could see the intermediate {old HI, new LO} value. The halt also bounds what a periodic timer has to catch up: a catch-up advances the comparator once per cycle and closes period - 1 counts each time, so a timer left far behind – a torn or stale half on a running counter is exactly how a 2^52 deficit appears – is silent for cycles in proportion to the deficit. A contract, not a defect; see Periodic Mode - Partial (byte-strobed, `PSTRB != 0xF`) counter writes REQUIRE a halted counter, which is the stricter rule: the register block merges the bytes software did not write from its own mirror of the field, and on a running counter that mirror is a cycle old, so the preserved bytes come back behind the counter they were meant to keep. Halted, the mirror is stable and the merge exact. Full-word writes never read the mirror and are unaffected - Counter write takes effect immediately (on next `hpet_clk`) - All timers compare against this counter value

14.2.2.5 HPET_COUNTER_HI (0x0F4) - Main Counter High

Access: Read-Write **Reset Value:** 0x00000000

Upper 32 bits of the 64-bit free-running main counter.

Bits	Field	Access	Reset	Description
[31:0]	counter_hi	RW	0	Main counter bits [63:32]

Behavior: - Same as HPET_COUNTER_LO but for upper 32 bits - Forms complete 64-bit counter value: `{counter_hi, counter_lo}`

Reading 64-bit Counter:

```
// LO-then-HI gives NO rollover protection (a carry between the reads
// yields old-LO with new-HI). Use the HI/LO/HI retry loop from
// ch01_overview/03_clocks_and_reset.md:
uint32_t hi1, lo, hi2;
do {
    hi1 = READ(HPET_COUNTER_HI);
    lo  = READ(HPET_COUNTER_LO);
    hi2 = READ(HPET_COUNTER_HI);
} while (hi1 != hi2);
uint64_t counter = ((uint64_t)hi1 << 32) | lo;
```

Writing 64-bit Counter:

```
// Halt the counter first; each write loads only its own half
WRITE(HPET_CONFIG, 0x0);
WRITE(HPET_COUNTER_LO, 0x00000000);
WRITE(HPET_COUNTER_HI, 0x00000000);
```

14.2.3 Per-Timer Register Descriptions

Each timer has a dedicated 32-byte register block. The following descriptions apply to Timer N at base address `0x100 + N*0x20`.

14.2.3.1 TIMER_CONFIG (Timer Base + 0x00) - Timer Configuration

Access: Read-Write **Reset Value:** 0x00000030 (the read-only `size_cap` and `per_int_cap` read 1; every writable bit resets to 0)

Configuration and control for individual timer.

Bits	Field	Access	Reset	Description
[31:16]	reserved	RO	0	Reserved
[15]	fsb_int_del_cap	RO	0	FSB (message) delivery capability. Reads 0, so the spec’s FSB route registers at +0x10/+0x14 are not implemented
[14]	timer_fsb_en	RW	0	FSB delivery enable. Storage only – see fsb_int_del_cap
[13:9]	timer_int_route	RW	0	I/O APIC input to drive, as a bit NUMBER chosen from TIMER_INT_ROUTE_CAP. STORAGE ONLY: general route selection is not implemented, and the spec has legacy replacement OVERRIDE this field for timers 0/1 rather than select through it
[8]	timer_32mode	RW	0	FORCE 32-bit operation. **Note the polarity:** 1 = 32-bit, so 64-bit operation is this bit

Bits	Field	Access	Reset	Description
				CLEAR. The retired `timer_size` meant the opposite
[7]	reserved	RO	0	Reserved
[6]	timer_value_set	RW	0	Stores and reads back, but has NO hardware effect: this core always loads both the accumulator and the period, so the bit has nothing to select
[5]	size_cap	RO	1	READ-ONLY capability: this timer is 64-bit capable
[4]	per_int_cap	RO	1	READ-ONLY capability: this timer supports periodic mode
[3]	timer_type	RW	0	Timer mode (0=one-shot, 1=periodic)
[2]	timer_int_enable	RW	0	Interrupt enable. The ONLY per-timer enable the spec defines
[1]	timer_int_type	RW	0	0=edge, 1=level. Storage only: this core always delivers the sticky level-plus-W1C behaviour HPET_STATUS implements
[0]	reserved	RO	0	Reserved

These are the published spec's TN_CONF positions (RLB/hpet TASK-006). **There is no per-timer RUN enable** – the spec defines none, so a timer's comparator is live whenever the main counter runs and `timer_int_enable` gates only the interrupt.

Field Descriptions:

timer_int_enable (bit 2): - 0 = Interrupt delivery disabled: the timer still fires and still sets its `HPET_STATUS` bit, but `timer_irq` (or the legacy route) stays low - 1 = Interrupt delivery enabled: a fire sets the `HPET_STATUS` bit AND raises the interrupt - `hpet_core` sets the sticky status bit on EVERY fire, whatever this bit says; it gates only the interrupt output, sampled at fire time

timer_type (bit 3): - 0 = **One-shot mode:** Timer fires once when counter >= comparator, then stays idle - 1 = **Periodic mode:** Timer fires repeatedly, auto-increments comparator by period

timer_32mode (bit 8): - 1 = FORCE 32-bit (uses only COMPARATOR_LO, ignores COMPARATOR_HI) - 0 = 64-bit (uses the full 64-bit comparator) – the DEFAULT - The polarity is the spec's and is the inverse of the retired `timer_size` - APB HPET supports 64-bit by default - Change it only with the counter halted, and rewrite the comparator afterwards. The change clears the next-epoch hold, but a comparator advanced at the old width is not on the new width's lattice, and a live 0->1 switch can leave the timer catching up for ~2^32 cycles

timer_value_set (bit 6): - No hardware effect: the register bit stores and reads back, but nothing consumes it (`hpet_core` has no such input; the wire dead-ends at the top level). Documented for completeness only.

Common Configurations:

```
// One-shot timer with interrupt (64-bit compare: 32MODE clear)
WRITE(TIMER0_CONFIG, 0x004);  // bit 2 = INT_ENB

// Periodic timer with interrupt
WRITE(TIMER0_CONFIG, 0x00C);  // bits 3, 2 = TYPE (periodic) | INT_ENB

// One-shot timer, forced 32-bit compare, with interrupt
WRITE(TIMER0_CONFIG, 0x104);  // bits 8, 2 = 32MODE | INT_ENB
```

14.2.3.2 TIMER_COMPARATOR_LO (Timer Base + 0x08) - Comparator Low

Access: Read-Write **Reset Value:** 0x00000000

Lower 32 bits of the 64-bit timer comparator value.

Bits	Field	Access	Reset	Description
[31:0]	timer_comp_lo	RW	0	Timer comparator bits [31:0]

Behavior: - Timer fires when `main_counter >= comparator` - For **one-shot mode:** Comparator value stays unchanged after fire - For **periodic mode:** the CORE's working comparator auto-increments by the period on each fire, and a read returns that live value (RLB/hpet TASK-002): the field is `hw=rw` with `precedence = sw` (the same idiom the main counter uses), so hardware continuously publishes its working comparator into the register while a software write still wins in its own cycle - Each half loads the core on its own write; writing the value the register already holds still counts (the internal comparator reloads, discarding any periodic advance) - The write re-arms the timer only while the COUNTER IS HALTED (`HPET_CONFIG.hpet_enable=0`). There is no per-timer stop: the published spec defines no per-timer run enable. On a running counter the half loads but the timer re-arms only when the completed value moves the match low – see Reprogramming a Running Timer below - Software writes to set initial comparator value

Usage:

```
// Set Timer 0 to fire at 1000 cycles (assuming HPET_clk = counter increment)
WRITE(TIMER0_COMPARATOR_LO, 1000);
WRITE(TIMER0_COMPARATOR_HI, 0);
```

14.2.3.3 TIMER_COMPARATOR_HI (Timer Base + 0x0C) - Comparator High

Access: Read-Write **Reset Value:** 0x00000000

Upper 32 bits of the 64-bit timer comparator value.

Bits	Field	Access	Reset	Description
[31:0]	timer_comp_hi	RW	0	Timer comparator bits [63:32]

Behavior: - Forms complete 64-bit comparator: `{timer_comp_hi, timer_comp_lo}` - Same behavior as COMPARATOR_LO but for upper 32 bits

64-bit Timer Example:

```
// Set Timer 1 to fire at 0x0000_0001_0000_0000 (4.3 billion cycles)
WRITE(TIMER1_COMPARATOR_LO, 0x00000000);
WRITE(TIMER1_COMPARATOR_HI, 0x00000001);
```

14.2.3.4 Reprogramming a Running Timer

A comparator write re-arms the timer only while the COUNTER IS HALTED (`HPET_CONFIG.hpet_enable=0` – there is no per-timer run enable in the spec), and then it always re-arms, whatever the value: a comparator at or below the counter fires as soon as the counter restarts (the >= match, the same rule that makes a deficit fire rather than be missed – there is no wait for a wrap). That is how software arms to an already-passed target: a zero comparator, or a periodic phase restarted after a counter reset.

On a running timer the write still loads the half it names, but does not re-arm; the timer re-arms only when the completed value moves the match low. Written above the counter, the match falls, the timer re-arms and fires at the new value, so the usual “next deadline = now + interval” one-shot restart works with the timer running.

The reason is the 64-bit comparator arriving as two 32-bit halves. Between the two writes the core holds the torn value `{old HI, new LO}`, which software never programmed; withholding the re-arm on a running timer is what guarantees that value can never fire.

The contract: to reprogram a running timer's comparator, halt the counter first (`HPET_CONFIG.hpet_enable=0` – the only stop the published map has), write both halves, then restart it. It fires once, at the programmed value – even one the counter has already passed. Clearing `timer_int_enable` does NOT stop a timer; it only masks the interrupt. The real HPET has the same requirement. Two consequences of reprogramming a running timer anyway are outside the contract, not defects:

- A torn 64-bit value that lands above the counter re-arms the timer through the natural path, and the timer then fires at the final value once the second half is written. The completed value fires it, not the tear.
- A same-value rewrite on a running periodic timer drags the comparator back from its auto-advanced position to the originally programmed value, and can add one off-lattice fire.

```
// Move a running 64-bit timer to a new deadline, any value
WRITE(HPET_CONFIG, 0x0);                    // halt the counter (block-wide stop)
WRITE(TIMER1_COMPARATOR_LO, new_lo);
WRITE(TIMER1_COMPARATOR_HI, new_hi);
WRITE(HPET_CONFIG, 0x1);                    // restart; TIMER1_CONFIG untouched
```

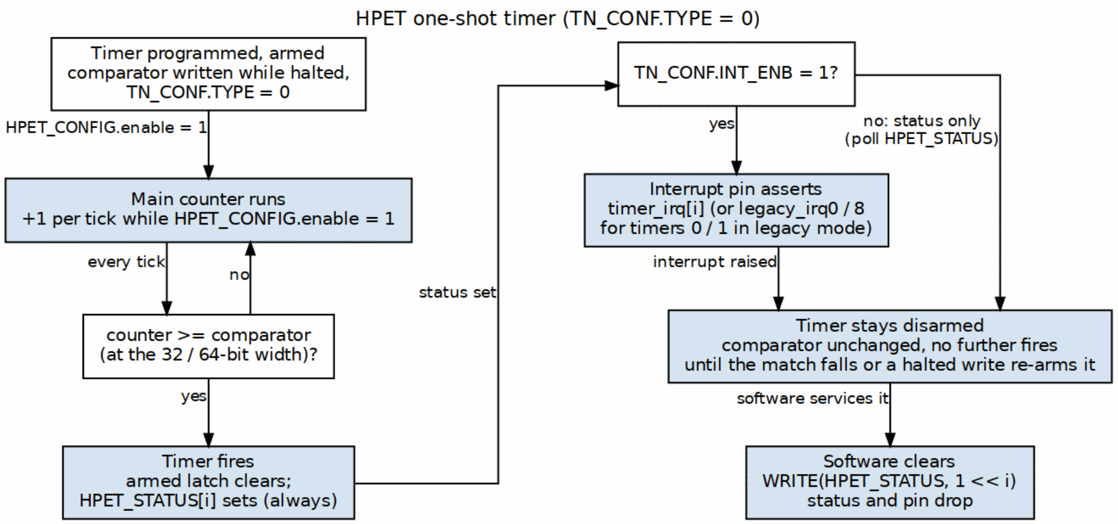
A software write landing in the same cycle as a fire on a running timer yields exactly one fire; the software value wins, and the periodic advance is skipped that cycle.

The same rule covers `timer_32mode`: change it only while the counter is halted, and rewrite the comparator afterwards. The next-epoch hold clears on the change, but a comparator that was advanced at the old width is not on the new width's lattice, and a live 0->1 switch can leave the timer catching up for ~2^32 cycles before it fires again.

14.2.4 Timer Operation Modes

14.2.4.1 One-Shot Mode (timer_type = 0)

14.2.5 Figure 5.2: One-Shot Timer Operation


HPET one-shot timer (TN_CONF.TYPE = 0)
Timer programmed, armed comparator written while halted, TN_CONF.TYPE = 0
HPET_CONFIG.enable = 1
Main counter runs +1 per tick while HPET_CONFIG.enable = 1
every tick
no
counter >= comparator (at the 32 / 64-bit width)?
yes
Timer fires armed latch clears; HPET_STATUS[i] sets (always)
status set
TN_CONF.INT_ENB = 1?
yes
no: status only (poll HPET_STATUS)
Interrupt pin asserts timer_irq[i] (or legacy_irq0 / 8 for timers 0 / 1 in legacy mode)
interrupt raised
Timer stays disarmed comparator unchanged, no further fires until the match falls or a halted write re-arms it
software services it
Software clears WRITE(HPET_STATUS, 1 << i) status and pin drop

One-Shot Timer Operation

One-shot timer operation flow showing counter increment, comparator match, and idle state after fire.

Behavior: 1. Counter increments: 0 → 1 → 2 → … → comparator 2. When `counter >= comparator` and the timer is armed: Timer fires, and the armed latch clears 3. Sets the HPET_STATUS bit; if `timer_int_enable`=1, also raises `timer_irq` 4. Timer stays complete until the comparator is rewritten with the counter halted (HPET_CONFIG.hpet_enable = 0), or written above the counter while it runs – halting and restarting the counter alone does not re-fire

Comparator Behavior: - Stays unchanged after fire - Software re-arms the timer by writing the comparator (either half, any value – even the same one) with the counter halted, or by writing a value above the counter while it runs (there is no per-timer stop)

Use Cases: - Single timeout events - Software-initiated timing - Watchdog timers (with software reload)

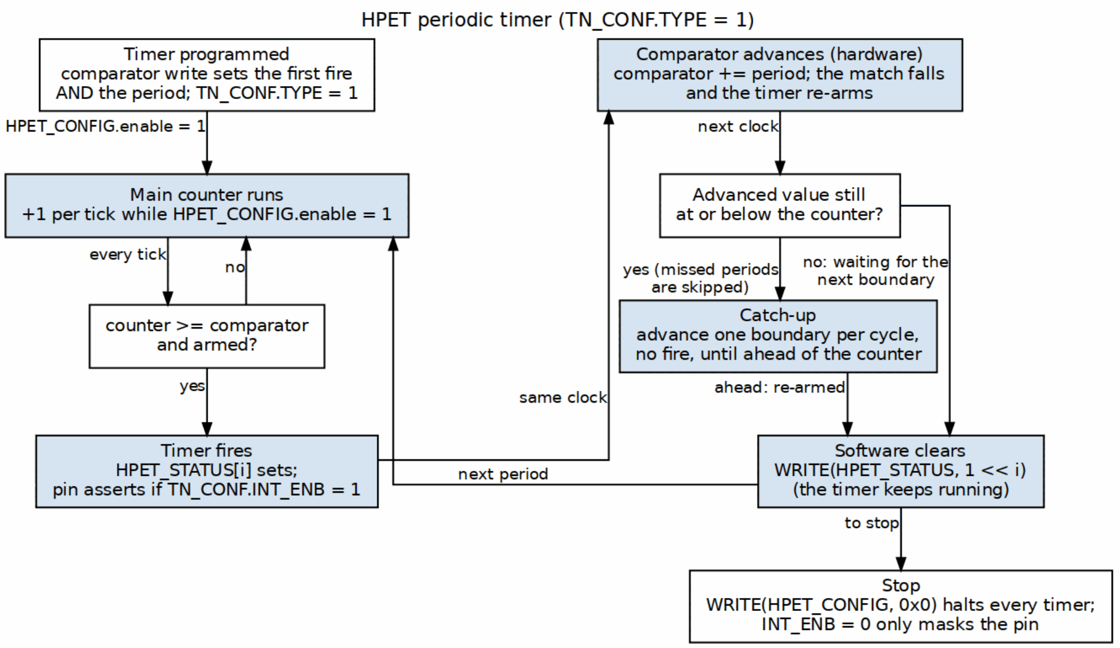
Example:

```
// Configure Timer 0: One-shot, 1000 cycles
WRITE(TIMER0_COMPARATOR_LO, 1000);
WRITE(TIMER0_CONFIG, 0x04);  // INT_ENB; TYPE = 0 (one-shot)

// Enable HPET
WRITE(HPET_CONFIG, 0x1);
```

```
// Wait for interrupt
while (!(READ(HPET_STATUS) & 0x1));

// Clear interrupt
WRITE(HPET_STATUS, 0x1);

// Re-arm for the next fire at 2000 cycles. Halt the counter around
the
// write: a comparator write re-arms only while
HPET_CONFIG.hpet_enable = 0 (the
// published map has no per-timer stop), so this works for any value,
even
// one the counter has already passed (which then fires on the first
enabled
// cycle). Writing 2000 with the counter running would also work here
-- but
// only because 2000 is above the counter, so the match falls and the
timer
// re-arms naturally.
WRITE(HPET_CONFIG, 0x0);
WRITE(TIMER0_COMPARATOR_LO, 2000);
WRITE(HPET_CONFIG, 0x1);
```

14.2.5.1 Periodic Mode (timer_type = 1)

14.2.6 Figure 5.3: Periodic Timer Operation


HPET periodic timer (TN_CONF.TYPE = 1)
Timer programmed
comparator write sets the first fire AND the period; TN_CONF.TYPE = 1
HPET_CONFIG.enable = 1
Main counter runs
+1 per tick while HPET_CONFIG.enable = 1
every tick
no
counter >= comparator and armed?
yes
Timer fires
HPET_STATUS[i] sets;
pin asserts if TN_CONF.INT_ENB = 1
same clock
next period
Comparator advances (hardware)
comparator += period; the match falls and the timer re-arms
next clock
Advanced value still at or below the counter?
yes (missed periods are skipped)
no: waiting for the next boundary
Catch-up
advance one boundary per cycle, no fire, until ahead of the counter
ahead: re-armed
Software clears
WRITE(HPET_STATUS, 1 << i)
(the timer keeps running)
to stop
Stop
WRITE(HPET_CONFIG, 0x0) halts every timer;
INT_ENB = 0 only masks the pin

Periodic Timer Operation

Periodic timer operation flow showing counter increment, comparator match, auto-increment, and continuous firing.

Behavior: 1. Counter increments: 0 → 1 → 2 → … → comparator 2. When `counter >= comparator` and the timer is armed: Timer fires 3. Sets the `HPET_STATUS` bit; if `timer_int_enable=1`, also raises `timer_irq` 4. **Comparator auto-increments:** `comparator = comparator + period` (the match drops, which re-arms the timer for the next period) 5. **Catch-up:** if the advanced comparator is still at or below the counter (timer programmed late, or the counter written forward), it keeps advancing one boundary per cycle WITHOUT firing until it is at or ahead of the counter, then re-arms. Missed periods are skipped, never burst: one interrupt for the whole batch, at the next boundary still in the future. The catch-up closes the deficit at period - 1 counts per cycle, one comparator advance per cycle (period 1 is the special lattice jump to counter + 1, closed in one), so its length is proportional to the deficit: a period-2 timer left 2^52 counts behind by a counter write catches up for ~2^52 cycles, the comparator rewritten every cycle and no interrupt raised. That bound is why the HPET specification and this book require the main counter halted (`HPET_CONFIG.hpet_enable=0`) before it is written, and why a periodic comparator is programmed near or ahead of the counter 6. **Epoch hold:** if an advance carries out of the compare width (past bit 31 for a 32-bit timer, bit 63 for a 64-bit one), the comparator keeps the wrapped low bits and the timer is held – no fire, no catch-up – until the counter wraps at that width, or software writes the comparator, the counter (which re-bases the epoch, so a comparator now behind the counter fires promptly) or `timer_32mode`. The write clears are evaluated at the compare width: a 32-bit timer is released only by a write to HPET_COUNTER_LO or TIMERn_COMPARATOR_LO, and a write to either HI half leaves it held; a 64-bit timer is released by either half 7. Timer repeats indefinitely (fires at 1×period, 2×period, 3×period, …)

Comparator Auto-Increment: - Hardware automatically adds period value to comparator - Period = initial comparator value written by software - Example: Initial comparator = 1000 → Fires at 1000, 2000, 3000, … - Period 0 behaves as one-shot: it can never get ahead of the counter, so the timer fires once and stays quiescent (no churn) - Period 1 gains nothing on the counter per cycle (both step by 1), so a catch-up step sets the comparator to counter + 1 – every integer is on the period-1 lattice – and re-arms at once. It delivers on every other tick, the fastest the one-cycle fire/re-arm loop allows, at any deficit; a live counter write that opens a gap recovers within a cycle - The advance is evaluated at the compare width. A 32-bit periodic timer whose comparator + period passes 2^32 fires at that boundary and then stays quiet until the low 32 bits of the counter wrap – up to about 2^32 ticks, though never longer than the period that caused the carry, so the next fire still lands on the programmed lattice. A 64-bit timer whose advance carries out of bit 63 waits for the 64-bit counter to wrap – in practice it fires once and never again until reprogrammed (chapter 2 puts a number on the wait). Both are the arithmetic of the register width, not defects; in 32-bit mode the advance never disturbs TIMER_COMPARATOR_HI's half of the comparator ([63:32])

Use Cases: - Periodic interrupts (e.g., 1 kHz tick) - PWM generation - Periodic data sampling - Heartbeat signals

Example:

```
// Configure Timer 1: Periodic, 2000 cycle period
WRITE(TIMER1_COMPARATOR_LO, 2000);  // Initial comparator = period
WRITE(TIMER1_CONFIG, 0x0C);  // TYPE (periodic) | INT_ENB

// Enable HPET
WRITE(HPET_CONFIG, 0x1);

// Timer fires at:
// - 2000 cycles (counter >= 2000)
// - 4000 cycles (counter >= 4000) [comparator auto-incremented to 4000]
// - 6000 cycles (counter >= 6000) [comparator auto-incremented to 6000]
// - ... indefinitely

// Interrupt handler
void timer1_isr(void) {
    // Clear interrupt
    WRITE(HPET_STATUS, 0x2);  // Clear bit 1 (Timer 1)

    // Handle periodic event
    // ...

    // No need to reconfigure - timer continues automatically
}
```

14.2.7 Register Access Conventions

14.2.7.1 Access Types

Type	Description	Behavior
RO	Read-Only	Software can read, writes ignored
RW	Read-Write	Software can read and write
W1C	Write-1-to-Clear	Write 1 to clear bit, write 0 has no effect
RW/W1C	Read-Write with W1C	Readable, writable, with W1C clear behavior

14.2.7.2 Reset Values

- **Global registers:** Reset to 0x00000000 (except HPET_ID)

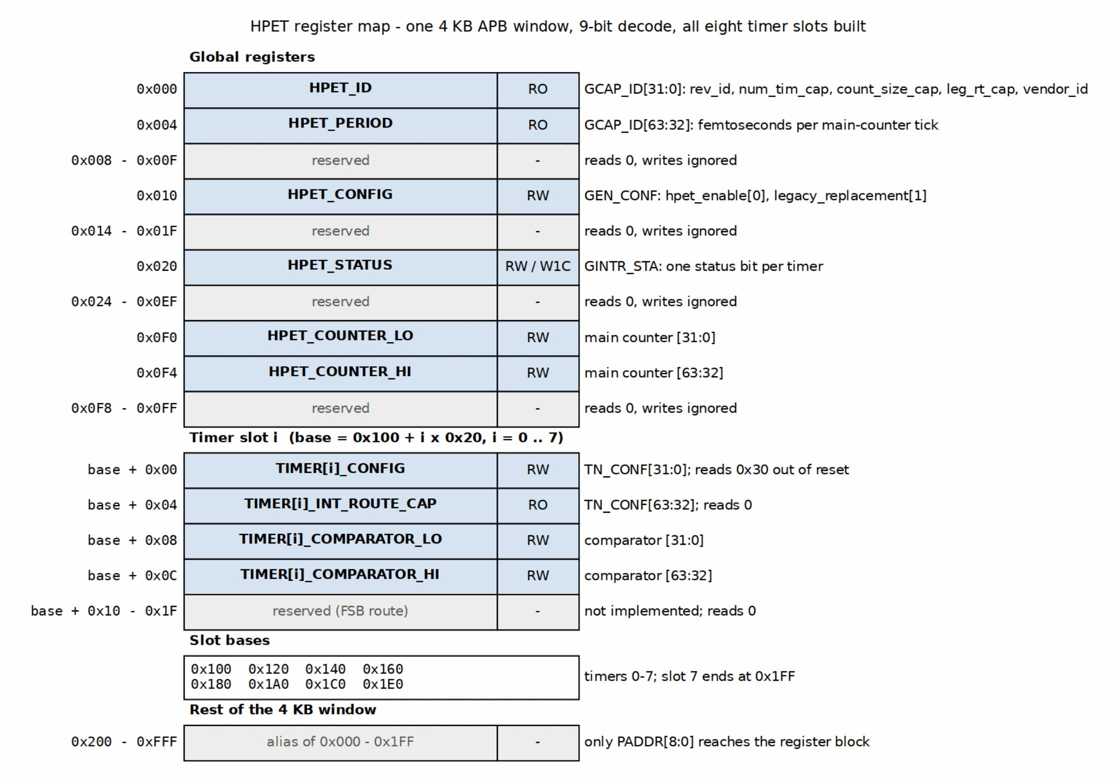
- **HPET_ID:** Constant GCAP_ID[31:0]: vendor_id = VENDOR_ID (full 16 bits at [31:16]), rev_id = REVISION_ID ([7:0]), num_tim_cap = NUM_TIMERS-1 ([12:8]), count_size_cap = 1 ([13]), leg_rt_cap = 1 ([15])
- **HPET_PERIOD:** Constant GCAP_ID[63:32]: COUNTER_CLK_PERIOD_FS femtoseconds
- **All timers:** TIMER_CONFIG resets to 0x00000030 (every writable bit 0; the read-only `per_int_cap` and `size_cap` read 1); comparators reset to 0
- **Main counter:** Reset to 0x00000000_00000000

14.2.7.3 Read/Write Ordering

64-bit Register Writes: Each 32-bit write lands its own half on the write itself, so the order is free, but the value is not atomic: between the two writes the hardware holds {old HI, new LO} (or the reverse). Halt the counter (`HPET_CONFIG[0] = 0`) before writing it, as on a real HPET – and always for a partial, byte-strobed write, whose merge reads a cycle-old mirror. The mixed comparator value can never fire, because a comparator write does not re-arm a running timer; the flip side is that a running timer re-arms only when the completed value is above the counter, so halt the counter around the pair whenever the new value may already be behind it (see Reprogramming a Running Timer).

64-bit Register Reads: Use the HI/LO/HI retry sequence (read HI, read LO, re-read HI; retry if the two HI reads differ). A plain LO-then-HI read has no rollover protection.

14.2.8 Figure 5.4: HPET Register Memory Map



HPET register map - one 4 KB APB window, 9-bit decode, all eight timer slots built

	Global registers		
`0x000`	**HPET_ID**	RO	GCAP_ID[31:0]: rev_id, num_tim_cap, count_size_cap, leg_rt_cap, vendor_id
`0x004`	**HPET_PERIOD**	RO	GCAP_ID[63:32]: femtoseconds per main-counter tick
`0x008 - 0x00F`	reserved	-	reads 0, writes ignored
`0x010`	**HPET_CONFIG**	RW	GEN_CONF: hpet_enable[0], legacy_replacement[1]
`0x014 - 0x01F`	reserved	-	reads 0, writes ignored
`0x020`	**HPET_STATUS**	RW / W1C	GINTR_STA: one status bit per timer
`0x024 - 0x0EF`	reserved	-	reads 0, writes ignored
`0x0F0`	**HPET_COUNTER_LO**	RW	main counter [31:0]
`0x0F4`	**HPET_COUNTER_HI**	RW	main counter [63:32]
`0x0F8 - 0x0FF`	reserved	-	reads 0, writes ignored
	Timer slot i (base = 0x100 + i x 0x20, i = 0 .. 7)		
`base + 0x00`	**TIMER[i]_CONFIG**	RW	TN_CONF[31:0]; reads 0x30 out of reset
`base + 0x04`	**TIMER[i]_INT_ROUTE_CAP**	RO	TN_CONF[63:32]; reads 0
`base + 0x08`	**TIMER[i]_COMPARATOR_LO**	RW	comparator [31:0]
`base + 0x0C`	**TIMER[i]_COMPARATOR_HI**	RW	comparator [63:32]
`base + 0x10 - 0x1F`	reserved (FSB route)	-	not implemented; reads 0
	Slot bases		
	`0x100  0x120  0x140  0x160` `0x180  0x1A0  0x1C0  0x1E0`		timers 0-7; slot 7 ends at 0x1FF
	Rest of the 4 KB window		
`0x200 - 0xFFF`	alias of 0x000 - 0x1FF	-	only PADDR[8:0] reaches the register block

HPET Register Memory Map

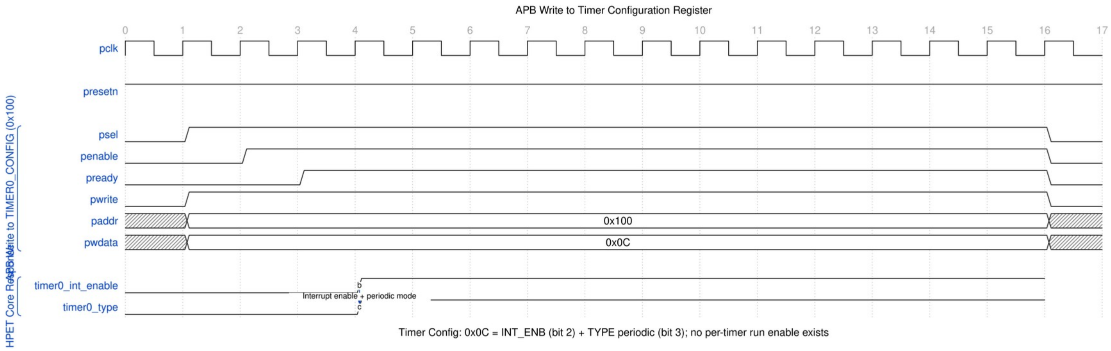
Only address bits [8:0] reach the register block, so 0x200-0xFFF alias back onto 0x000-0x1FF (0x200 reads HPET_ID, and so on). No error is raised for any address: reserved offsets inside the window read 0 and ignore writes.

Timer slots at or above NUM_TIMERS (e.g. 0x140-0x1FF on a 2-timer build) are REAL decoded storage: they read back written values but reach no core timer. Probe HPET_ID.num_tim_cap, not register writability, to discover the timer count.

14.3 Waveforms

The following timing diagrams illustrate key register access sequences:

14.3.1 Waveform 5.1: APB Write Timer Config

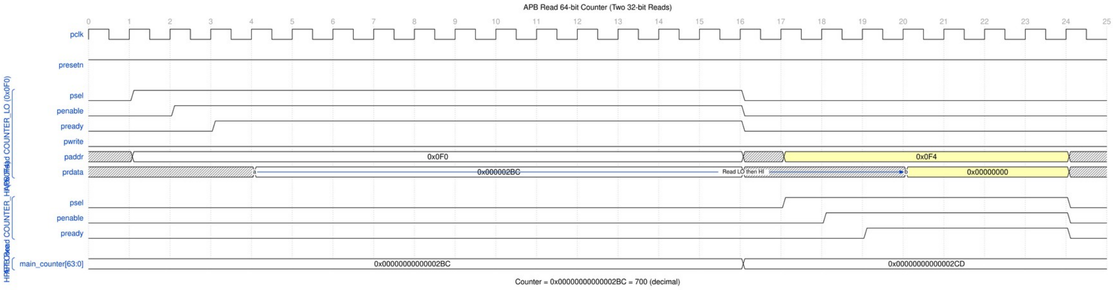

APB Write to Timer Configuration Register
pclk
presetn
psel
penable
pready
pwrite
paddr
pwdata
0x100
0x0C
APB Write to TIMER0_CONFIG (0x100)
timer0_int_enable
timer0_type
Interrupt enable + periodic mode
HPET Core Result
a
b

Timer Config: 0x0C = INT_ENB (bit 2) + TYPE periodic (bit 3); no per-timer run enable exists

APB Write Timer Config

APB write to TIMER0_CONFIG register (0x100). Source: assets/wavedrom/apb_write_timer_config.json

14.3.2 Waveform 5.2: APB Read Counter

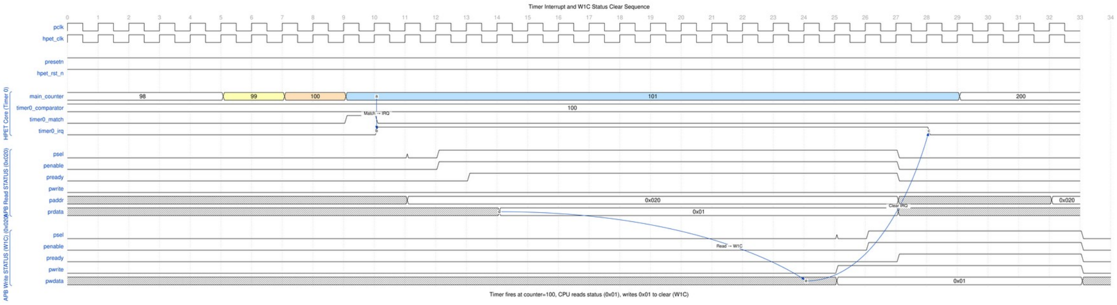

APB Read 64-bit Counter (Two 32-bit Reads)
pclk
presetn
psel
penable
pready
pwrite
paddr
prdata
0x0F0
0x000002BC
Read LO then HI
0x0F4
0x00000000
APB Read COUNTER_LO (0x0F0)
psel
penable
pready
APB Read COUNTER_HI
main_counter[63:0]
0x00000000000002BC
0x00000000000002CD
HPET Core

Counter = 0x00000000000002BC = 700 (decimal)

APB Read Counter

APB read of 64-bit counter (two 32-bit reads from COUNTER_LO and COUNTER_HI). Source: assets/wavedrom/apb_read_counter.json

14.3.3 Waveform 5.3: Interrupt W1C Sequence

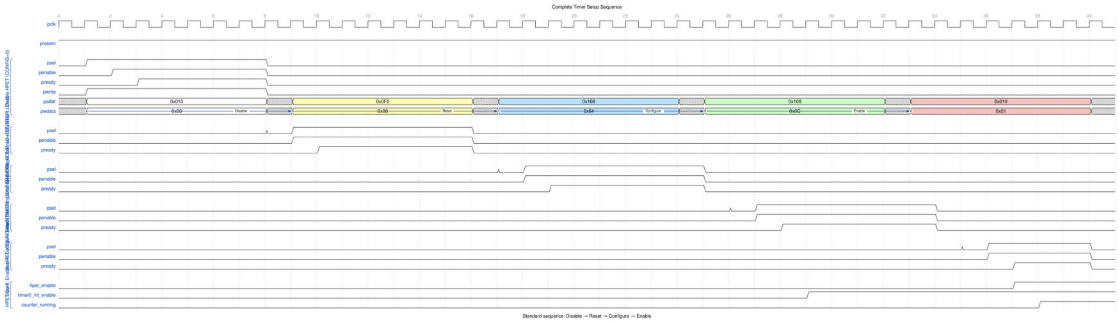

Timer Interrupt and W1C Status Clear Sequence
pclk
hpet_clk
presetn
hpet_rst_n
main_counter
98
99
100
101
200
timer0_comparator
100
timer0_match
timer0_irq
Match → IRQ
HPET Core (Timer 0)
psel
penable
pready
pwrite
paddr
prdata
0x020
0x01
Clear IRQ
APB Read STATUS (0x020)
psel
penable
pready
pwrite
pwdata
Read → W1C
0x01
APB Write STATUS (W1C) (0x020)

Timer fires at counter=100, CPU reads status (0x01), writes 0x01 to clear (W1C)

Interrupt W1C Sequence

Timer interrupt generation and W1C (Write-1-to-Clear) status clearing sequence. Source: assets/wavedrom/interrupt_w1c_sequence.json

14.3.4 Waveform 5.4: Timer Setup Sequence


Complete Timer Setup Sequence
0
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
32
34
36
38
40
pclk
presetn
psel
penable
pready
pwrite
paddr
pwdata
0x010
0x00
Disable
0x0F0
0x00
Reset
0x108
0x64
Configure
0x100
0x0C
Enable
0x010
0x01
psel
penable
pready
psel
penable
pready
psel
penable
pready
psel
penable
pready
hpet_enable
timer0_int_enable
counter_running
Standard sequence: Disable → Reset → Configure → Enable

Timer Setup Sequence

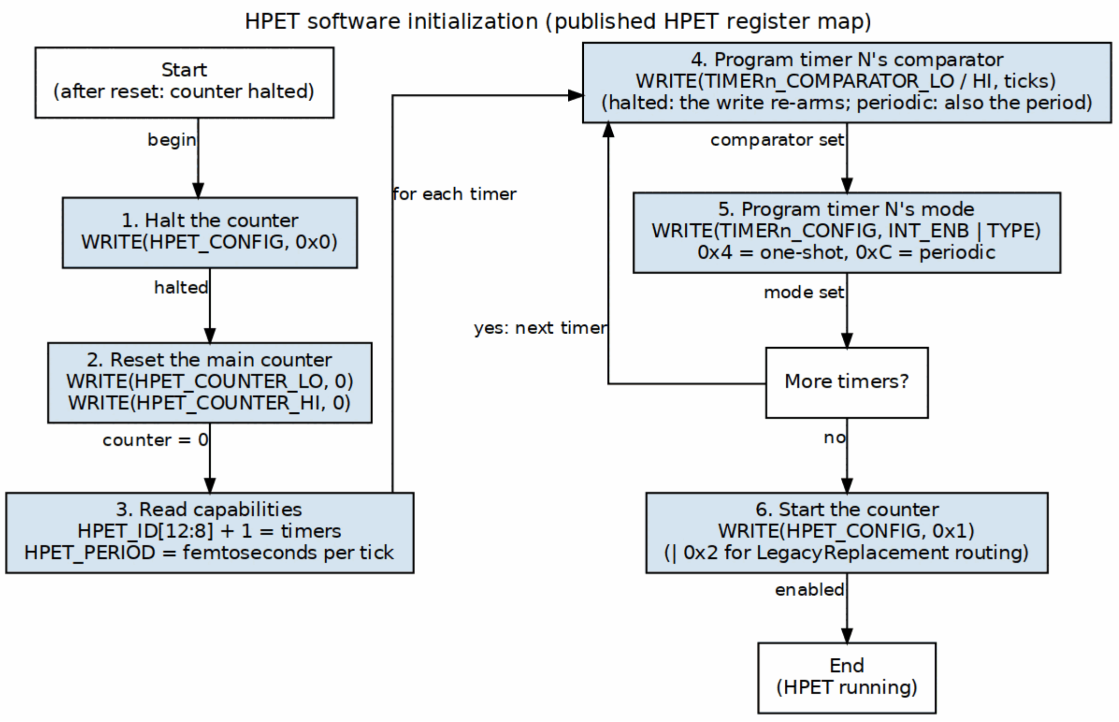
Complete timer setup sequence: disable HPET, reset counter, configure comparator, configure timer (INT_ENB | TYPE), enable HPET. Source: assets/wavedrom/timer_setup_sequence.json

---

14.4 Usage Example

14.4.1 Initialization Sequence

14.4.2 Figure 5.5: Software Initialization Flow


HPET software initialization (published HPET register map)
Start
(after reset: counter halted)
begin
1. Halt the counter
WRITE(HPET_CONFIG, 0x0)
halted
2. Reset the main counter
WRITE(HPET_COUNTER_LO, 0)
WRITE(HPET_COUNTER_HI, 0)
counter = 0
3. Read capabilities
HPET_ID[12:8] + 1 = timers
HPET_PERIOD = femtoseconds per tick
for each timer
4. Program timer N's comparator
WRITE(TIMERn_COMPARATOR_LO / HI, ticks)
(halted: the write re-arms; periodic: also the period)
comparator set
5. Program timer N's mode
WRITE(TIMERn_CONFIG, INT_ENB | TYPE)
0x4 = one-shot, 0xC = periodic
mode set
yes: next timer
More timers?
no
6. Start the counter
WRITE(HPET_CONFIG, 0x1)
(| 0x2 for LegacyReplacement routing)
enabled
End
(HPET running)

Software Initialization Flow

Software initialization sequence showing configuration steps from disable to enable.

```
// 1. Disable HPET
WRITE(HPET_CONFIG, 0x0);

// 2. Reset main counter
WRITE(HPET_COUNTER_LO, 0x0);
WRITE(HPET_COUNTER_HI, 0x0);

// 3. Configure Timer 0 (one-shot, 10ms @ 10MHz)
WRITE(TIMER0_COMPARATOR_LO, 100000);  // 100,000 cycles = 10ms
WRITE(TIMER0_COMPARATOR_HI, 0x0);
WRITE(TIMER0_CONFIG, 0x04);  // INT_ENB; TYPE = 0 (one-shot)

// 4. Configure Timer 1 (periodic, 1ms @ 10MHz)
WRITE(TIMER1_COMPARATOR_LO, 10000);  // 10,000 cycles = 1ms period
WRITE(TIMER1_COMPARATOR_HI, 0x0);
WRITE(TIMER1_CONFIG, 0x0C);  // TYPE (periodic) | INT_ENB
```

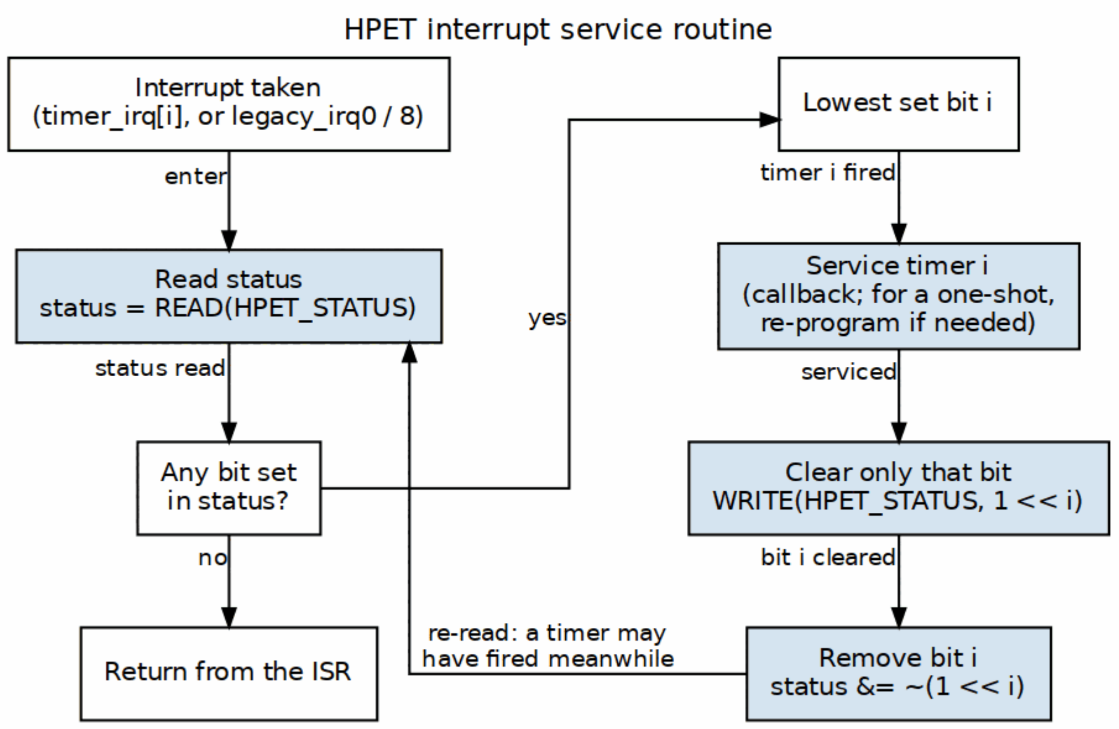
```
// 5. Enable HPET
WRITE(HPET_CONFIG, 0x1);
```

14.4.3 Reading Capabilities

```
// Read identification register
uint32_t id = READ(HPET_ID);

// Extract fields
uint16_t vendor_id = (id >> 16) & 0xFFFF;      // GCAP_ID[31:16]
uint8_t rev_id = id & 0xFF;                     // GCAP_ID[7:0]
uint8_t num_timers = ((id >> 8) & 0x1F) + 1;    // num_tim_cap + 1
uint8_t is_64bit = (id >> 13) & 0x1;            // count_size_cap
uint8_t leg_cap = (id >> 15) & 0x1;             // leg_rt_cap
uint32_t period_fs = READ(HPET_PERIOD);         // femtoseconds per
tick

printf("HPET: Vendor=0x%04X, Rev=%d, Timers=%d, 64-bit=%d\n",
       vendor_id, rev_id, num_timers, is_64bit);
```

14.4.4 Interrupt Handling

14.4.5 Figure 5.6: Interrupt Handling Flow


HPET interrupt service routine
Interrupt taken
(timer_irq[i], or legacy_irq0 / 8)
enter
Read status
status = READ(HPET_STATUS)
status read
Any bit set
in status?
no
Return from the ISR
yes
Lowest set bit i
timer i fired
Service timer i
(callback; for a one-shot,
re-program if needed)
serviced
Clear only that bit
WRITE(HPET_STATUS, 1 << i)
bit i cleared
Remove bit i
status &= ~(1 << i)
re-read: a timer may
have fired meanwhile

Interrupt Handling Flow

Interrupt handling flow showing status check, handler dispatch, and W1C clear sequence.

```
// Generic interrupt handler
void hpet_interrupt_handler(void) {
    // Read status register
    uint32_t status = READ(HPET_STATUS);

    // Check which timers fired
    if (status & (1 << 0)) {
        // Timer 0 fired
        handle_timer0();
        WRITE(HPET_STATUS, (1 << 0));  // Clear Timer 0 interrupt
    }

    if (status & (1 << 1)) {
        // Timer 1 fired
        handle_timer1();
        WRITE(HPET_STATUS, (1 << 1));  // Clear Timer 1 interrupt
    }

    // Each write clears only the bit(s) written with 1; a timer that
    // fires after the READ keeps its bit for the next pass.
}
```

14.5 Related Modules

- Chapter 2: Blocks - Block-level architecture
- Chapters 3 (Interfaces) and 4 (Programming Model) are planned and not yet written – see the index
- PeakRDL Specification - SystemRDL register definition

14.5.1 Additional Diagrams

- Block Diagram - Top-level architecture
- One-Shot Timer - One-shot mode operation
- Periodic Timer - Periodic mode operation
- Software Init - Initialization sequence
- Interrupt Handling - Interrupt flow
- Timer Mode Switch - Mode switching
- Multi-Timer Concurrent - Concurrent operation
- CDC Crossings - Clock domain crossing (CDC_ENABLE=1)

Document Version: 1.1 **Generated:** 2026-09-08 **Based on:** hpet_regs.rdl v2